

pytrf: Python toolbox for the analysis of time series and terrestrial reference frames

- **Python3 toolbox with two main components:**
 - Analysis and modeling of (geodetic) time series (like CATS, Hector, ...)
 - Analysis and combination of terrestrial frame (SINEX) solutions (like CATREF)
- **Motivation:**
 - Missing functionalities in existing time series analysis software
 - Develop modern, flexible software to succeed CATREF for:
 - ITRF combination center
 - IGS terrestrial frame combinations
 - Galileo terrestrial reference frame
- **History:**
 - **2011–2019:** Progressive buildup of python2 code in the frame of IGS reference frame activities
 - **2020:** Organization, complementation, optimization and modernization into python3 package
Creation of (private) github repository
 - **2021:** Extensive use for IGS repro3 SINEX combinations and station position time series analysis ([Rebischung et al., 2024](#))
 - **2021–2025:** Collaborative development within IGN geodesy research team
Transfer to individual researchers & students, as well as to IGN Survey Department, Geoscience Australia & NRCan
 - **2026:** pytrf publicly released under the MIT License
 - **Future:** Looking forward to more collaborative development!

Installation

- Required packages:

- python3, python3-pip, git

- Clone github project

- Create github account and [personal access token](#)

```
$ git clone https://github.com/prebischung/pytrf
```

- Install pytrf

```
$ cd pytrf
```

```
$ pip install [-e] .
```

- This also installs the missing required packages.

(tqdm, pyyaml, platformdirs, numpy, scipy, networkx, astropy, matplotlib, cartopy, pdoc3, geopandas)

Use the `-e` option if you plan on coding within pytrf,
so you won't have to reinstall after every change.

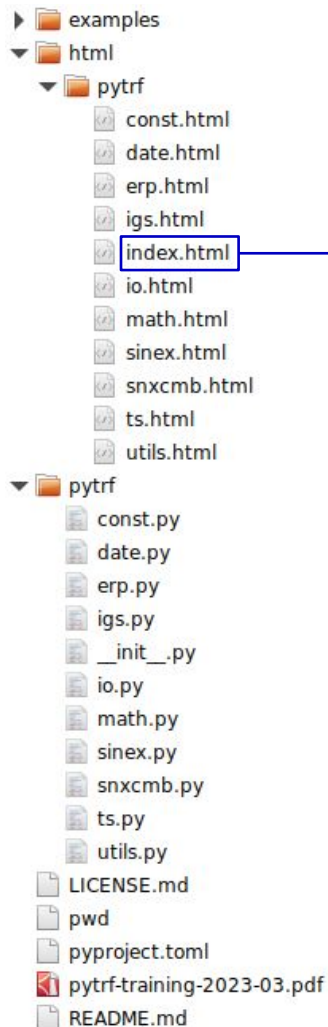
- Generate API documentation

```
$ pdoc --html pytrf
```

- Update with latest version

```
$ git pull
```

- Re-install



Sub-modules

pytrf.const
pytrf.date
pytrf.erp
pytrf.igs
pytrf.io
pytrf.math
pytrf.sinex
pytrf.snxcmb
pytrf.ts
pytrf.utils

The 3 main pytrf modules:

Package **pytrf**

pytrf

Python toolbox for the analysis of time series of terrestrial reference frames

This module imports the following classes and modules into the main namespace:

date : Class for handling dates in various formats
sinex : Class for reading, writing and manipulating SINEX files
erp : Class for reading, writing and manipulating ERP files

const : pytrf constants
utils : pytrf miscellaneous utilities
math : pytrf math utilities
io : pytrf input/output utilities
igs : pytrf IGS utilities
ts : pytrf time series utilities
snxcmb : SINEX combination utilities

Sub-modules

pytrf.const

pytrf constants ...

pytrf.date

pytrf class for handling dates in various formats

pytrf.erp

pytrf class for reading, writing and manipulating ERP files

pytrf.date module – Date objects (date class)

```
class date (*args)
```

Class for handling dates in various formats

A date instance can be initialized in one of the following ways:

```
t = date()
t = date(tsys)
t = date.from_mjd(mjd)
t = date.from_ymdhms(year, month, day, [hour, minute, second])
t = date.from_wd(week, dow)
t = date.from_tsnx(tsnx)
t = date.from_tiso(tiso)
t = date.from_ydec(ydec)
t = date.from_wdec(wdec)
```

See the documentation of each constructor method for details.

Once initialized, each date instance has the following attributes:

```
tsys : System time
mjd  : Modified Julian Day
yyyy : 4-char year
yy   : 2-char year
mm   : 2-char month
dd   : 2-char day of month
hour : 2-char hour
min  : 2-char minute
sec  : 2-char second
doy  : 3-char day of year
week : 4-char GPS week
dow  : 1-char day of week
wk   : 2-char week of year
```

Each date instance has the following methods:

```
__str__ : Print date in 'YYYY-MM-DD hh:mm:ss' format
add_s(n) : Add n seconds to instance
add_h(n) : Add n hours to instance
add_d(n) : Add n days to instance
ydec()   : Return decimal year
wdec()   : Return decimal GPS week
tsnx()   : Return date in SINEX date format ('yy:ddd:sssss')
tiso()   : Return date in ISO format ('YYYY-MM-DDThh:mm:ss')
```

```
$ from pytrf import date
```

```
$ t = date()
```

```
$ print(t)
```

```
> 2023-01-30 08:34:56
```

```
$ t = date.from_mjd(51544.5)
```

```
$ print(t)
```

```
> 2000-01-01 12:00:00
```

```
$ t = date.from_ymdhms(2023, 1, 1, 12)
```

```
$ t.mjd
```

```
> 59945.5
```

```
$ import numpy as np
```

```
$ mjd = np.arange(51544.5, 59945.5)
```

```
$ y = [date.from_mjd(d).ydec() for d in mjd]
```

```
$ print(y)
```

```
> [2000.00136612 2000.00409836 ... 2022.99863014]
```

Part 1 – Time series analysis:

`pytrf.ts` module

pytrf.ts – Philosophy

- Read some time series, i.e.:

```
$ r = ts.read(...)
```

- Dates: $t = [t_1, t_2, \dots, t_n]$
- Values: $y = [y_1, y_2, \dots, y_n]$
- Formal errors: $\sigma = [\sigma_1, \sigma_2, \dots, \sigma_n]$ (optional)
- Integration intervals: $T = [T_1, T_2, \dots, T_n]$ (optional)

- Specify a model: $y \sim \mathcal{N}(f(x), Q(\beta))$

```
$ m = model(r, ...)
```



- 'Fit' the model to the time series, i.e., estimate:

```
$ m.fit()
```

- Vector of deterministic parameters x
- Vector of stochastic parameters β

- Compare different models using model selection criteria

- Ultimate goal:

- Obtain a realistic model of the time series
- Obtain realistic uncertainties on signals extracted from the time series (e.g., its trend)

pytrf.ts – Time series objects (ts class)

```
class ts (*args, t=None, T=None, y=None, Q=None, tunit='d', yunit='m', dims=None, rotate=False, t0=None, dtrd=None)
```

pytrf time series class

A ts instance is initialized in one of the following ways:

```
dat = ts()
dat = ts.read(text_file)
dat = ts.load(pickle_file)
```

Once initialized, each ts instance has the following attributes:

```
nd      : Number of components (e.g., 3 for station position time series)
n       : Length
t       : Dates (n,)
T       : Integration interval(s)
y       : Values (n, nd)
Q       : Formal covariance matrices (n, nd, nd)
tunit   : Time unit
yunit   : Series unit
dims    : Component names (e.g., ['East', 'North', 'Up'])
R       : XYZ -> ENH rotation matrix (in case of a series given in
          geocentric coordinates and rotated into topocentric coordinates)
dtrd    : Degree of detrending polynomial
ctrd    : Coefficients of detrending polynomial (dtrd+1, nd)
t0      : Origin of time used for detrending
ndel    : Number of detected outliers
tdel    : Dates of detected outliers (ndel,)
Tdel    : Integration intervals of detected outliers (ndel,)
ydel    : Values of detected outliers (ndel, nd)
Qdel    : Covariance matrices of detected outliers (ndel, nd, nd)
```

Each ts instance has the following methods:

```
__len__()      : Get number of components of ts instance
__getitem__()  : Get specific component of ts instance
dump()         : Dump ts instance into pickle file
trim()         : Trim ts instance down to period of interest
detrend()      : (Re-)detrend ts instance
del_points()   : Flag outliers
clean_sigmas() : Flag observations with large formal errors as outliers
plot()         : Plot time series
```

- A simple container for time series data
- Supports multi-dimensional time series (e.g., 3D station position time series)

• Main attributes:

- **nd** number of dimensions
- **n** length
- **t** array of dates (**n**,)
- **y** array of values (**n**,**nd**)
- **Q** formal covariance matrices (**n**,**nd**,**nd**)
- **tunit**, **yunit** units of dates and values (e.g., 'd', 'm')
- **dims** dimension labels (e.g., ('E', 'N', 'H'))

• Some basic utilities:

- **slicing** **r[d]** returns a 1D **ts** object
- **trim()** restrict time series to period of interest
- **detrend()** remove (and store) linear trend
- **del_points()** flag specified points as outliers
- **clean_sigmas()** flag points with large sigmas as outliers
- **clean_outliers()** automatic outlier detection
- **plot()** plot time series

NEW

pytrf.ts – Examples of ts object construction

```
$ from pytrf.ts import ts
```

```
$ r = ts(100)
```

1D time series of length 100 with zero values

```
$ r = ts((100, 3))
```

3D time series of length 100 with zero values

```
$ import numpy as np
```

```
$ y = np.random.randn(100, 3)
```

```
$ r = ts(y)
```

3D time series from numpy array of values

```
$ t = np.arange(100)
```

```
$ y = np.random.randn(100, 3)
```

```
$ r = ts(t=t, y=y)
```

3D time series from numpy arrays of dates and values

```
$ t = np.arange(100)
```

```
$ y = np.random.randn(100)
```

```
$ s = np.ones(100)
```

```
$ r = ts(t=t, y=y, Q=s**2)
```

1D time series from numpy arrays of dates, values and sigmas

```
$ r = ts(t=t, y=y, Q=s**2, tunit='s', yunit='V', dims=('voltage'))
```

```
$ r.plot()
```

pytrf.ts – Examples of ts object construction

```
$ from pytrf.ts import ts
```

```
$ r = ts(100)
```

```
$ r = ts((100, 3))
```

```
$ import numpy as np
```

```
$ y = np.random.randn(100, 3)
```

```
$ r = ts(y)
```

```
$ t = np.arange(100)
```

```
$ y = np.random.randn(100, 3)
```

```
$ r = ts(t=t, y=y)
```

```
$ t = np.arange(100)
```

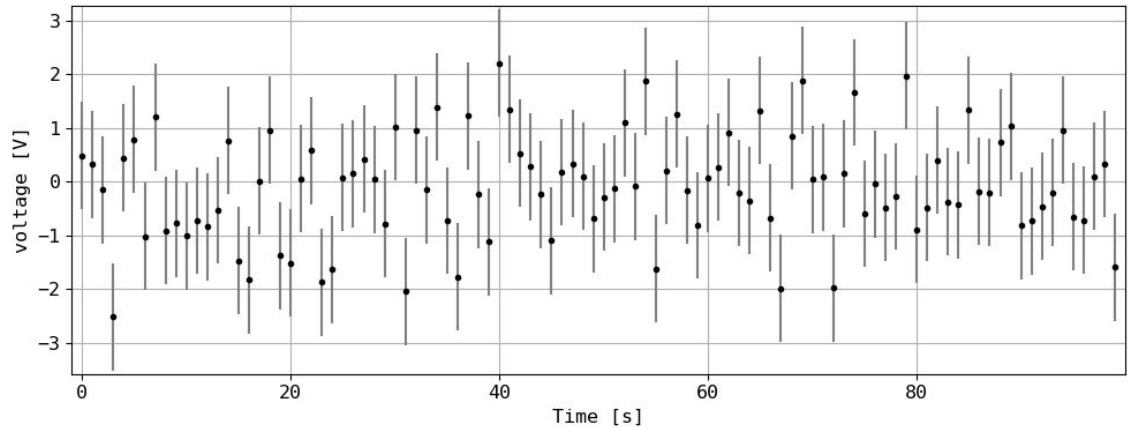
```
$ y = np.random.randn(100)
```

```
$ s = np.ones(100)
```

```
$ r = ts(t=t, y=y, Q=s**2)
```

```
$ r = ts(t=t, y=y, Q=s**2, tunit='s', yunit='V', dims=('voltage'))
```

```
$ r.plot()
```



pytrf.ts – Construct ts object from data file: ts.read()

- Example 1: NGL station position time series

File IGUA.tenv3:

```
site YMMDD yyyy.yyyy __MJD week d reflon _e0(m) __east(m) ____n0(m) _north(m) u0(m) ____up(m) _ant(m) sig_e(m) sig_n(m) sig_u(m)
IGUA 01JAN01 2001.0021 51910 1095 1 -99.5 -252 -0.023105 2034367 0.383471 924 0.831862 0.0000 0.001043 0.000881 0.004584
IGUA 01JAN02 2001.0048 51911 1095 2 -99.5 -252 -0.021988 2034367 0.381620 924 0.838939 0.0000 0.001068 0.000907 0.004764
IGUA 01JAN03 2001.0075 51912 1095 3 -99.5 -252 -0.020196 2034367 0.380511 924 0.839956 0.0000 0.001083 0.000905 0.004736
IGUA 01JAN04 2001.0103 51913 1095 4 -99.5 -252 -0.021100 2034367 0.382354 924 0.838035 0.0000 0.001093 0.000921 0.004677
IGUA 01JAN05 2001.0130 51914 1095 5 -99.5 -252 -0.021007 2034367 0.381116 924 0.841144 0.0000 0.001072 0.000889 0.004586
...
```

```
$ from pytrf.ts import ts
```

```
$ r = ts.read('IGUA.tenv3', skiprows=1,
              usecols=(3, 8, 10, 12, 14, 15, 16),
              format=('t', 'x', 'y', 'z', 'sx', 'sy', 'sz'),
              tunit='d', yunit='m',
              dims=('East', 'North', 'Up'),
              dtrd=1)
```

```
$ r.plot(tunit='y')
```

Skip header line

Columns to read

What's in the columns?

Units

Dimension labels

Remove (and store) linear trend

- Notes:

- When possible, prefer the use of regularly spaced dates. In this case: MJDs rather than decimal years.
- If `r.t` is in MJDs, `r.plot(tunit='y')` can be used to show a plot in decimal years.

pytrf.ts – Construct ts object from data file: ts.read()

- Example 1: NGL station position time series

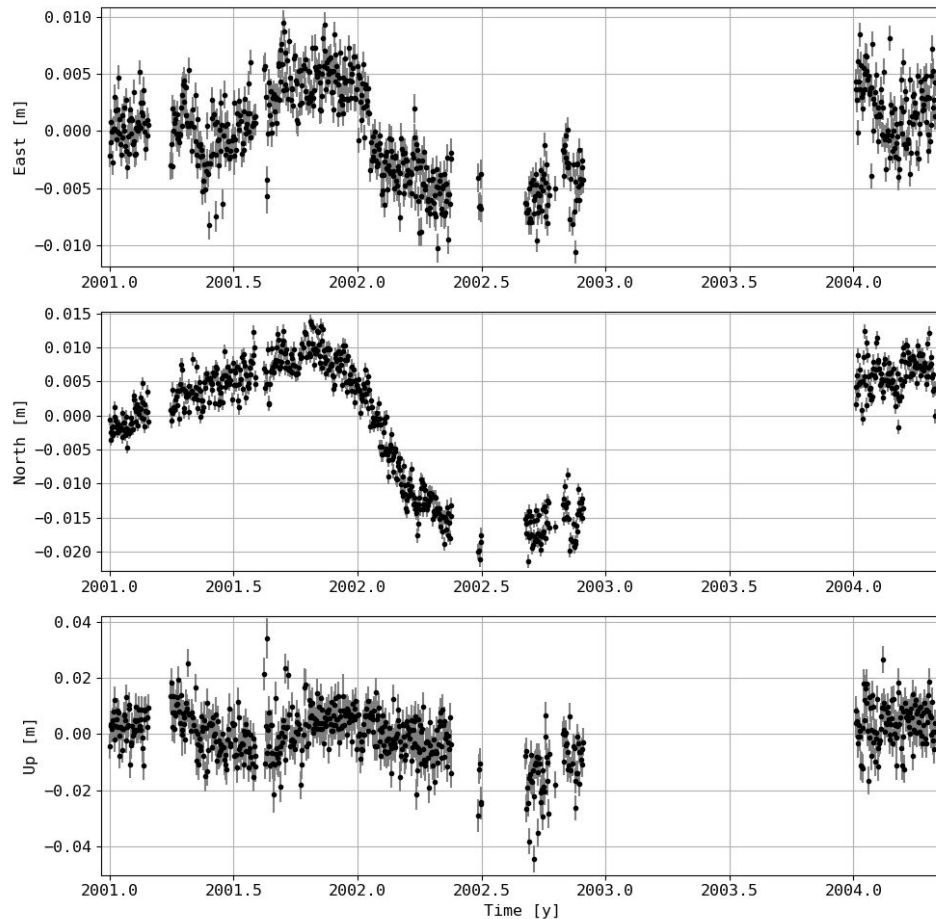
File [IGUA.tenv3](#):

```
site YMMDD yyyy.yyyy __MJD week d reflon _e0(m) __east(m)
IGUA 01JAN01 2001.0021 51910 1095 1 -99.5 -252 -0.023105
IGUA 01JAN02 2001.0048 51911 1095 2 -99.5 -252 -0.021988
IGUA 01JAN03 2001.0075 51912 1095 3 -99.5 -252 -0.020196
IGUA 01JAN04 2001.0103 51913 1095 4 -99.5 -252 -0.021100
IGUA 01JAN05 2001.0130 51914 1095 5 -99.5 -252 -0.021007
...
```

```
$ from pytrf.ts import ts
$ r = ts.read('IGUA.tenv3', skiprows=1,
              usecols=(3, 8, 10, 12, 14),
              format=('t', 'x', 'y', 'z'),
              tunit='d', yunit='m',
              dims=('East', 'North', 'Up'),
              dtrd=1)
$ r.plot(tunit='y')
```

- Notes:

- When possible, prefer the use of [regularly spaced data](#)
- If `r.t` is in MJDs, `r.plot(tunit='y')` can be used



pytrf.ts – Construct ts object from data file: ts.read()

- Example 2: JPL daily station position time series

File [RIOP.series](#):

```
1996.44186345  0.000000  0.000000  0.000000  0.001534  0.001398  0.006768 -0.056874 -0.396217  0.135094 -112286250.00 1996  6 10 21 22 30
1996.44352707  0.006897 -0.001001  0.000581  0.000929  0.000820  0.003990  0.062801 -0.362029  0.060964 -112233750.00 1996  6 11 11 57 30
1996.44626493  0.008950 -0.002451 -0.003403  0.001025  0.000894  0.004346  0.042462 -0.461615  0.070431 -112147350.00 1996  6 12 11 57 30
1996.44917389  0.007931  0.000441  0.002158  0.000931  0.000828  0.004017  0.011214 -0.021647  0.069317 -112055550.00 1996  6 13 13 27 30
...
```

```
$ from pytrf.ts import ts
$ r = ts.read('RIOP.series', usecols=(10, 1, 2, 3, 4, 5, 6),
              format=('t', 'x', 'y', 'z', 'sx', 'sy', 'sz'),
              tunit='d', yunit='m', dims=('East', 'North', 'Up'))
$ import numpy as np; from pytrf import date
$ r.t = date.from_ymdhms(2000, 1, 1, 12).mjd + r.t/86400
$ r.t = np.round(r.t+0.5) - 0.5
$ r.T = 1
$ ind = np.where(np.diff(r.t) == 0)[0]
$ r.del_points(ind)
```

Columns to read?

What's in the columns?

Units & dimension labels

Seconds since J2000 → MJDs

Round MJDs to nearest noon

Set integration interval to 1 day

Deal with duplicate dates

- Notes:

- When possible, try to get back to regularly spaced dates. In this case: MJDs at noon.
- Model fitting will fail in case of duplicate dates.

pytrf.ts – Construct ts object from data file: ts.read()

- Example 3: IGS daily station position time series

File [CKSV_igs.xyz](#):

```
1669 0 55927.5 2012-01-01T12:00:00 -2.95661922594522e+06 5.07590215464785e+06 2.47662558357152e+06 1.80283e-03 2.41290e-03 1.41944e-03 -7.43317e-01 -6.19305e-01 6.97584e-01
1669 1 55928.5 2012-01-02T12:00:00 -2.95661922657225e+06 5.07590215840779e+06 2.47662558381471e+06 1.53790e-03 2.19454e-03 1.25092e-03 -7.64861e-01 -6.46890e-01 7.42791e-01
1669 2 55929.5 2012-01-03T12:00:00 -2.95661922535649e+06 5.07590215596657e+06 2.47662558240214e+06 1.53928e-03 2.18276e-03 1.23370e-03 -7.64436e-01 -6.42093e-01 7.45103e-01
1669 3 55930.5 2012-01-04T12:00:00 -2.95661922887771e+06 5.07590215742212e+06 2.47662558463411e+06 2.01915e-03 2.62643e-03 1.67429e-03 -7.26278e-01 -6.92189e-01 6.98270e-01
1669 4 55931.5 2012-01-05T12:00:00 -2.95661922656351e+06 5.07590215508118e+06 2.47662557979883e+06 1.90349e-03 2.51647e-03 1.56632e-03 -7.27054e-01 -6.47637e-01 6.97179e-01
1669 5 55932.5 2012-01-06T12:00:00 -2.95661922382112e+06 5.07590215072674e+06 2.47662558306583e+06 1.87088e-03 2.74883e-03 1.51947e-03 -7.85782e-01 -6.71061e-01 7.46552e-01
...
```

```
$ from pytrf.ts import ts
$ r = ts.read('CKSV_igs.xyz', usecols=(2, 4, 5, 6, 7, 8, 9, 10, 11, 12),
             format=('t', 'x', 'y', 'z', 'sx', 'sy', 'sz', 'cxy', 'cxz', 'cyz'),
             dtrd=1,          Remove (and store) linear trend
             rotate=True)    Rotate time series from geocentric (XYZ) to topocentric (ENH) frame
```

- Notes:

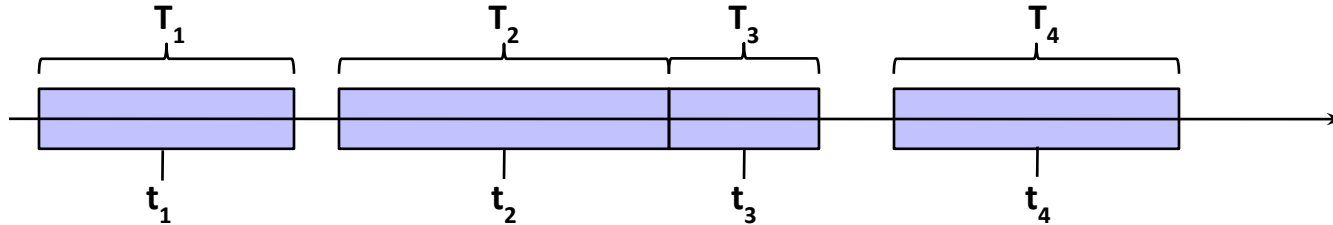
- Correlations between components are not used when adjusting models to time series.
- In this case however, it is necessary to read them, in order to compute the ENH formal errors rigorously.
- With `rotate=True`, `r.dims` is automatically set to `('East', 'North', 'Up')`.

- See more examples in:

[pytrf/examples/time series analysis/1 - reading GNSS time series in different formats](#)

pytrf.ts – More on dates and integration intervals

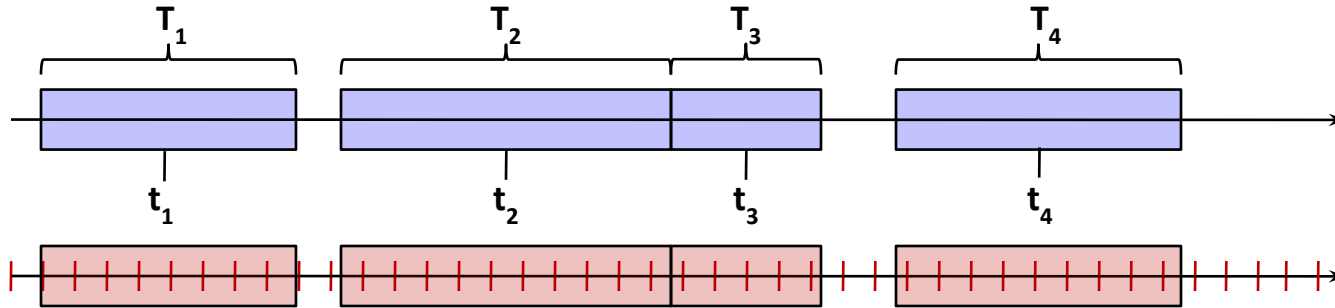
- Each observation is assumed to be 'integrated' over some time interval:



- Nominal integration interval of GNSS station position time series is (usually) 1 day.
- Even though actual integration intervals may differ, e.g., due to missing observations, it is preferable to assume a single nominal integration interval when modeling the noise in GNSS time series (see why in next slide).
- Other types of time series do not have a single nominal integration interval (e.g, 15-day then 7-day SLR time series).
- In such cases, it is necessary to account for the different integration intervals when modeling the noise in the time series.
- If not specified when constructing a `ts` object, `r.T` is assumed to be the same for all observations and set to the smallest time interval between two observations: `r.T = min(diff(r.t))`
- This works with NGL time series, which have dates given as whole MJDs.
- But this does not work with JPL time series, which is why `r.T` was manually set to 1 day in slide 12.
- This would not work either with 15-day then 7-day SLR time series, and `r.T` should be manually set to some array of values.

pytrf.ts – More on dates and integration intervals

- Noise processes in `pytrf.ts` are discrete, with a custom time step.
 - By default, noise process time step = minimum integration interval of observations.
- Noise processes are averaged over the same integration intervals as the observations.



- A fine time step can be used to provide a faithful representation of the integration intervals...
 - In case of 15-day, then 7-day SLR time series, use a 1-day time step for the noise processes.
- ... in the limit of available computing resources.
 - Modeling a T -long time series with a dt -sampled noise process will temporarily require $8 \times (T/dt)^2$ bytes of RAM.
 - Computation time also increases with smaller time step for the noise processes.
 - In case of GNSS time series, better use a nominal 1-day time step for both the time series and the noise processes.

• **Always check integration intervals and noise time steps before fitting a model to a time series!**

pytrf.ts – Model objects (model class)

```
class model(r, t0=None, deg=None, per=None, noise=None)
```

Class for deterministic+noise models adjusted to time series

A model instance is initialized in one of the following ways:

```
m = model()
m = model.from_solns()
m = model.load()
```

Once initialized, each model instance has the following attribute:

```
nd : Number of dimensions
```

If $nd > 1$, then the model instance has the following attributes

```
r : The corresponding time series
t0 : The origin of time
md : List of 1-dimensional model instances
```

A 1-dimensional model instance (either m if $nd = 1$, or any $m.md[i]$ if $nd > 1$) has the following attributes

```
r : The corresponding time series
f : List of functions
n : List of noise components
```

Once (a priori or estimated) values have been assigned to the unknown deterministic parameters of a 1-dimensional model, its following attributes (initialized to None) may be set by calling `set_oeq()`:

```
yc : Computed observations
A : Design matrix
```

Once (a priori or estimated) values have been assigned to the unknown noise parameters of a 1-dimensional model, its following attributes (initialized to None) may be set by calling `set_cov()`:

```
Q : Covariance matrix
L : Cholesky factorization of Q
```

...

- A **model** object encodes a model ($f(x)$, $Q(\beta)$) [to be] adjusted to a given time series.
 - It is attached to particular time series: **`m = model(r, ...)`**
 - A **model** object attached to an nD time series is basically a list of n 1D **model** objects.
- When initialized, a 1D **model** object essentially consists of:
 - A list of deterministic **function** objects: **`m.f`**
 - A list of **noise** component objects: **`m.n`**
- After calling **`m.fit()`**, a lot of other attributes are set up, including:
 - Estimated values of deterministic and noise parameters: **$(\hat{x}, \hat{\beta})$**
 - Covariance matrices of deterministic and noise parameters
 - Adjusted deterministic model: **$f(\hat{x})$**
 - Covariance matrix of adjusted noise model: **$Q(\hat{\beta})$**
 - Power spectra of residuals and noise model
 - ... (more in next slides)

pytrf.ts – Deterministic functions

- The deterministic model $f(x)$ of a time series is encoded in a list of **function objects**: `m.f`
- Six **function sub-classes** are implemented: `polynom`, `sine`, `poisson`, `fexp`, `flog`, `dirac`.
 - More custom sub-classes can easily be implemented upon request.
- Each function f_k depends on a set of parameters x_k , whose values can either be specified or estimated from the time series.
 - e.g., the amplitude and relaxation time of an exponential transient.
- Once a priori or estimated values are attributed to its parameters x_k , a function f_k can be evaluated at the dates of the time series $\rightarrow f_k(x_k)(t)$
- The deterministic model $f(x)$ of a time series is actually a sum over the **function objects** in `m.f`:
$$f(x) = \sum f_k(x_k)$$



pytrf.ts – Deterministic functions

- The deterministic model $f(x)$ of a time series is encoded in a list of **function objects**: `m.f`
- Six **function sub-classes** are implemented: `polynom`, `sine`, `poisson`, `fexp`, `flog`, `dirac`.
 - More custom sub-classes can easily be implemented (see `example3.py`).

- **polynom** = piecewise polynomial function of time of a unique specified degree

– `polynom(deg=d, t=tc)` encodes the function:

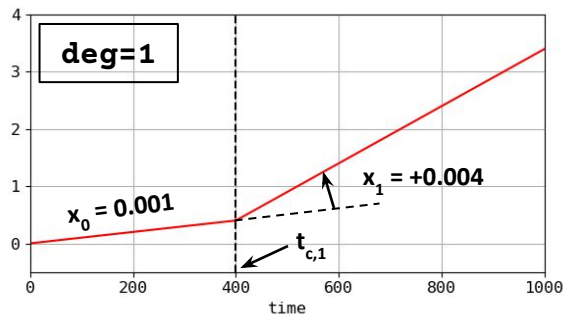
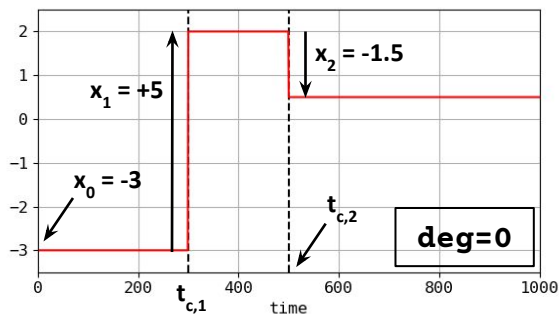
$$f(x)(t) = x_0 \frac{(t-t_0)^d}{d!} + \sum_{i=1}^{n_c} x_i \frac{(t-t_{c,i})^d}{d!} \times H(t-t_{c,i})$$

single degree

list of change dates
($t_{c,1}, \dots, t_{c,n_c}$)

Heaviside function

- x_0 represents a mean offset / trend / acceleration... at t_0 (the beginning of the time series.)
- Each x_i represents a **change** in the mean offset / trend / acceleration... at time $t_{c,i}$.

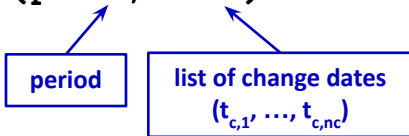


pytrf.ts – Deterministic functions

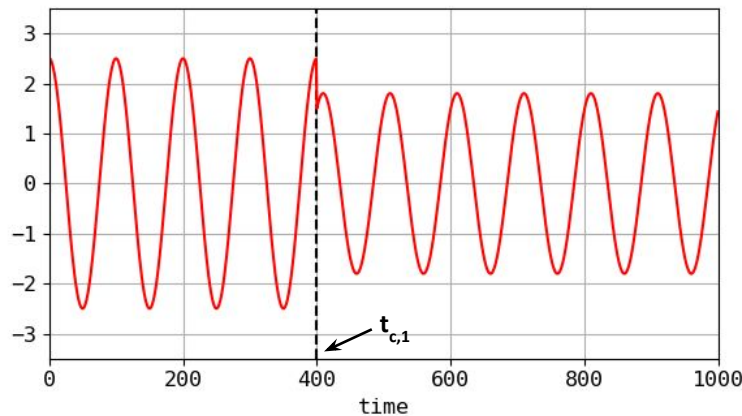
- The deterministic model $f(x)$ of a time series is encoded in a list of **function objects**: `m.f`
- Six **function sub-classes** are implemented: `polynom`, `sine`, `poisson`, `fexp`, `flog`, `dirac`.
 - More custom sub-classes can easily be implemented (see `example3.py`).

- **sine = piecewise sine wave of specified period**

– `sine(per=T, t=tc)` encodes the function: $f(x)(t) = x_0 \cos(2\pi(t - t_0)/T) + x_1 \sin(2\pi(t - t_0)/T)$



$$+ \sum_{i=1}^{n_c} [x_{2i} \cos(2\pi(t - t_0)/T) + x_{2i+1} \sin(2\pi(t - t_0)/T)] \times H(t - t_{c,i})$$

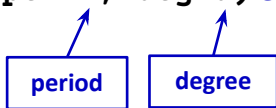


- x_0, x_1 are the cosine and sine coefficients at the beginning of the series.
- x_{2i}, x_{2i+1} represent **changes** in the cosine and sine coefficients at time $t_{c,i}$.
- In this case, t_0 is not necessarily the beginning of the series. It can be specified by setting `m.t0` to any custom value.

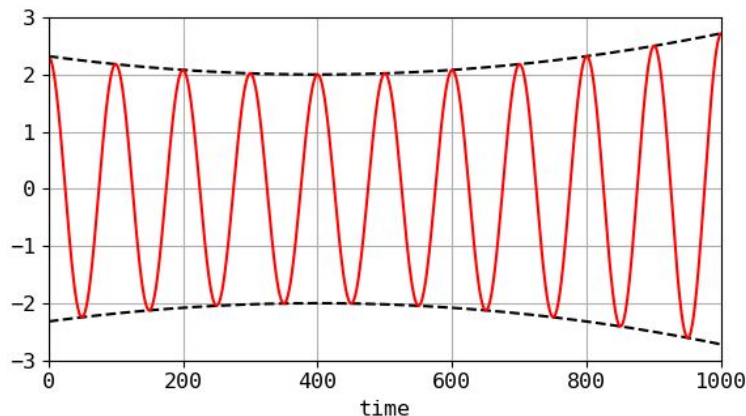
pytrf.ts – Deterministic functions

- The deterministic model $f(x)$ of a time series is encoded in a list of **function objects**: `m.f`
- Six **function sub-classes** are implemented: `polynom`, `sine`, `poisson`, `fexp`, `flog`, `dirac`.
 - More custom sub-classes can easily be implemented (see `example3.py`).
- **`poisson` = sine wave of specified period with polynomially-varying coefficients**

– `poisson(per=T, deg=d)` encodes the function:



$$f(x)(t) = \left(\sum_{k=0}^d x_{2k} (t - t_0)^k \right) \cos(2\pi(t - t_0)/T) + \left(\sum_{k=0}^d x_{2k+1} (t - t_0)^k \right) \sin(2\pi(t - t_0)/T)$$



- Like for **sine** functions, t_0 can be specified by setting `m.t0` to any custom value.

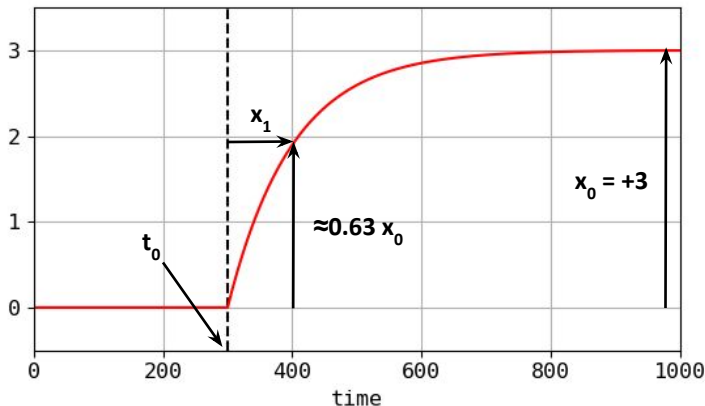
pytrf.ts – Deterministic functions

- The deterministic model $f(x)$ of a time series is encoded in a list of **function objects**: `m.f`
- Six **function sub-classes** are implemented: `polynom`, `sine`, `poisson`, `fexp`, `flog`, `dirac`.
 - More custom sub-classes can easily be implemented (see `example3.py`).

- **fexp = exponential transient**

- `fexp(t0=t0)` encodes the function: $f(x)(t) = x_0 [1 - \exp(-(t - t_0)/x_1)]$

start time

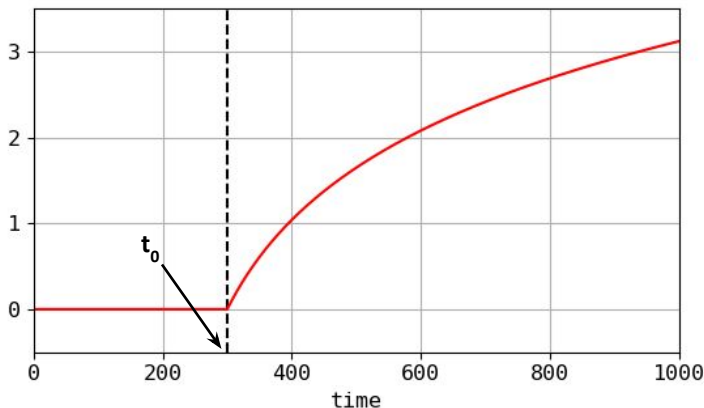


- x_0 is the amplitude of the transient.
 - x_1 is the 'characteristic', 'relaxation', or 'decay' time, after which $\approx 63\%$ of the amplitude is reached.

pytrf.ts – Deterministic functions

- The deterministic model $f(x)$ of a time series is encoded in a list of **function objects**: `m.f`
- Six **function sub-classes** are implemented: `polynom`, `sine`, `poisson`, `fexp`, `flog`, `dirac`.
 - More custom sub-classes can easily be implemented (see `example3.py`).
- **flog = logarithmic transient**
 - `flog(t0=t0)` encodes the function: $f(x)(t) = x_0 \log(1 + (t - t_0)/x_1)$

start time



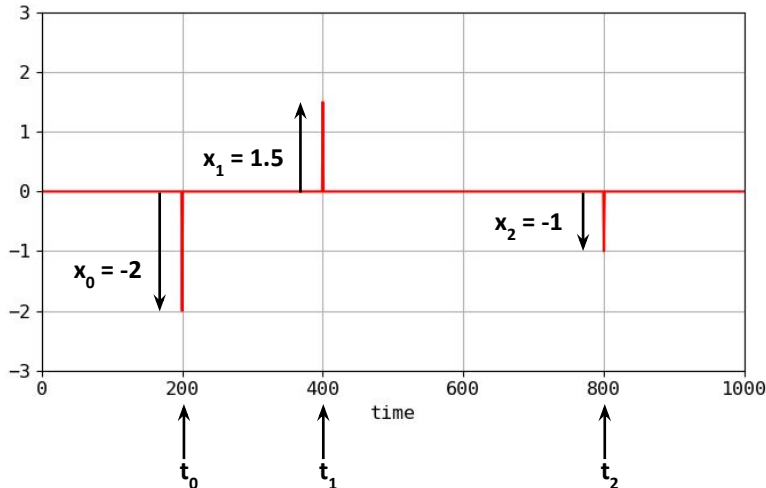
- Unlike an exponential transient, a logarithmic transient never reaches a limit.
- x_0 and x_1 do not have as clear interpretations as in the exponential case, but still describe the 'size' and 'speed' of the transient.

pytrf.ts – Deterministic functions

- The deterministic model $f(\mathbf{x})$ of a time series is encoded in a list of **function objects**: `m.f`
- Six **function sub-classes** are implemented: `polynom`, `sine`, `poisson`, `fexp`, `flog`, `dirac`.
 - More custom sub-classes can easily be implemented (see `example3.py`).

NEW `dirac` = sum of Dirac functions at specified dates

- `dirac(t=[t0, ..., tn])` encodes the function:
$$f(\mathbf{x})(t) = \sum_{i=0}^n \mathbf{x}_i \delta_{t,t_i}$$



- `dirac` functions provide a way to deal with outliers without rejecting them from the time series.

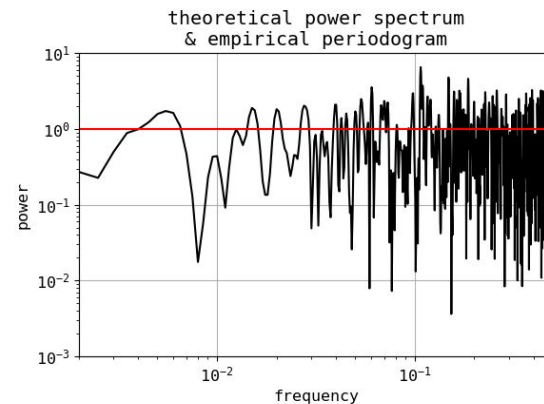
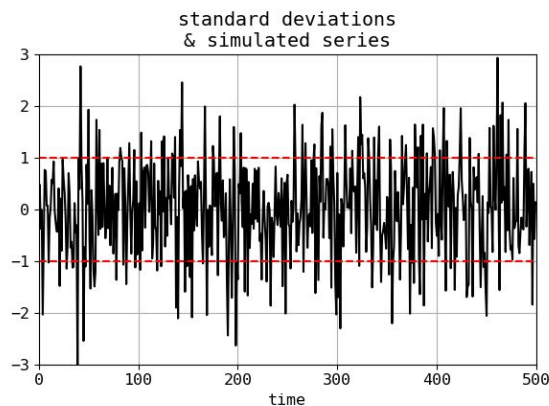
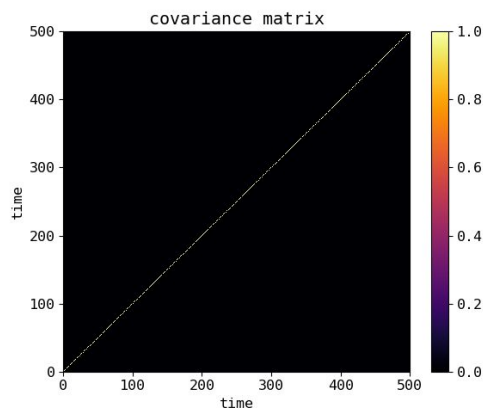
pytrf.ts – Noise components

- The stochastic model $Q(\beta)$ of a time series is encoded in a list of **noise** objects: **m.n**
- Six **noise** sub-classes are implemented: **wn**, **vw**, **ar1**, **pl**, **ggm**, **figgm**.
 - More custom sub-classes can be implemented upon request.
- The covariance matrix of a noise component n_k depends on a set of parameters β_k , whose values can either be specified or estimated from the time series.
 - e.g., the variance and spectral index of a power-law process.
- Once a priori or estimated values are attributed to its parameters β_k , the covariance matrix of a noise component n_k can be evaluated at the dates of the time series
→ $Q_k(\beta_k)$
- The stochastic model $Q(\beta)$ of a time series is actually a sum over the **noise** objects in **m.n**:
$$Q(\beta) = \sum Q_k(\beta_k)$$

NEW

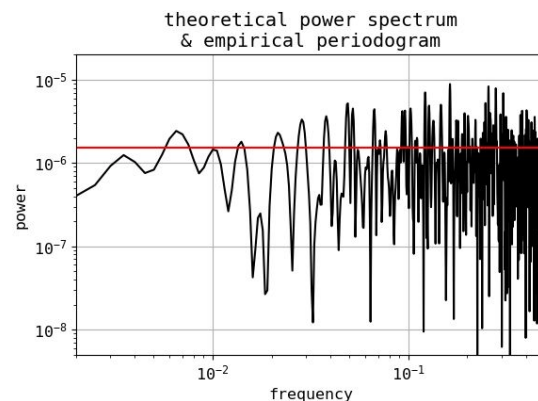
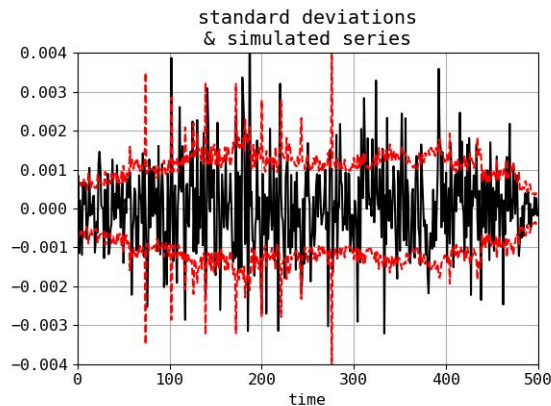
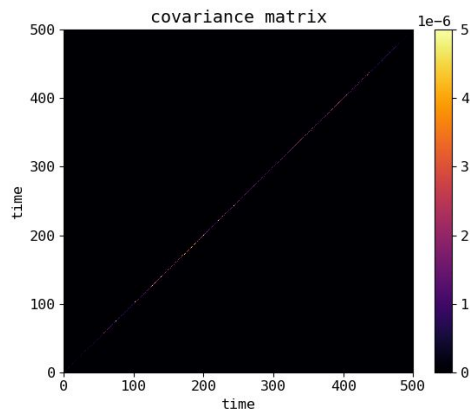
pytrf.ts – Noise components

- The stochastic model $Q(\beta)$ of a time series is encoded in a list of **noise** objects: **m.n**
- Six **noise** sub-classes are implemented: **wn**, **vw**, **ar1**, **pl**, **ggm**, **figgm**.
 - More custom sub-classes can be implemented upon request.
- **wn** = **homogeneous white noise (independent, identically distributed errors)**
 - **wn(dt=dt)** encodes a white noise process with time step **dt**.
 - Before averaging over the same integration intervals as the observations, its covariance matrix is: $Q_k(\beta) = \beta_0 \times \mathbf{I}$
 - After averaging, the covariance matrix remains diagonal, but in case of varying integration intervals, it may not be homoscedastic anymore.
 - The only parameter β_0 is the variance of the process.



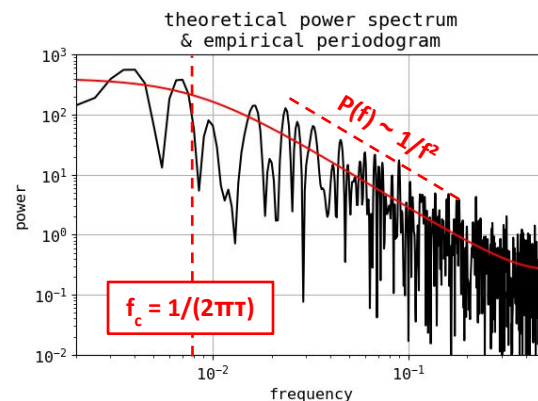
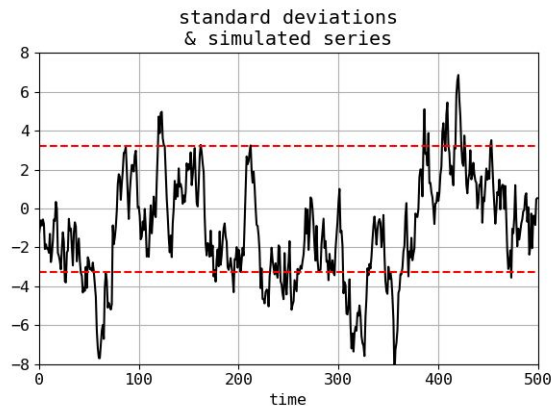
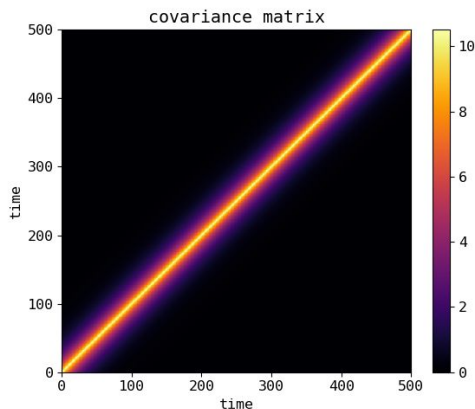
pytrf.ts – Noise components

- The stochastic model $Q(\beta)$ of a time series is encoded in a list of **noise** objects: **m.n**
- Six **noise** sub-classes are implemented: **wn**, **vw**, **ar1**, **pl**, **ggm**, **figgm**.
 - More custom sub-classes can be implemented upon request.
- **vw** = **variable white noise** (independent errors proportional to formal errors)
 - The corresponding covariance matrix is: $Q_k(\beta) = \beta_0 \times \text{diag}(\sigma_1^2, \sigma_2^2, \dots, \sigma_n^2)$
 - $\sigma_1, \sigma_2, \dots, \sigma_n$ are formal errors provided with the time series (required).
 - The only parameter β_0 is a corrective variance factor.
 - **vw** processes are the only noise components that are not constructed with a regular sampling **dt**, then averaged over the same integration intervals as the observations.



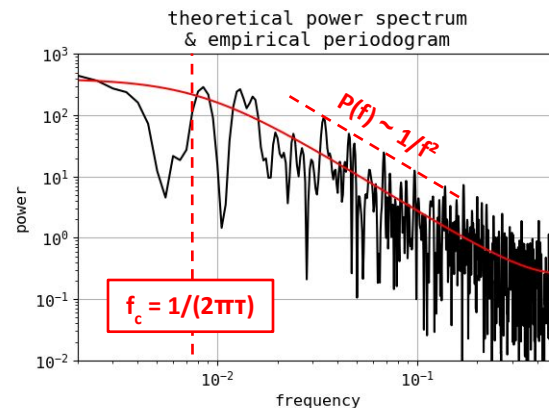
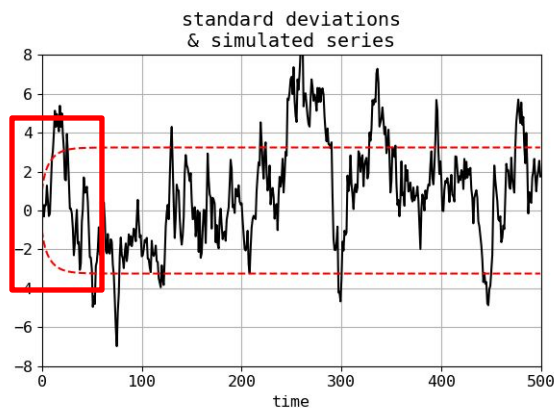
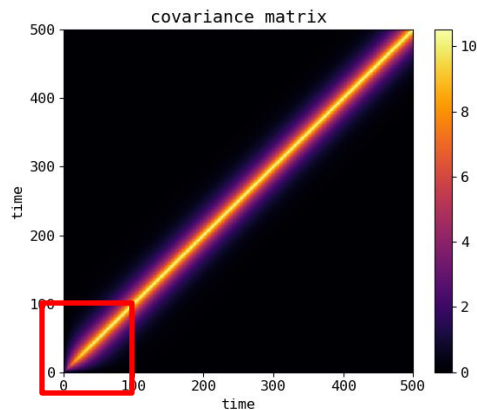
pytrf.ts – Noise components

- The stochastic model $Q(\beta)$ of a time series is encoded in a list of **noise** objects: **m.n**
- Six **noise** sub-classes are implemented: **wn**, **vw**, **ar1**, **pl**, **ggm**, **figgm**.
- **ar1** = first-order auto-regressive process (AR(1))
 - **ar1(dt=dt, flow='stationary')** encodes a stationary AR(1) process with time step **dt**.
 - An AR(1) process $x(t)$ is obtained from a WN process $w(t)$ by: $x(t) = \varphi \cdot x(t-dt) + w(t)$ with $0 < \varphi < 1$
 - It has two parameters: β_0 , the variance of the underlying WN process; $\beta_1 = \tau = -dt / \log(\varphi)$, a characteristic correlation time.
 - 'Stationary' means that the process is assumed to extend infinitely in time.
 - Before averaging, its covariance matrix is Toeplitz and such that: $Q_k(\beta)(t, t+j \cdot dt) = \beta_0 \cdot \varphi^j / (1 - \varphi^2) \sim \exp(-j \cdot dt / \tau)$
 - Its autocorrelation decays exponentially (with decay time τ) $\rightarrow$ 'short-memory' process



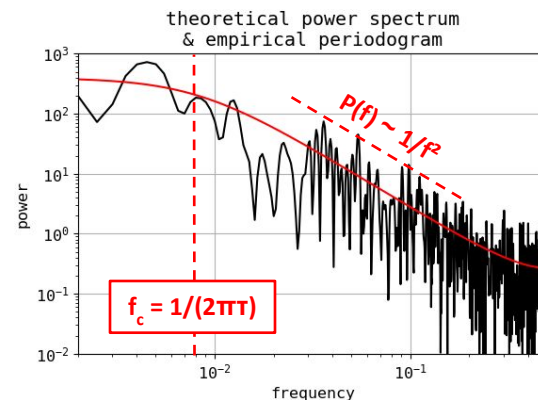
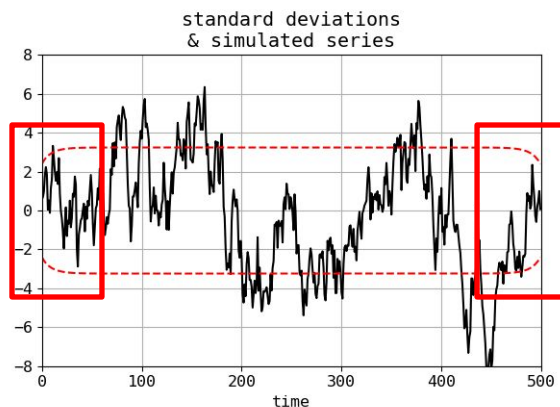
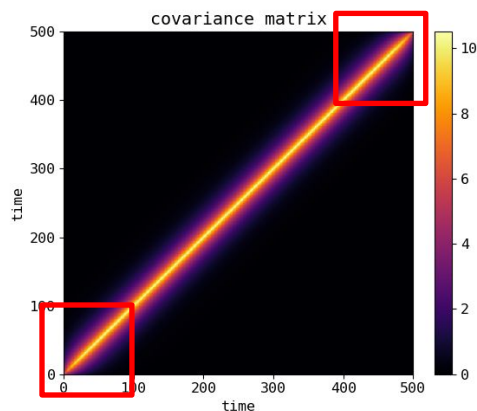
pytrf.ts – Noise components

- The stochastic model $Q(\beta)$ of a time series is encoded in a list of **noise** objects: `m.n`
- Six **noise** sub-classes are implemented: `wn`, `vw`, `ar1`, `pl`, `ggm`, `figgm`.
- **ar1** = first-order auto-regressive process (AR(1))
 - `ar1(dt=dt, flow='1-way', tn=t1)` encodes an AR(1) process with time step `dt`, starting at a specified time `t1` (by default: the beginning of the time series).
 - It is not stationary: its variance grows with time (but 'quickly' reaches a plateau).
 - Its covariance matrix has no symmetry, even before averaging.



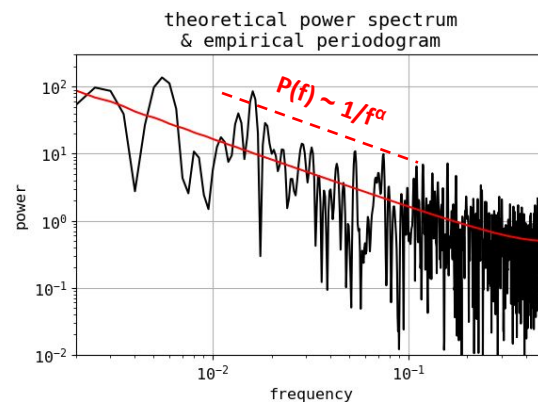
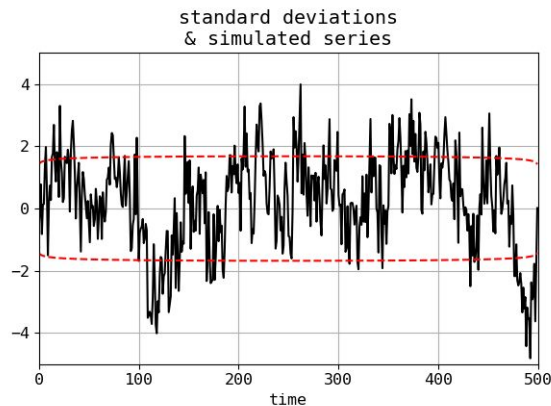
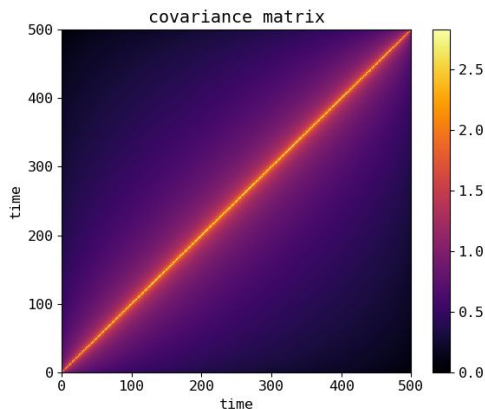
pytrf.ts – Noise components

- The stochastic model $Q(\beta)$ of a time series is encoded in a list of **noise** objects: **m.n**
- Six **noise** sub-classes are implemented: **wn**, **vw**, **ar1**, **pl**, **ggm**, **figgm**.
- **ar1** = first-order auto-regressive process (AR(1))
 - **ar1(dt=dt, flow='2-way', tn=[t1,t2])** encodes the average of:
 - a forward AR(1) process starting at time **t1** (by default: the beginning of the time series);
 - a backward AR(1) process starting at time **t2** (by default: the end of the time series).
 - It is not stationary: its variance reaches two minima at **t1** and **t2**.
 - Before averaging, its covariance matrix is centrosymmetric.



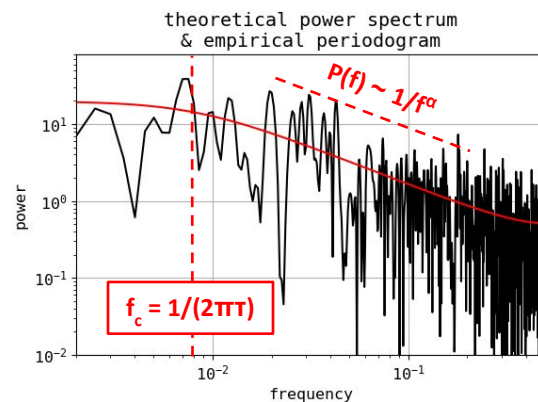
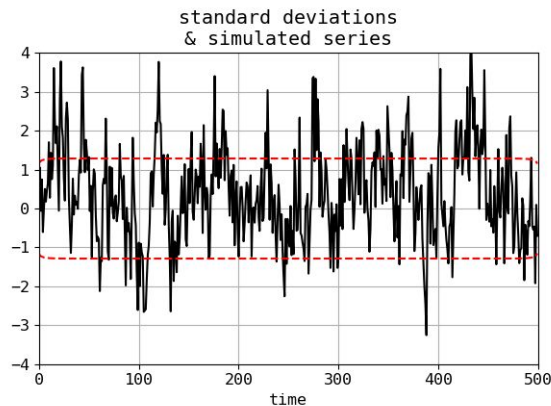
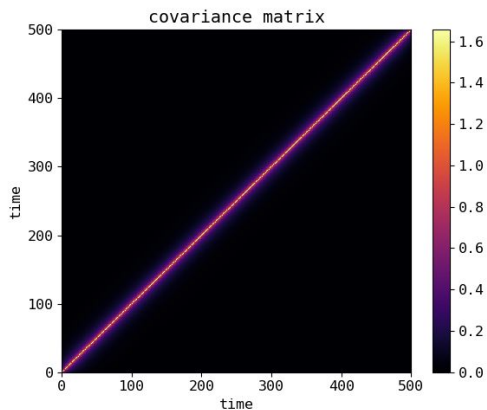
pytrf.ts – Noise components

- The stochastic model $Q(\beta)$ of a time series is encoded in a list of **noise** objects: **m.n**
- Six **noise** sub-classes are implemented: **wn**, **vw**, **ar1**, **pl**, **ggm**, **figgm**.
- **pl** = power-law process (a.k.a. fractional Gaussian noise / fractional Brownian motion)
 - **pl(dt=dt)** encodes a power-law process with time step **dt**.
 - A power-law process $x(t)$ is obtained from a WN process $w(t)$ by fractional integration: $x(t) = (1 - B)^{-\alpha/2} w(t)$ with $-1 < \alpha < 3$
 - **B** is the backward-shift operator such that: $B \cdot w(t) = w(t-dt)$
 - It has two parameters: β_0 , the variance of the underlying WN process; $\beta_1 = \alpha$, the spectral index.
 - It is available as '1-way' and '2-way', but not 'stationary', since for $\alpha \geq 1$, its variance grows indefinitely.
 - Its autocorrelation decays slowly $\rightarrow$ 'long-memory' process



pytrf.ts – Noise components

- The stochastic model $Q(\beta)$ of a time series is encoded in a list of **noise** objects: **m.n**
- Six **noise** sub-classes are implemented: **wn**, **vw**, **ar1**, **pl**, **ggm**, **figgm**.
- **ggm** = **generalized Gauss-Markov (GGM) process**
 - **ggm(dt=dt)** encodes a generalized Gauss-Markov process with time step **dt**.
 - A GGM process $x(t)$ is obtained from a WN process $w(t)$ by: $x(t) = (1 - \phi B)^{-\alpha/2} w(t)$ with $0 < \phi < 1$
 - It has three parameters: $\beta_0 = \text{var}(w)$; $\beta_1 = \alpha$, the spectral index; $\beta_2 = \tau = -dt / \log(\phi)$, a characteristic correlation time
 - Its power spectrum follows a power-law (like a PL process) then flattens at low frequencies (like an AR(1) process).
 - It is available as '1-way' and '2-way'.



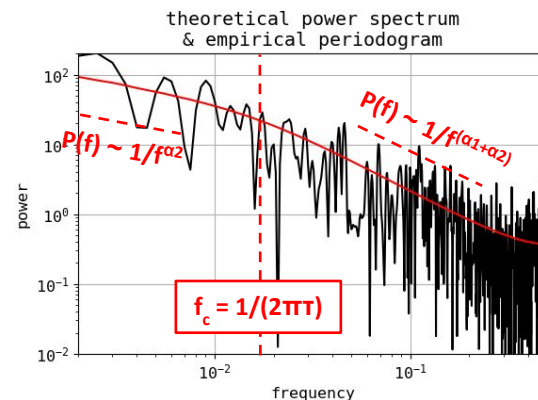
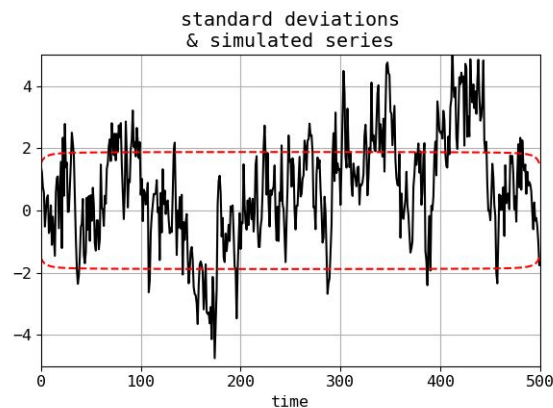
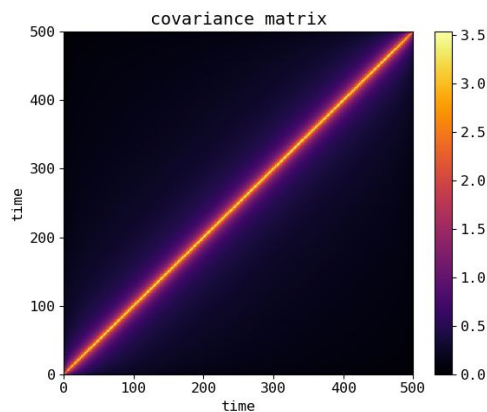
pytrf.ts – Noise components

- The stochastic model $Q(\beta)$ of a time series is encoded in a list of **noise** objects: **m.n**
- Six **noise** sub-classes are implemented: **wn**, **vw**, **ar1**, **pl**, **ggm**, **figgm**.



figgm = fractionally integrated generalized Gauss-Markov process

- **figgm(dt=dt)** encodes a fractionally integrated generalized Gauss-Markov process with time step **dt**.
- A FIGGM process $x(t)$ is obtained fractional integration of a GGM process: $x(t) = (1 - B)^{-\alpha_2/2} (1 - \phi B)^{-\alpha_1/2} w(t)$ with $-1 < \alpha_2 < 3$ and $0 < \phi < 1$
- It has four parameters: $\beta_0 = \text{var}(w)$; $\beta_1 = \alpha_1$, the 'high-frequency' spectral index; $\beta_2 = \alpha_2$, the 'low-frequency' spectral index; $\beta_3 = \tau = -dt / \log(\phi)$, a characteristic correlation time
- It is available as '1-way' and '2-way'.

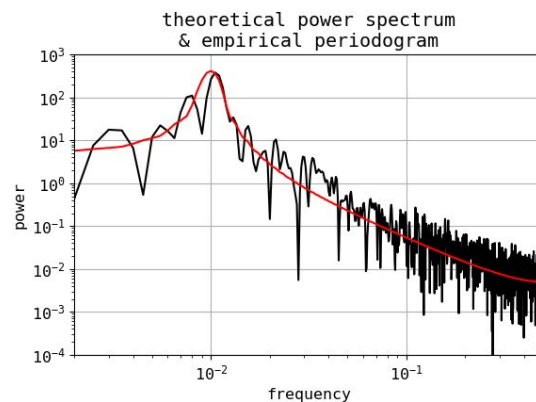
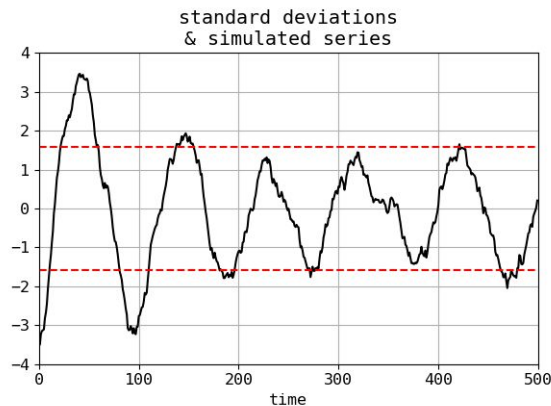
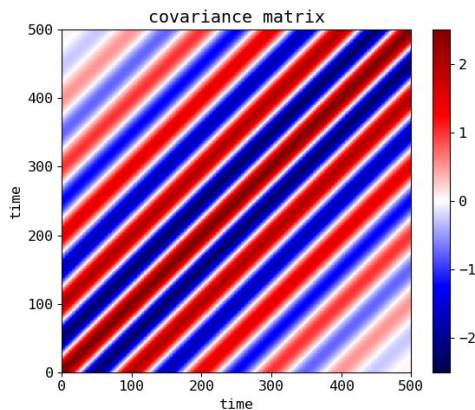


pytrf.ts – Noise components

- The stochastic model $Q(\beta)$ of a time series is encoded in a list of **noise** objects: **m.n**
- Six **noise** sub-classes are implemented: **wn**, **vw**, **ar1**, **pl**, **ggm**, **figgm**.
- The **ar1**, **pl**, **ggm** and **figgm** classes can be used to represent time-variable sine waves, i.e., processes of the form: $x_c(t)\cos(2\pi t/T) + x_s(t)\sin(2\pi t/T)$

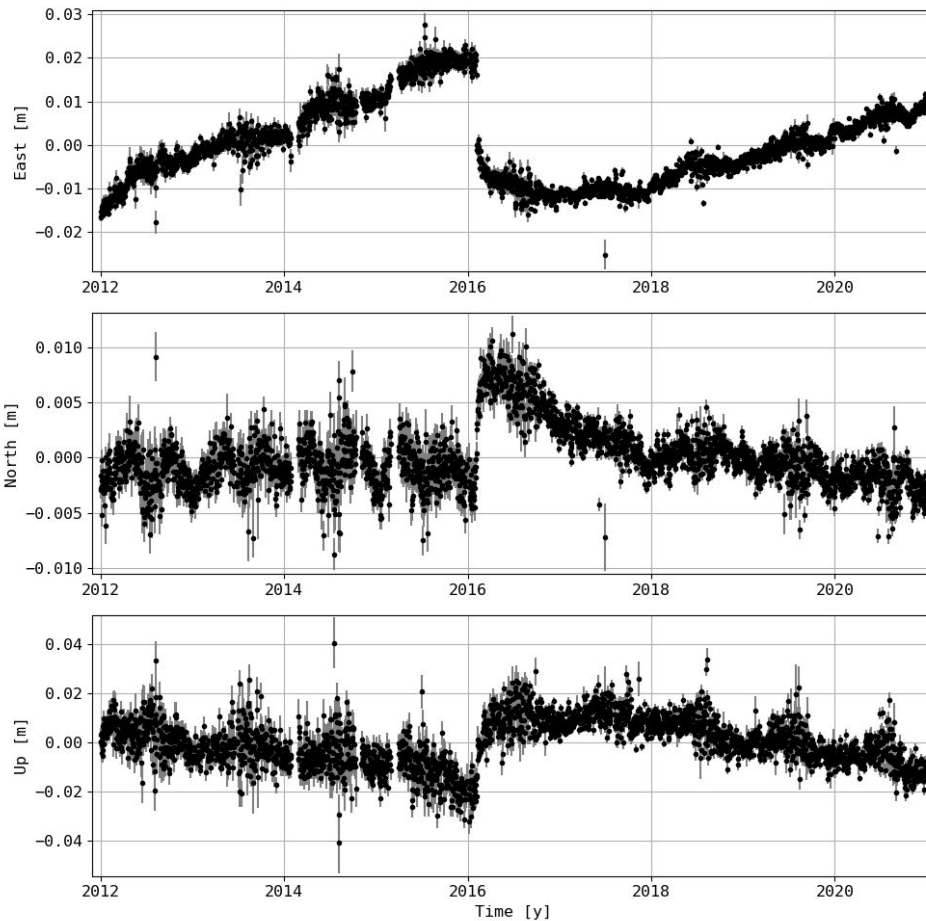
Two independent AR(1), PL, GGM or FIGGM processes

- Example: **pl(a=2.0, T=100)** encodes a sine wave of period 100 whose sine and cosine coefficients evolve as random walks.
- Parameters are the same as for usual **ar1**, **pl**, **ggm** or **figgm** objects. (The period **T** is fixed.)



pytrf.ts – How to specify a model

```
$ from pytrf.ts import ts, model  
$ r = ts.read('CKSV_igs.xyz', ...)  
$ r.clean_sigmas()  
$ r.plot(tunit='y')
```



pytrf.ts – How to specify a model

```
$ from pytrf.ts import ts, model  
$ r = ts.read('CKSV_igs.xyz', ...)  
$ r.clean_sigmas()  
$ r.plot(tunit='y')
```

Initialize model – the basic way

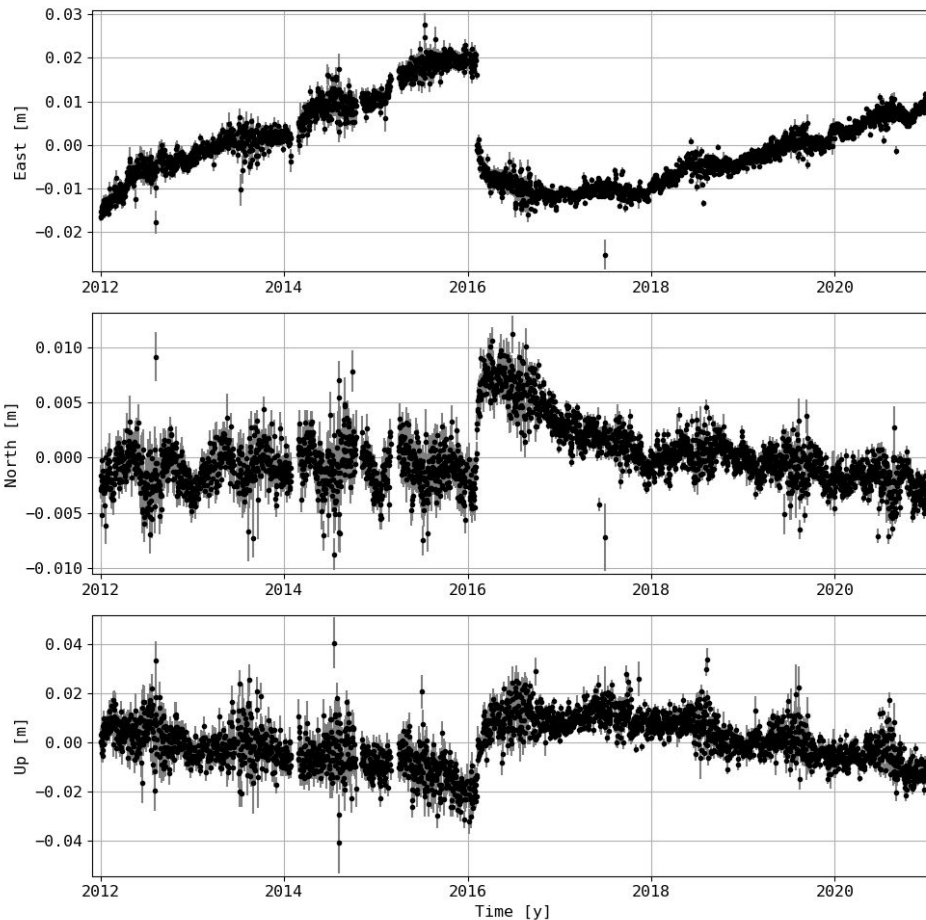
```
$ m = model(r)
```

Add deterministic functions one by one

```
$ m.add_polynom(deg=0, t=[56028.36, 57423.83])  
$ m.add_polynom(deg=1)  
$ m.add_sine(per=365.25)  
$ m.add_sine(per=182.625)  
$ m[0].add_log(t0=57423.83)  
$ m[1].add_exp(t0=57423.83)  
$ m[2].add_exp(t0=57423.83)
```

Add variable white noise

```
$ m.add_vw()
```



pytrf.ts – How to specify a model

```
$ from pytrf.ts import ts, model  
$ r = ts.read('CKSV_igs.xyz', ...)  
$ r.clean_sigmas()  
$ r.plot(tunit='y')
```

Initialize model – the basic way

```
$ m = model(r)
```

Add deterministic functions one by one

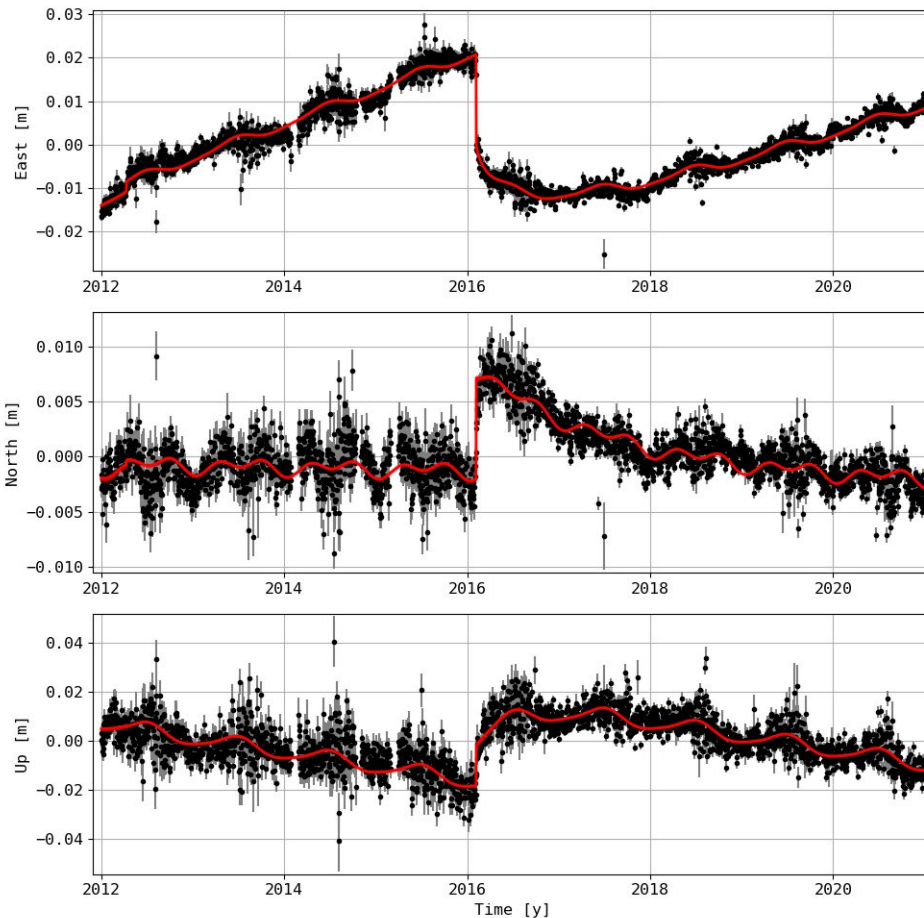
```
$ m.add_polynom(deg=0, t=[56028.36, 57423.83])  
$ m.add_polynom(deg=1)  
$ m.add_sine(per=365.25)  
$ m.add_sine(per=182.625)  
$ m[0].add_log(t0=57423.83)  
$ m[1].add_exp(t0=57423.83)  
$ m[2].add_exp(t0=57423.83)
```

Add variable white noise

```
$ m.add_vw()
```

Fit model

```
$ m.fit()  
$ m.plot_fit(tunit='y')
```



pytrf.ts – How to specify a model

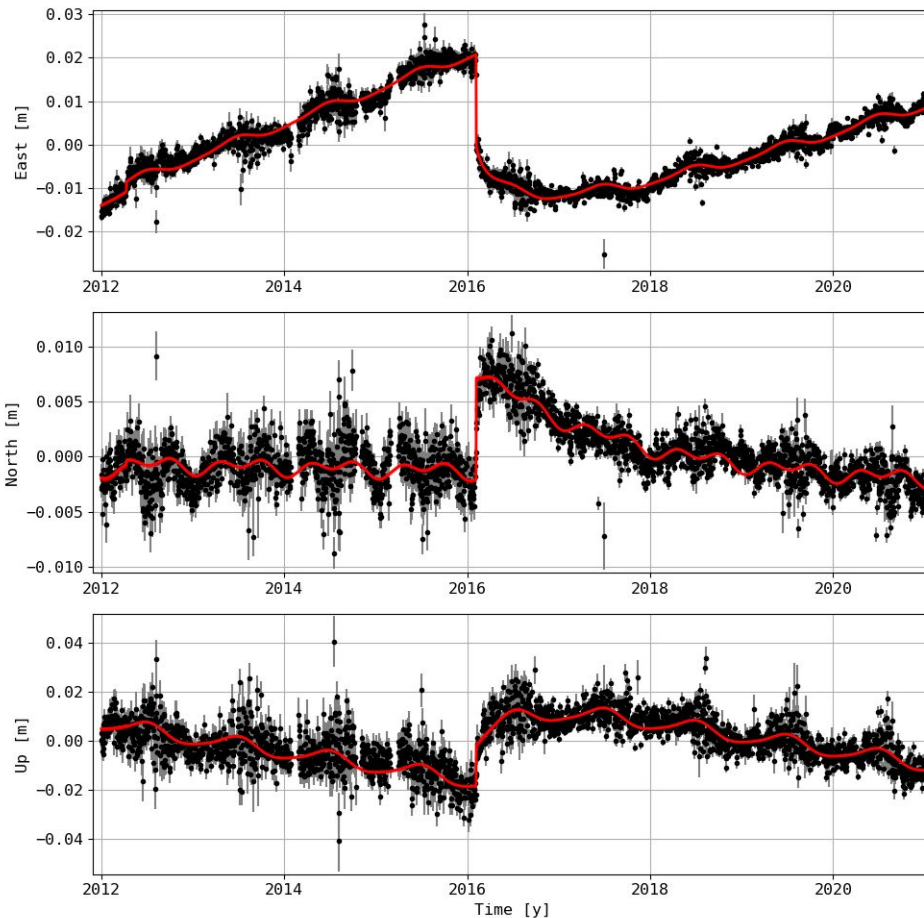
```
$ from pytrf.ts import ts, model
$ r = ts.read('CKSV_igs.xyz', ...)
$ r.clean_sigmas()
$ r.plot(tunit='y')

# Initialize model using shortcuts
$ m = model(r, deg=[0,1], per=[365.25,182.625],
            noise=['vw'])

# Add position jumps
$ m.add_jumps(deg=[0], t=[56028.36,57423.83])

# Add remaining deterministic functions
$ m[0].add_log(t0=57423.83)
$ m[1].add_exp(t0=57423.83)
$ m[2].add_exp(t0=57423.83)

# Fit model
$ m.fit()
$ m.plot_fit(tunit='y')
```



pytrf.ts – How to specify a model

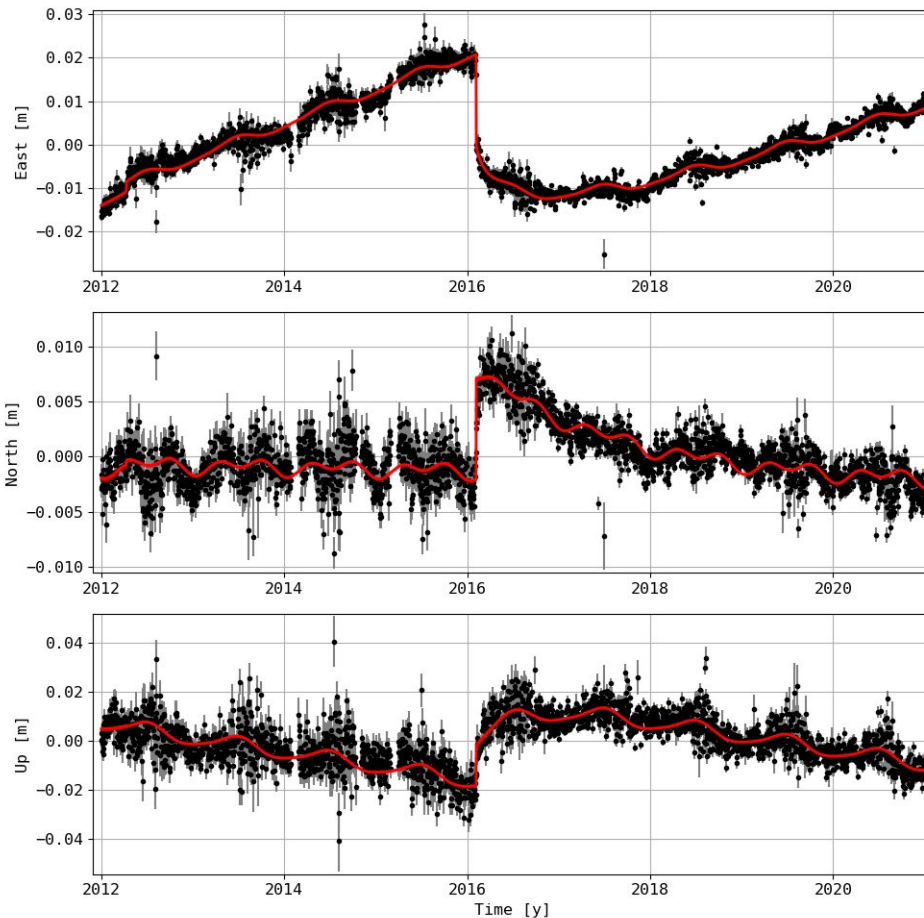
```
$ from pytrf.ts import ts, model
$ r = ts.read('CKSV_igs.xyz', ...)
$ r.clean_sigmas()
$ r.plot(tunit='y')

# Initialize model from:
# - discontinuity list (e.g., soln\_IGc20.snx)
# - PSD SINEX file (e.g., psd\_IGc20.snx)
$ from pytrf.io import read_solns
$ solns = read_solns('soln_IGc20.snx')

$ from pytrf import sinex
$ psd = sinex.read('psd_IGc20.snx')

$ m = model.from_solns(r, solns, code='CKSV',
                      per=[365.25, 182.625],
                      noise=['vw'], psd=psd)

# Fit model
$ m.fit()
$ m.plot_fit(tunit='y')
```



pytrf.ts – How to specify a model

- Any deterministic or noise parameter can be fixed to some a priori value.

```
# Example 1: Logarithmic transient with fixed relaxation time
```

```
$ m = model(r, deg=[0,1], per=[365.25,182.625], noise=['vw'])
$ m[0].add_log(t0=57423.83)
$ m[0].f[4].par[1].x = 23
$ m[0].f[4].par[1].fixed = True
```

```
# A simpler syntax for the same purpose
```

```
$ m = model(r, deg=[0,1], per=[365.25,182.625], noise=['vw'])
$ m[0].add_log(t0=57432.83, tau=23, fix_tau=True)
```

```
# Note: When initializing a model from a discontinuity list and a PSD SINEX file,  
# you can choose to fix all parameters of the PSD models:
```

```
$ m = model.from_solns(r, solns, code='CKSV', per=[365.25, 182.625], noise=['vw'],  
                      psd=psd, fix_amp=True, fix_tau=True)
```

```
# Example 2: Fix the amplitude of a position jump to some known value
```

```
$ m = model(r, deg=[0,1], per=[365.25,182.625], noise=['vw'])
$ m.add_jumps(deg=[0], t=[56028.36,57423.83])
$ m[0].f[0].par[1].x = 0.0021 ; m[0].f[0].par[1].fixed = True
$ m[1].f[0].par[1].x = 0.0011 ; m[1].f[0].par[1].fixed = True
$ m[2].f[0].par[1].x = 0.0000 ; m[2].f[0].par[1].fixed = True
```

pytrf.ts – How to specify a model

- Any deterministic or noise parameter can be fixed to some a priori value.

```
# Example 3: Power-law noise with spectral index fixed to 1.0 (flicker noise)
```

```
$ m = model(r, deg=[0,1], per=[365.25,182.625], noise=['vw','pl'])
```

```
$ for d in range(m.nd):
```

```
$     m[d].n[1].par[1].x = 1.0
```

```
$     m[d].n[1].par[1].fixed = True
```

```
# A simpler syntax for the same purpose:
```

```
$ m = model(r, deg=[0,1], per=[365.25,182.625], noise=['vw'])
```

```
$ m.add_pl(a=1.0, fix_a=True)
```

```
# An even simpler syntax for this particular value of the spectral index:
```

```
$ m = model(r, deg=[0,1], per=[365.25,182.625], noise=['vw','fn'])
```

```
# Note: There is a similar shortcut to fix the spectral index to 2.0 (random walk).
```

```
$ m = model(r, deg=[0,1], per=[365.25,182.625], noise=['vw','rw'])
```

pytrf.ts – How to specify a model

- Any deterministic parameter can be constrained to some reference value.

```
# Example: Constrain the amplitude of a position jump to some reference value +/- 1 mm
$ m = model(r, deg=[0,1], per=[365.25,182.625], noise=['vw'])
$ m.add_jumps(deg=[0], t=[56028.36,57423.83])
$ m[0].f[0].par[1].xc = 0.0021 ; m[0].f[0].par[1].sigc = 0.0010
$ m[1].f[0].par[1].xc = 0.0011 ; m[1].f[0].par[1].sigc = 0.0010
$ m[2].f[0].par[1].xc = 0.0000 ; m[2].f[0].par[1].sigc = 0.0010
```

pytrf.ts – How `m.fit()` works

- Two different estimators of $\mathbf{x}$ and $\boldsymbol{\beta}$ are implemented:

- the usual maximum likelihood estimator: `m.fit(estimator='ml')`
- the restricted maximum likelihood estimator (default): `m.fit(estimator='reml')`

- In Bayesian terms, assuming uniform priors, the ML estimator maximizes the joint probability of $\mathbf{x}$ and $\boldsymbol{\beta}$ given the observations $\mathbf{y}$:

$$(\hat{\mathbf{x}}_{\text{ML}}, \hat{\boldsymbol{\beta}}_{\text{ML}}) = \operatorname{argmax}(\mathbf{p}(\mathbf{x}, \boldsymbol{\beta} | \mathbf{y}))$$

with $\mathbf{p}(\mathbf{x}, \boldsymbol{\beta} | \mathbf{y}) \propto \mathbf{p}(\mathbf{y} | \mathbf{x}, \boldsymbol{\beta}) = \frac{1}{\sqrt{(2\pi)^n |\mathbf{Q}(\boldsymbol{\beta})|}} \exp\left(-\frac{1}{2}(\mathbf{y} - \mathbf{f}(\mathbf{x}))^T \mathbf{Q}(\boldsymbol{\beta})^{-1}(\mathbf{y} - \mathbf{f}(\mathbf{x}))\right)$

- In many cases, $\mathbf{f}(\mathbf{x})$ is linear: $\mathbf{f}(\mathbf{x}) = \mathbf{A}\mathbf{x}$. Thus, for a given $\boldsymbol{\beta}$, the optimal $\mathbf{x}$ can be found analytically using weighted least-squares:
$$\hat{\mathbf{x}}(\boldsymbol{\beta}) = (\mathbf{A}^T \mathbf{Q}(\boldsymbol{\beta})^{-1} \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Q}(\boldsymbol{\beta})^{-1} \mathbf{y}$$
- If the deterministic model includes non-linear parameters, e.g., exp or log relaxation times, their weighted-least squares estimates for a given $\boldsymbol{\beta}$ are found by numerical optimization.
- On the other hand, there is in general no analytical expression for the optimal noise parameters, and they are found by maximizing $\mathbf{p}(\hat{\mathbf{x}}(\boldsymbol{\beta}), \boldsymbol{\beta} | \mathbf{y})$ numerically.
- The only exception is when there's just one noise component with just its variance factor unknown ($\mathbf{Q}(\boldsymbol{\beta}) = \boldsymbol{\beta} \times \mathbf{Q}_0$) in which case $\hat{\mathbf{x}}$ does not depend on $\boldsymbol{\beta}$ and $\hat{\boldsymbol{\beta}} = (\mathbf{y} - \mathbf{f}(\hat{\mathbf{x}}))^T \mathbf{Q}_0^{-1}(\mathbf{y} - \mathbf{f}(\hat{\mathbf{x}})) / n$

pytrf.ts – How `m.fit()` works

- Two different estimators of $\mathbf{x}$ and $\boldsymbol{\beta}$ are implemented:

- the usual maximum likelihood estimator: `m.fit(estimator='ml')`
- the restricted maximum likelihood estimator (default): `m.fit(estimator='reml')`

- On the other hand, the REML estimator maximizes the marginal probability of $\boldsymbol{\beta}$ given the observations $\mathbf{y}$:

$$\hat{\boldsymbol{\beta}}_{\text{REML}} = \operatorname{argmax}(\mathbf{p}(\boldsymbol{\beta}|\mathbf{y}))$$

$$\text{with } \mathbf{p}(\boldsymbol{\beta}|\mathbf{y}) = \int \mathbf{p}(\mathbf{x}, \boldsymbol{\beta}|\mathbf{y}) d\mathbf{x} \propto \frac{1}{\sqrt{(2\pi)^{n-p} |\mathbf{Q}(\boldsymbol{\beta})| |\mathbf{A}^T \mathbf{Q}(\boldsymbol{\beta})^{-1} \mathbf{A}|}} \exp\left(-\frac{1}{2}(\mathbf{y} - \mathbf{f}(\hat{\mathbf{x}}(\boldsymbol{\beta})))^T \mathbf{Q}(\boldsymbol{\beta})^{-1} (\mathbf{y} - \mathbf{f}(\hat{\mathbf{x}}(\boldsymbol{\beta})))\right)$$

- Like in the ML case, $\hat{\mathbf{x}}(\boldsymbol{\beta})$ is found by weighted least-squares for a given $\boldsymbol{\beta}$.
 - But $\hat{\boldsymbol{\beta}}$ is found by numerical optimization...
 - ... except in the trivial case $\mathbf{Q}(\boldsymbol{\beta}) = \boldsymbol{\beta} \times \mathbf{Q}_0 \rightarrow \hat{\boldsymbol{\beta}} = (\mathbf{y} - \mathbf{f}(\hat{\mathbf{x}}))^T \mathbf{Q}_0^{-1} (\mathbf{y} - \mathbf{f}(\hat{\mathbf{x}})) / (n - p)$
 - Unlike the ML estimator, the REML estimator of $\boldsymbol{\beta}$ accounts for the simultaneous estimation of the deterministic parameters, which makes it unbiased.
- Prefer the REML estimator in general.

pytrf.ts – How `m.fit()` works

- The ML or REML estimate of β can be obtained by different optimization methods:

	Nelder-Mead aka downhill simplex	Powell	Conjugate gradients	BFGS	Newton-Raphson
<code>m.fit(method=?)</code>	'Nelder-Mead'	'Powell'	'CG'	'BFGS'	'Newton'
Use gradient?	No	No	Yes	Yes	Yes
Use hessian?	No	No	No	No	Yes
Support Williams' trick?	Yes	Yes	No	No	No
Notes	- Fast iterations - But many iterations - Most often converges	Not much tested	Not as efficient as BFGS	- Few iterations - Often converges	- Few iterations - But long iterations - Often diverges

- From personal experience, BFGS is the best option:
 - Faster than Nelder-Mead thanks to much less iterations and than Newton-Raphson thanks to shorter iterations
 - Almost always converges

→ Start with BFGS. If convergence fails, fall back on Nelder-Mead.

pytrf.ts – After $\hat{\mathbf{x}}$ and $\hat{\boldsymbol{\beta}}$ have been found...

- Estimated values are assigned to the deterministic parameters: `m.f[i].par[j].x`
 - The formal covariance matrix of $\hat{\mathbf{x}}$, $\text{var}(\hat{\mathbf{x}}) = \left(\mathbf{A}^T \mathbf{Q}(\hat{\boldsymbol{\beta}})^{-1} \mathbf{A}\right)^{-1}$, is stored as `m.Qx`.
 - Formal errors are assigned to the deterministic parameters: `m.f[i].par[j].sig`
- Estimated values are assigned to the noise parameters: `m.n[i].par[j].x`
 - An approximate formal covariance matrix of $\hat{\boldsymbol{\beta}}$ is stored as `m.Qb`.
 - Formal errors are assigned to the noise parameters: `m.n[i].par[j].sig`
 - `m.Qb = -H-1`, where $\mathbf{H}$ is the Hessian of the [restricted] log-likelihood at $\hat{\boldsymbol{\beta}}$.
 - It would be the covariance matrix of $\hat{\boldsymbol{\beta}}$ if the [restricted] log-likelihood was a parabola, i.e., if $p(\boldsymbol{\beta}|\mathbf{y})$ was a Gaussian.
 - $\mathbf{H}$ can be either computed by numerical differentiation: `m.fit(hessian='numeric')`
or approximated by the Hessian of the expected [restricted] log-likelihood: `m.fit(hessian='expected')`
- Values of the deterministic and noise parameters, as well as other outputs, can be printed in YAML format by calling `print(m)`.

pytrf.ts – After $\hat{x}$ and $\hat{\beta}$ have been found...

```
ndim: 3
dims: [East, North, Up]
nobs: 3011
time_unit: d
series_unit: m
date_format: decimal d
t0: 57626.5678

npar: [10, 10, 10]
wrms: [ 1.340861e-03,  1.238111e-03,  4.277529e-03]
logl: [ 1.503547e+04,  1.530326e+04,  1.165261e+04]
loglr: [ 1.494602e+04,  1.521320e+04,  1.157392e+04]
bicr: [ 1.494202e+04,  1.520920e+04,  1.156992e+04]
Er:    [ 1.493522e+04,  1.520252e+04,  1.156310e+04]

noise_params:
- {idim: 0, type: VW variance factor, start: -.Inf, value: 3.571250e+00, sigma: 8.105882e-06, unit: }
- {idim: 1, type: VW variance factor, start: -.Inf, value: 3.810034e+00, sigma: 9.211097e-06, unit: }
- {idim: 2, type: VW variance factor, start: -.Inf, value: 2.868380e+00, sigma: 7.954422e-06, unit: }

params:
- {idim: 0, type: polynomial coefficient, degree: 0, start: -.Inf, value: -1.321589e-02, sigma: 2.150813e-04, unit: m}
- {idim: 0, type: polynomial coefficient jump, degree: 0, start: 56028.3602, value: 2.254345e-03, sigma: 2.460801e-04, unit: m}
- {idim: 0, type: polynomial coefficient jump, degree: 0, start: 57423.8316, value: -2.103148e-02, sigma: 6.521766e-04, unit: m}
- {idim: 0, type: polynomial coefficient, degree: 1, start: -.Inf, value: 2.152378e-05, sigma: 1.354089e-07, unit: m/d}
- {idim: 0, type: cos amplitude, period: 365.250, start: -.Inf, value: 2.505873e-04, sigma: 3.765214e-05, unit: m}
- {idim: 0, type: sin amplitude, period: 365.250, start: -.Inf, value: -1.015902e-03, sigma: 3.506910e-05, unit: m}
- {idim: 0, type: cos amplitude, period: 182.625, start: -.Inf, value: -5.876712e-05, sigma: 3.535190e-05, unit: m}
- {idim: 0, type: sin amplitude, period: 182.625, start: -.Inf, value: -2.340640e-04, sigma: 3.567495e-05, unit: m}
- {idim: 0, type: log amplitude, start: 57423.8316, value: -6.712612e-03, sigma: 1.368472e-04, unit: m}
- {idim: 0, type: log relaxation time, start: 57423.8316, value: 2.162232e+01, sigma: 2.957916e+00, unit: d}
- {idim: 1, type: polynomial coefficient, degree: 0, start: -.Inf, value: -1.079797e-03, sigma: 1.952929e-04, unit: m}
- {idim: 1, type: polynomial coefficient jump, degree: 0, start: 56028.3602, value: 4.211363e-04, sigma: 2.298323e-04, unit: m}
- {idim: 1, type: polynomial coefficient jump, degree: 0, start: 57423.8316, value: 9.186088e-03, sigma: 2.222161e-04, unit: m}
...
```

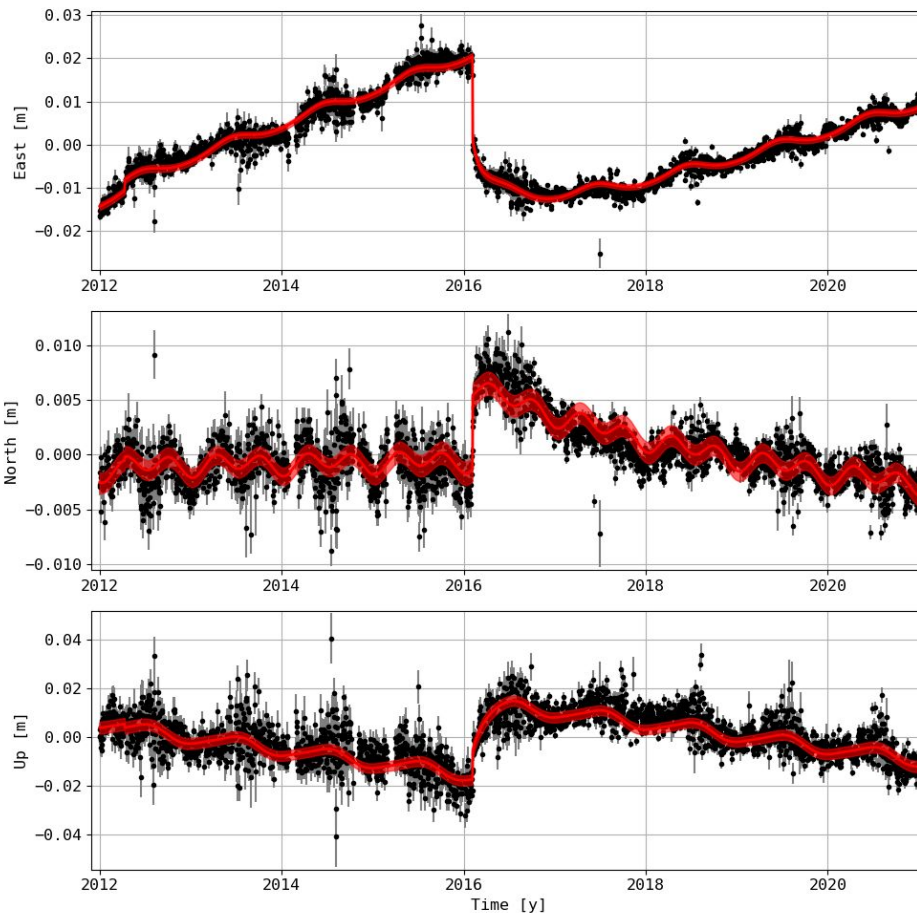
pytrf.ts – After $\hat{x}$ and $\hat{\beta}$ have been found...

- A lot more attributes are also set up, including:

- Deterministic model $f(x)$: `m.yc`
- Associated covariance matrix & formal errors: `m.Qc, m.sc`
- Contribution of function k to the deterministic model ($f_k(x_k)$): `m.f[k].yc`
- Covariance matrix of full noise model $Q(\beta)$: `m.Q`
- Contribution of noise component k ($Q_k(\beta_k)$): `m.n[k].Q`
- Residuals $v = y - f(x)$: `m.v`
- Associated covariance matrix & formal errors: `m.Qv, m.sv`
- Normalized residuals: `m.vn`
- WRMS of residuals: `m.wrms`
- [Restricted] log-likelihood: `m.logl or m.loglr`
- Bayesian information criterion (BIC), or rather $-BIC/2$: `m.bic or m.bicr`
- Lomb-Scargle periodogram of residuals: `m.pv`
- Theoretical power spectral density of noise model: `m.pn`
- Associated covariance matrix & formal errors: `m.Qn, m.sn`
- Note: `m.pv` and `m.pn` are computed over a certain range of frequencies, `m.fr`, which is automatically set by default. It is however possible to choose a custom range of frequencies by setting `m.fr` manually before calling `m.fit()`.

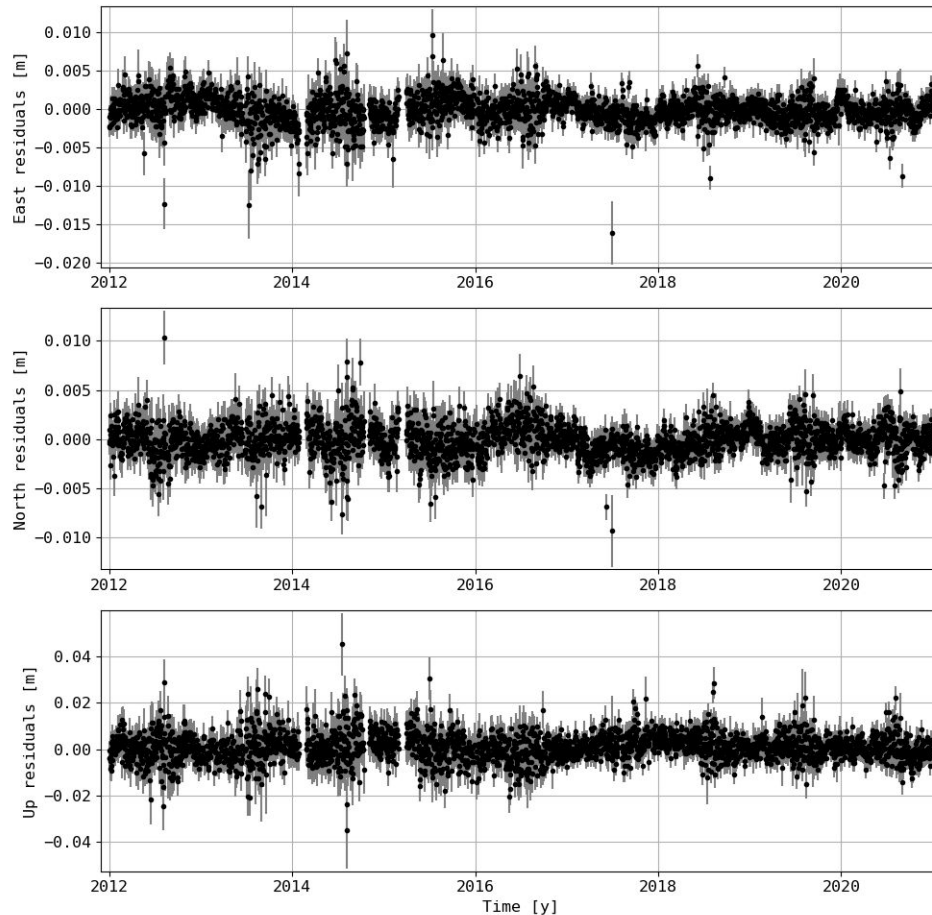
pytrf.ts – After $\hat{x}$ and $\hat{\beta}$ have been found...

- It is possible to make some easy figures.
 - `m.plot_fit()` shows the deterministic model on top of the observations.



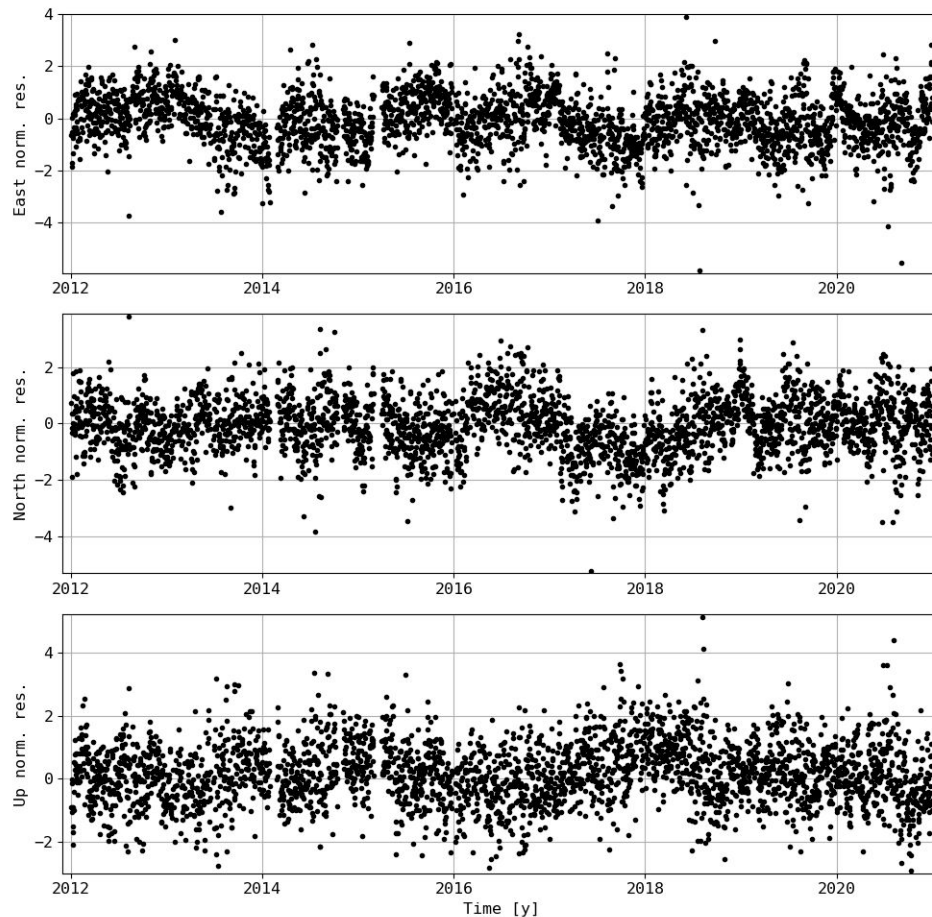
pytrf.ts – After $\hat{x}$ and $\hat{\beta}$ have been found...

- It is possible to make some easy figures.
 - `m.plot_fit()` shows the deterministic model on top of the observations.
 - `m.plot_res()` shows the residuals of the model.



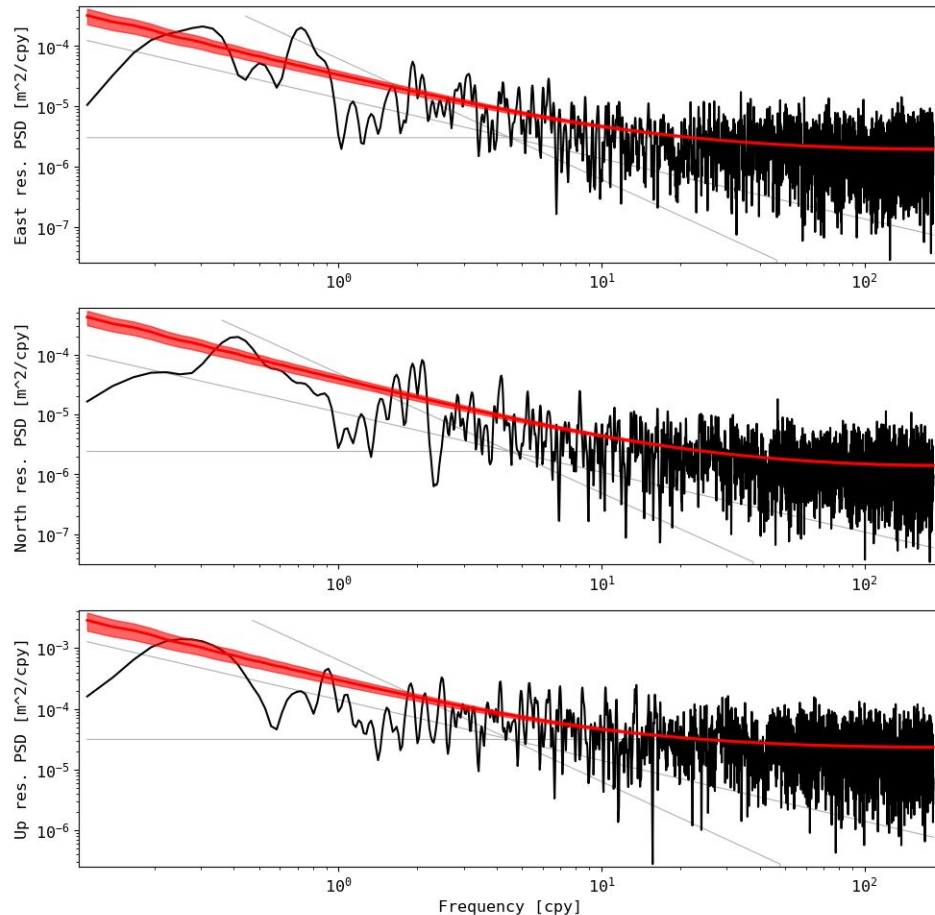
pytrf.ts – After $\hat{x}$ and $\hat{\beta}$ have been found...

- It is possible to make some easy figures.
 - `m.plot_fit()` shows the deterministic model on top of the observations.
 - `m.plot_res()` shows the residuals of the model.
 - `m.plot_normres()` shows the normalized residuals.



pytrf.ts – After $\hat{x}$ and $\hat{\beta}$ have been found...

- It is possible to make some easy figures.
 - `m.plot_fit()` shows the deterministic model on top of the observations.
 - `m.plot_res()` shows the residuals of the model.
 - `m.plot_normres()` shows the normalized residuals.
 - `m.plot_psd()` shows the power spectral density of the noise model on top of the Lomb-Scargle periodogram of the residuals.



pytrf.ts – After $\hat{x}$ and $\hat{\beta}$ have been found...

- It is possible to make some easy figures.
 - `m.plot_fit()` shows the deterministic model on top of the observations.
 - `m.plot_res()` shows the residuals of the model.
 - `m.plot_normres()` shows the normalized residuals.
 - `m.plot_psd()` shows the power spectral density of the noise model on top of the Lomb-Scargle periodogram of the residuals.
 - `m.plot_all()` shows all 4 previous figures.

pytrf.ts – After $\hat{x}$ and $\hat{\beta}$ have been found...

- An estimate of each individual noise process is obtained by Wiener-filtering the residuals.

- For each noise process k , $\hat{\xi}_k = Q_k(\beta_k)Q(\beta)^{-1}\hat{v}$, is stored as: `m.n[k].xi`
- Associated covariance matrix & formal errors: `m.n[k].Qxi`, `m.n[k].sxi`

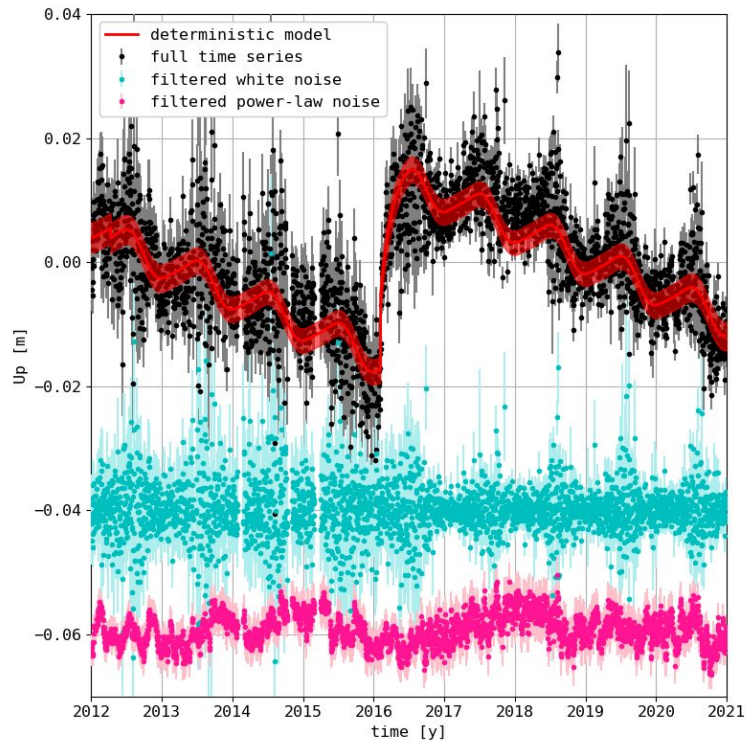
```
$ r = ts.read('CKSV.xyz', ...)
$ t = [date.from_mjd(d).ydec() for d in r.t]
$ m = model(r, ..., noise=['vw', 'pl'])
$ m.fit()

# Initialize figure
$ pp.figure(figsize=(8, 8), tight_layout=True)

# Plot Up component of time series + deterministic model
$ pp.errorbar(t, r[2].y, yerr=np.sqrt(r[2].Q), fmt='.k',
             ecol='gray', label='full time series')
$ pp.plot(t, m[2].yc, 'r', linewidth=2, label='deterministic model')
$ pp.fill_between(t, m[2].yc-m[2].sc, m[2].yc+m[2].sc, color='r', alpha=0.6)

# Plot filtered white noise & filtered power-law noise
$ pp.errorbar(t, m[2].n[0].xi-0.04, yerr=m[2].n[0].sxi, fmt='.c',
             ecol='paleturquoise', label='filtered white noise')
$ pp.errorbar(t, m[2].n[1].xi-0.06, yerr=m[2].n[1].sxi, fmt='.', mfc='deeppink',
             mec='deeppink', ecol='pink', label='filtered power-law noise')

# Finalize figure
$ pp.axis([2012, 2021, -0.07, 0.04]) ; pp.grid()
$ pp.xlabel('time [y]') ; pp.ylabel('Up [m]')
$ pp.legend(framealpha=1) ; pp.show()
```



pytrf.ts – Iterative model fitting & outlier rejection

- A given deterministic+noise model can be adjusted iteratively, with possible outliers removed at each iteration, by calling `m.fit_iter()`.

- `m.fit_iter(thr_norm=X)` filters data points with normalized residuals along any component larger than X.
- `m.fit_iter(thr_raw=X)` filters data points with residuals along any component larger than X times the WRMS of the residuals along that component.
- `m.fit_iter(thr_mad=X, win_mad=W)` filters data points whose residuals fall outside the 'running median $\pm$ X times running MAD' limit along any component.
The running median and MAD (median absolute deviation) are both computed over W-long time windows.
- Any combination of the three criteria can be used.

- Example:

```
$ m = model(r, ..., noise=['vw'])
$ m.fit_iter(thr_norm=5)
$ m.fit_iter(thr_norm=5, thr_mad=5, win_mad=365)
$ m.fit_iter(thr_norm=4, thr_mad=4, win_mad=365)
$ m.add_pl()
$ m.fit()
```

Initialize model with only variable white noise

1st iterative outlier rejection based on normalized residuals

2nd iterative outlier rejection with additional threshold for raw residuals

3rd iterative outlier rejection with lower thresholds

Add power-law noise into model

Final fit

- Notes:

- Use `fit_iter()` only if the deterministic model is appropriate.
- To save time, better filter outliers using (variable) white noise only, then add time-correlated noise model.

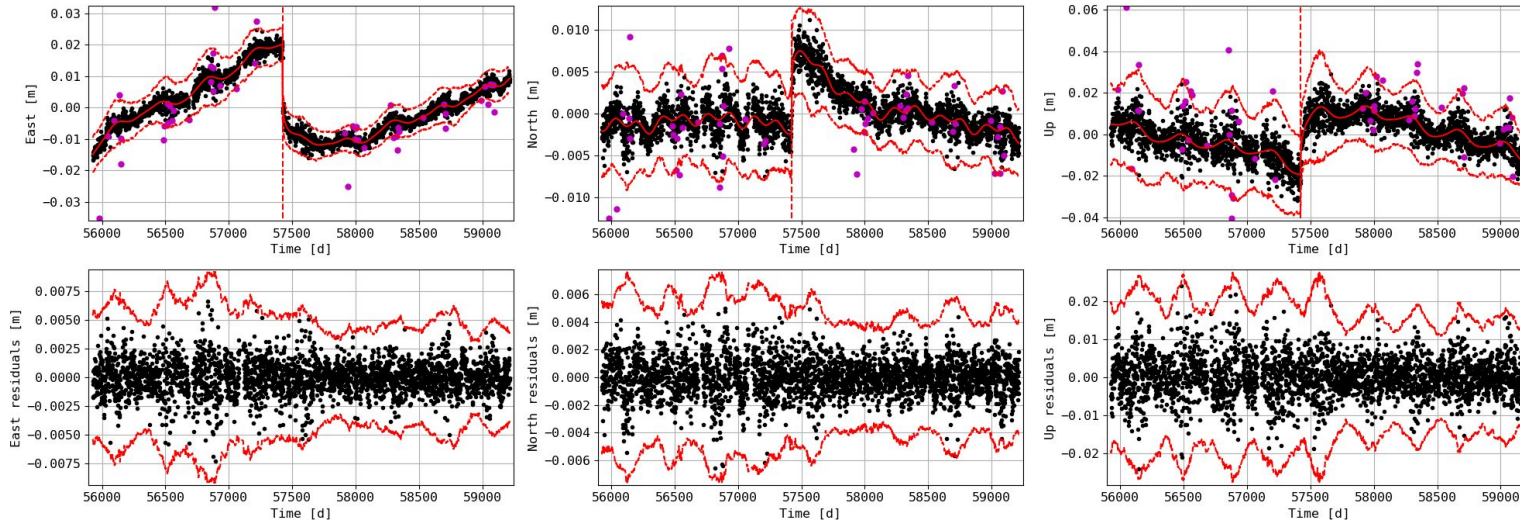
pytrf.ts – “Model-free” outlier rejection

- If you don't have an appropriate deterministic model available a priori for your time series, you may nevertheless identify and reject outliers with `ts.clean_outliers()`.
 - `ts.clean_outliers()` robustly fits a semi-parametric model to the time series.
 - Points with residuals outside a 'running median $\pm$ X times running MAD' limit along any component are iteratively filtered.

- **Example:**

```
$ r = ts.read('CKSV_igs.xyz', ...)  
$ r.clean_outliers(final_plot=True)
```

Note: `ts.clean_outliers()` has many parameters whose default values are tailored to daily GNSS station position time series expressed in days and meters. Be sure to adapt the parameter values if you use different types of time series or different units.

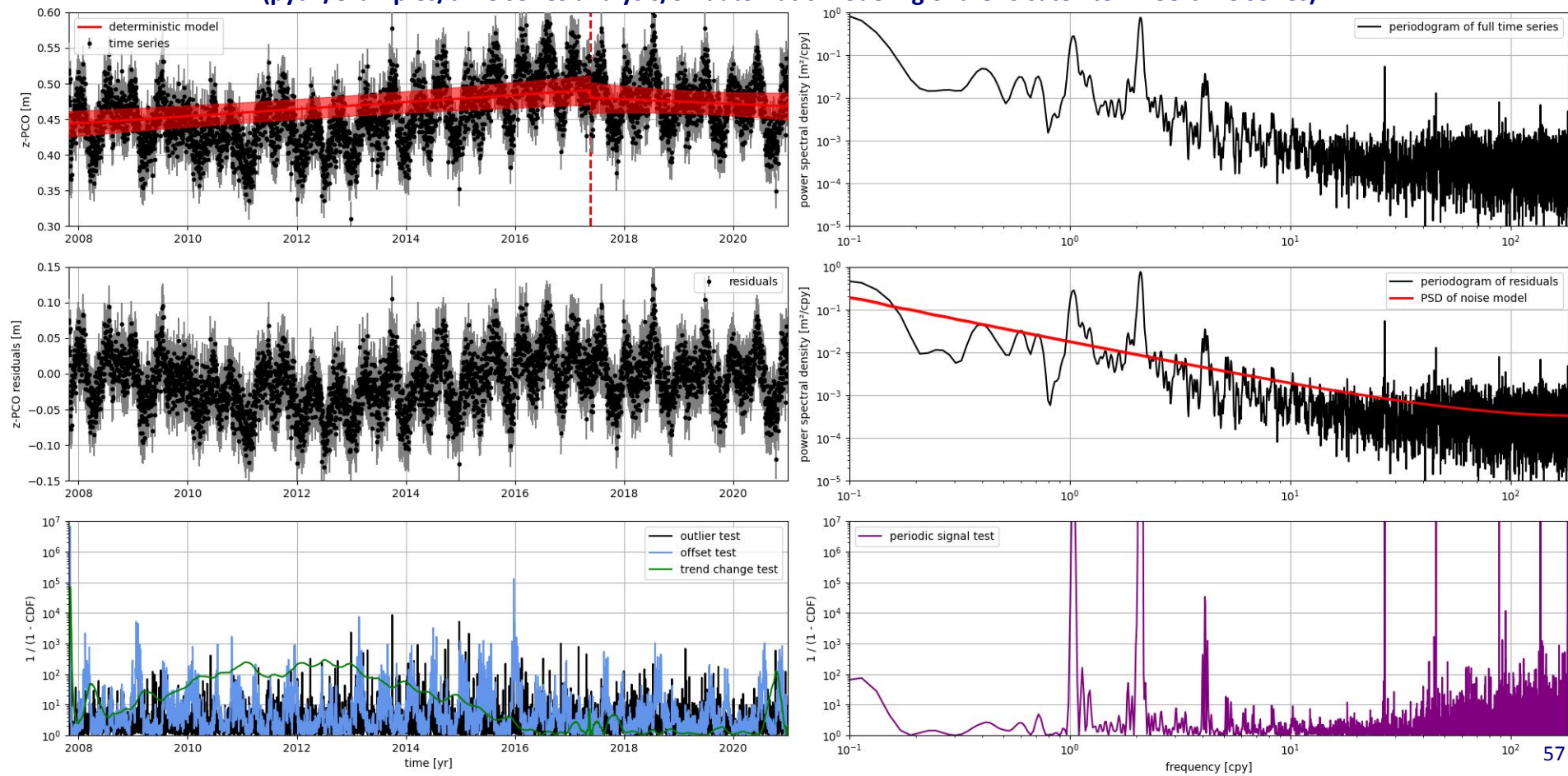


pytrf.ts – Model selection

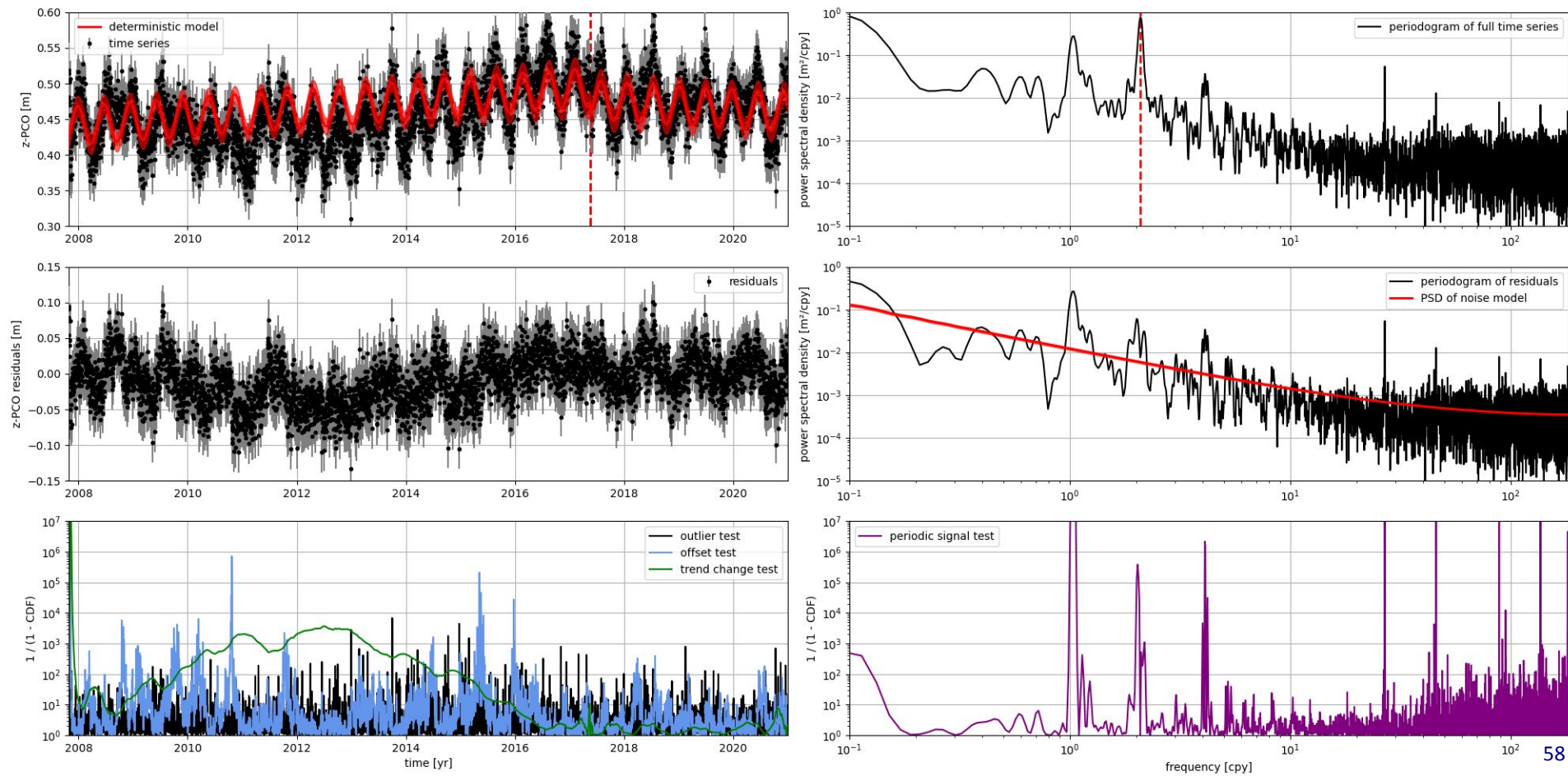
- Several likelihood ratio (LR) tests are implemented:
 - `m.glr_outlier()` tests the removal of every point of the series.
 - `m.glr_mean()` tests the addition of a mean change (position discontinuity) at every date of the series.
 - `m.glr_trend()` tests the addition of a trend change (velocity discontinuity) at every date of the series.
 - `m.glr_mean_trend()` tests the addition of a mean+trend change (position+velocity discontinuity) at every date of the series.
 - `m.glr_sine()` tests the addition of a sine wave at every frequency in `m.fr`.
 - Each function returns an array of test statistics that are distributed as either $\chi^2(n_d)$ (outlier, mean, trend) or $\chi^2(2n_d)$ (mean+trend, sine) under the hypothesis that the current deterministic+noise model is appropriate.
 - The test statistics can be confronted with quantiles of the respective χ^2 distributions to identify outliers or missing components in the deterministic model.
- The BIC can also be used to select among different models.
 - Found to perform satisfactorily when comparing nested models. (Not tested in other cases.)
 - A non-zero threshold however seems necessary, e.g., select more complex model only if $-\Delta\text{BIC} / 2 > 8$.
 - For the systematic testing of additional mean/trend changes or sine waves, better use the LR test functions, which do not require fitting each alternative model, and are optimized.

pytrf.ts – Usage example of LR tests: Step 0

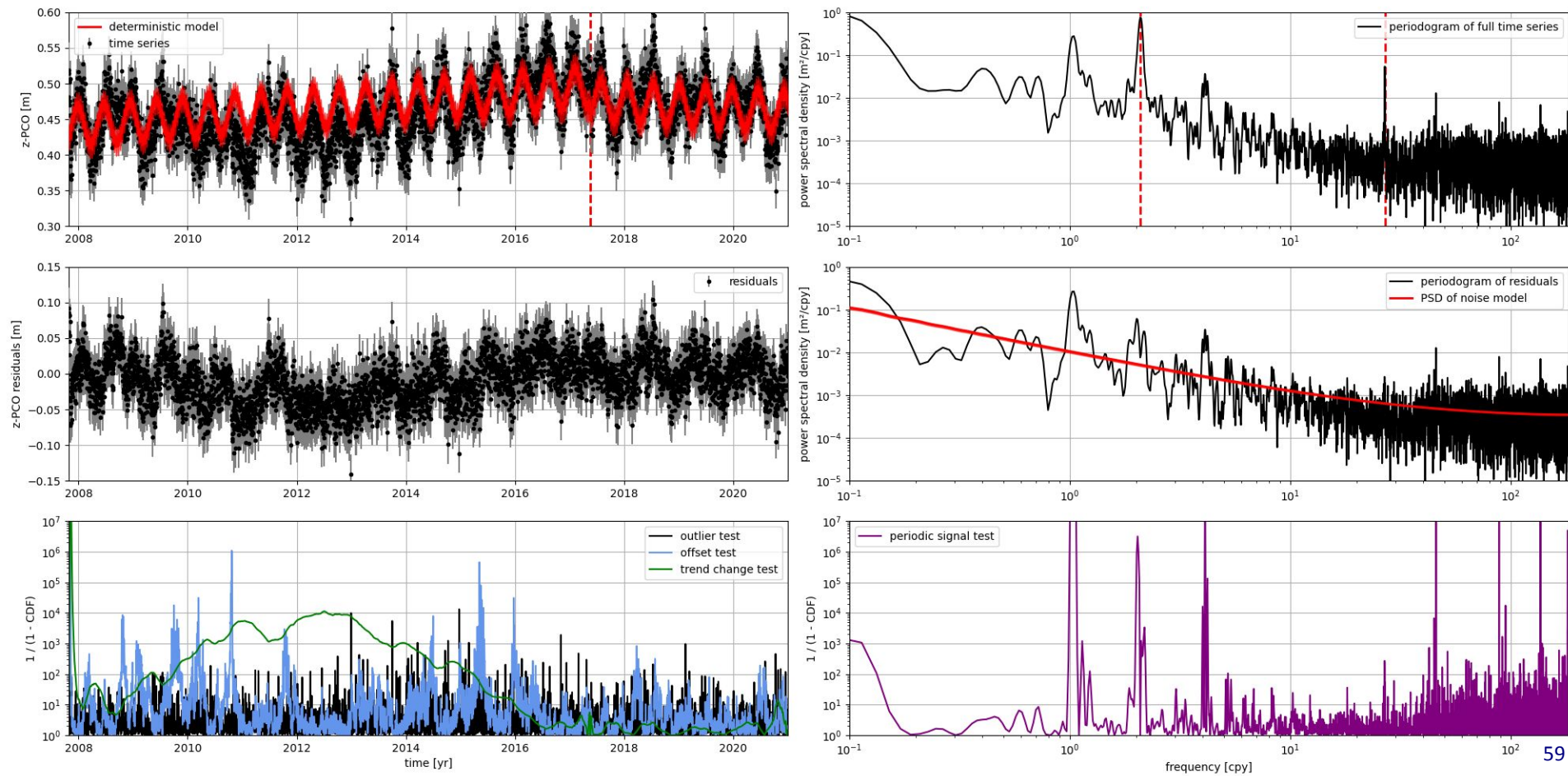
(pytrf/examples/time series analysis/5 - automatic modeling of a GPS satellite z-PCO time series)



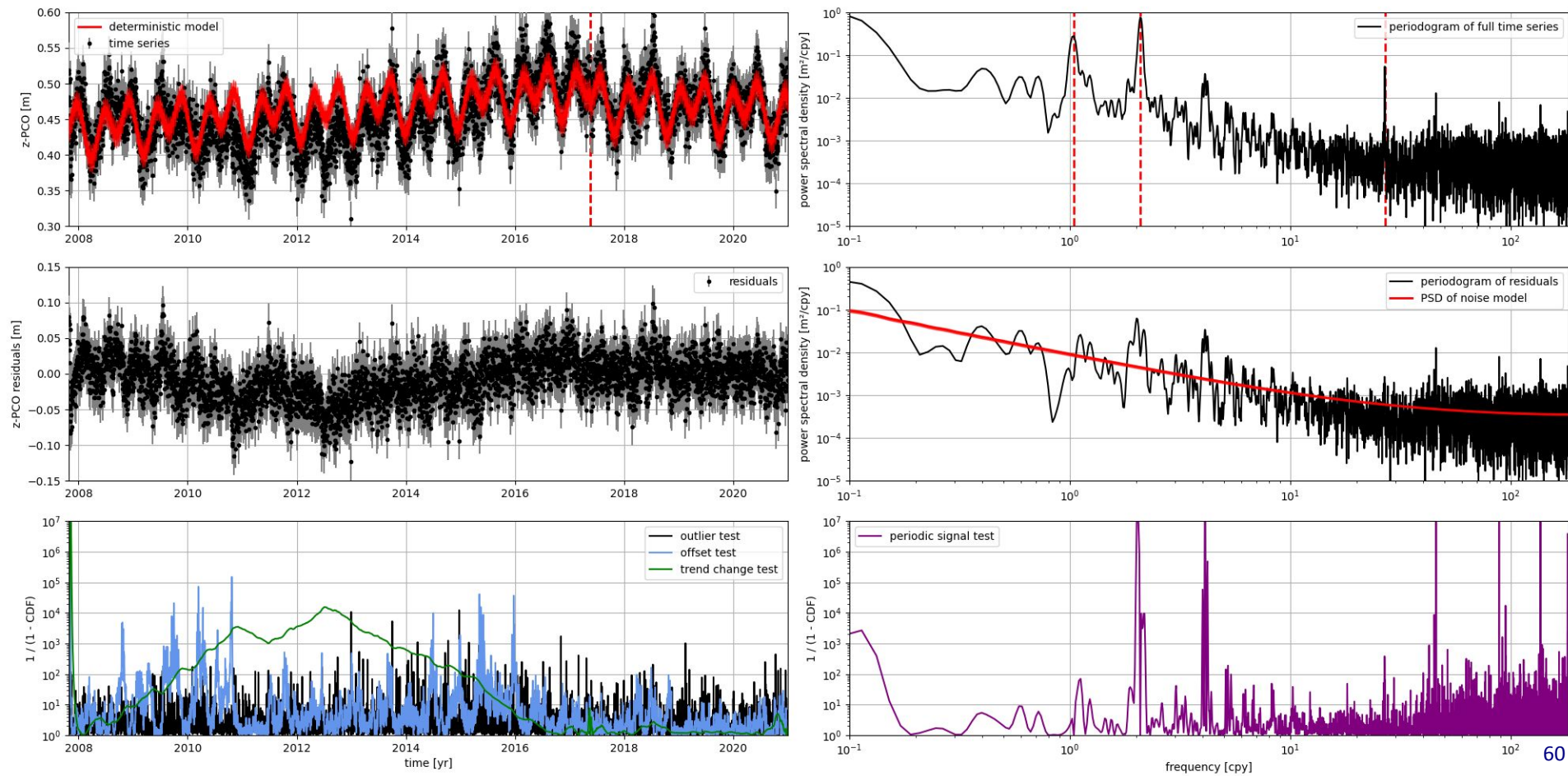
pytrf.ts – Usage example of LR tests: Step 1



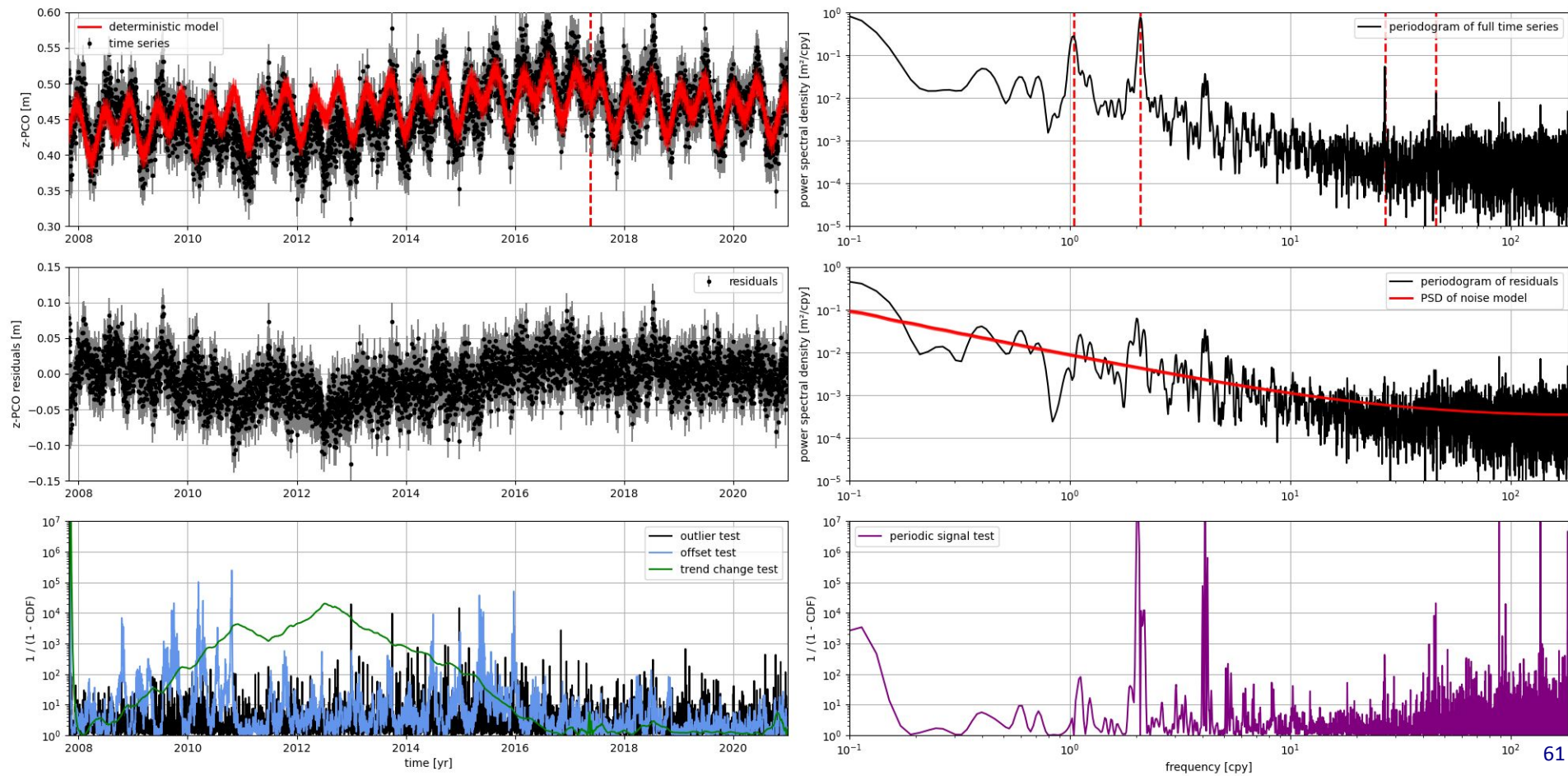
pytrf.ts – Usage example of LR tests: Step 2



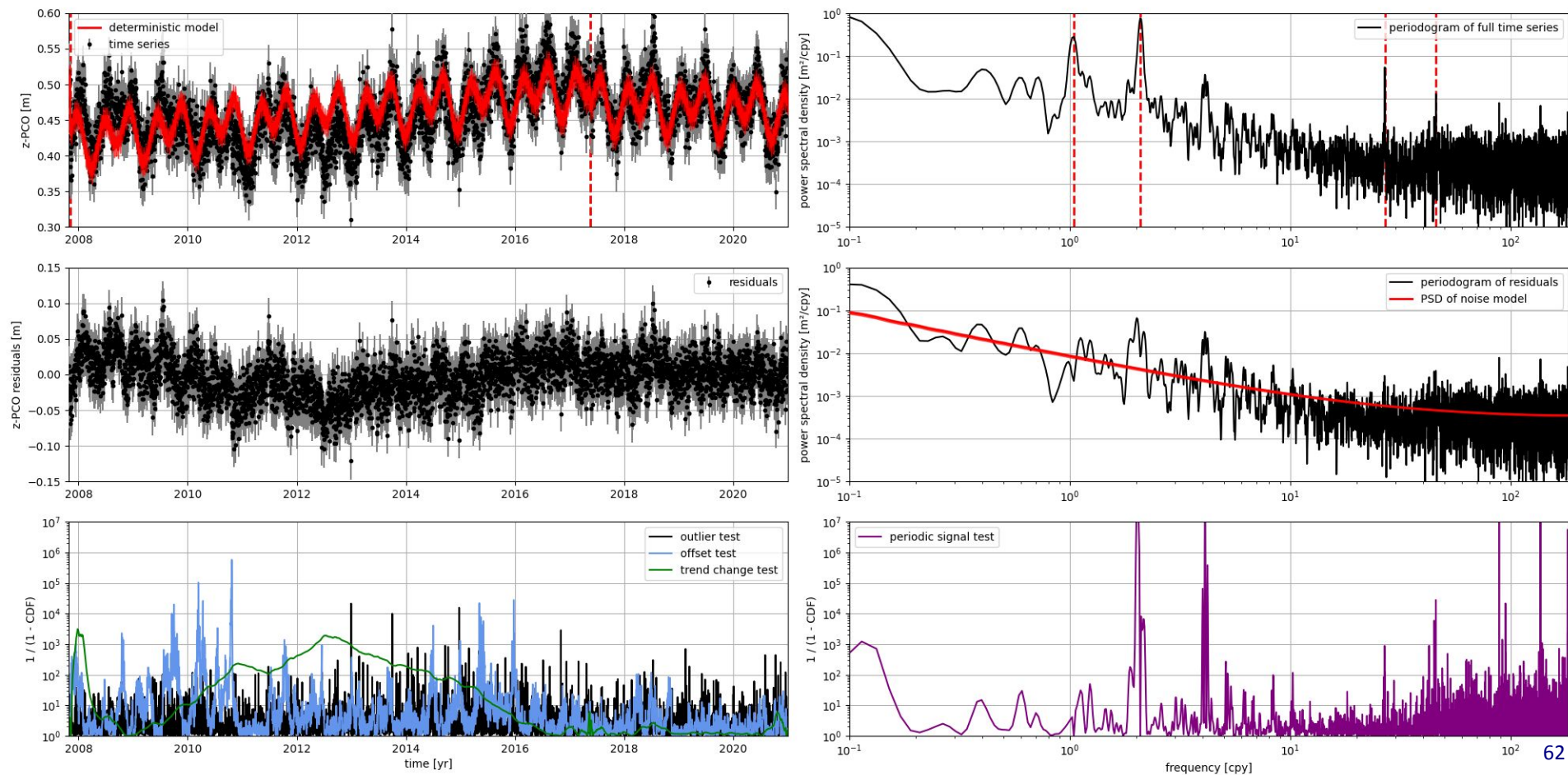
pytrf.ts – Usage example of LR tests: Step 3



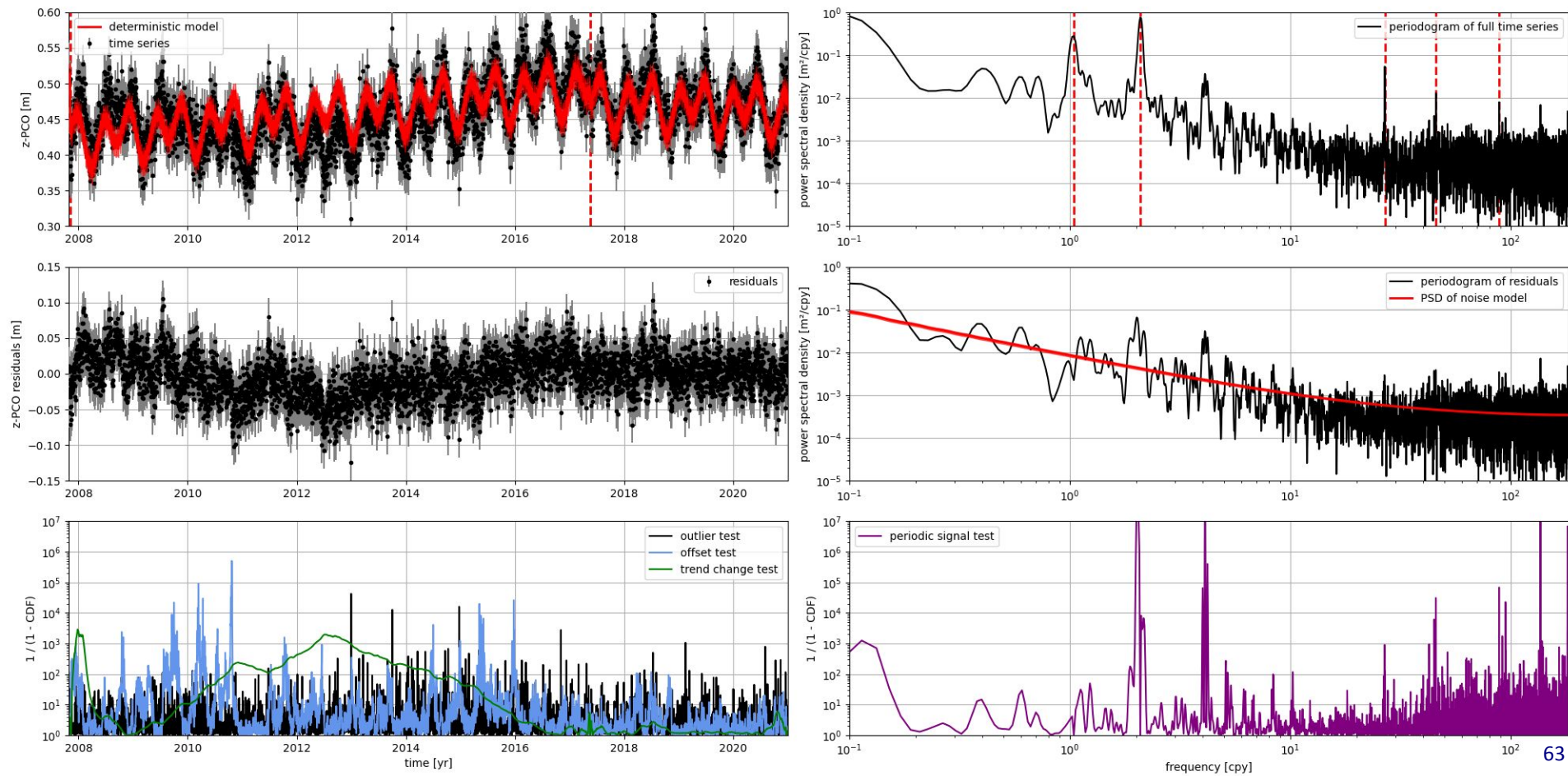
pytrf.ts – Usage example of LR tests: Step 4



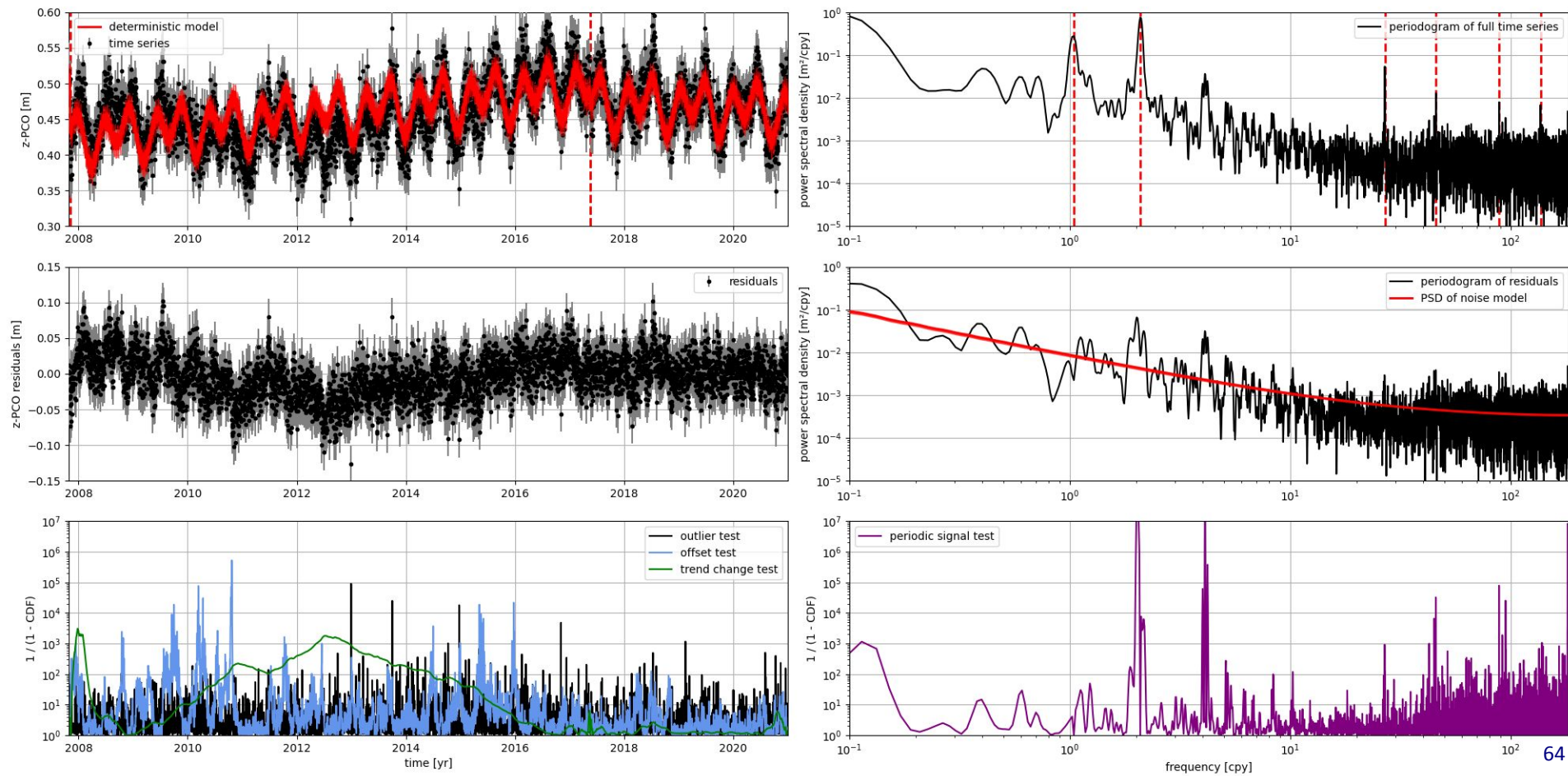
pytrf.ts – Usage example of LR tests: Step 5



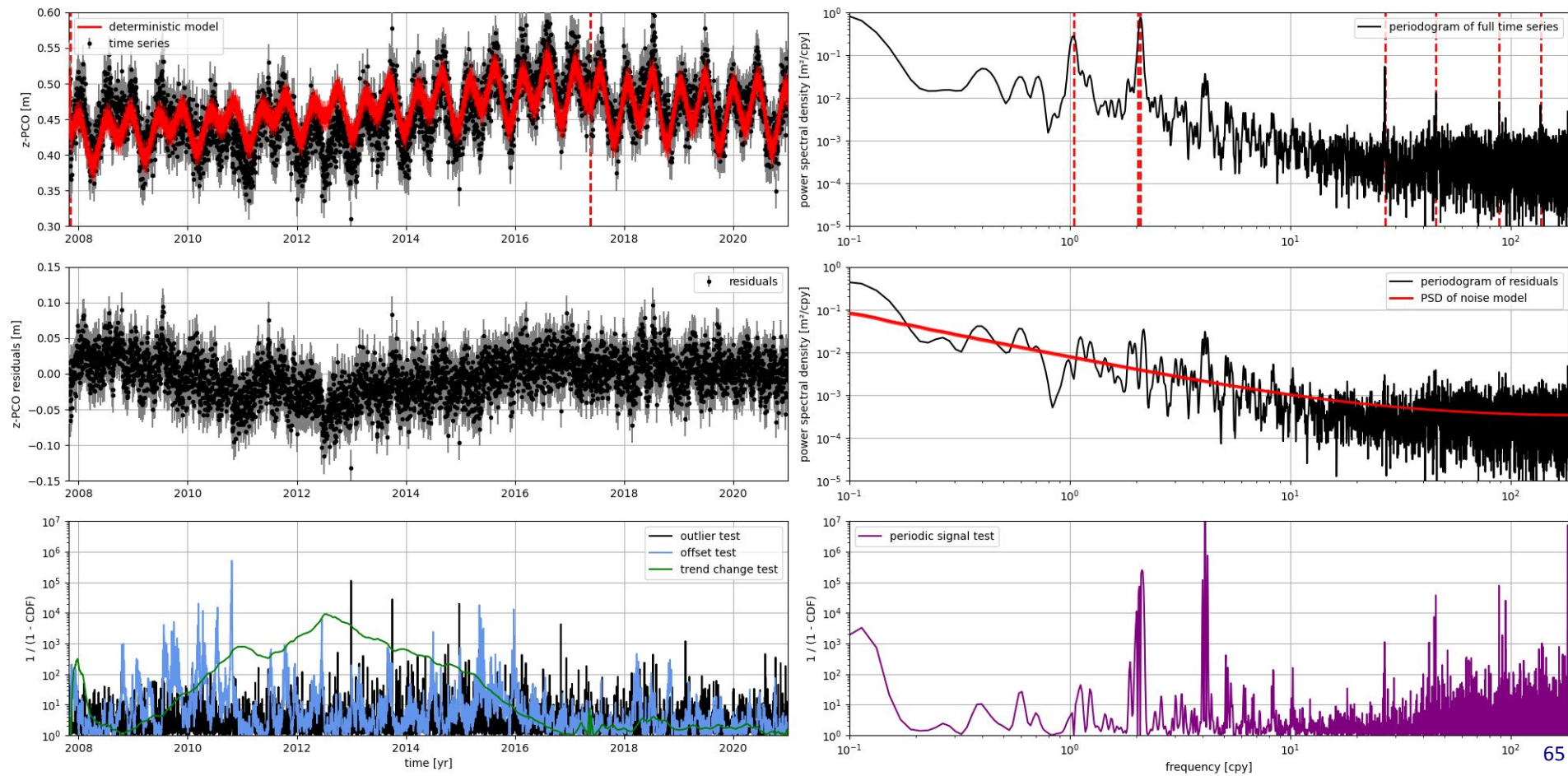
pytrf.ts – Usage example of LR tests: Step 6



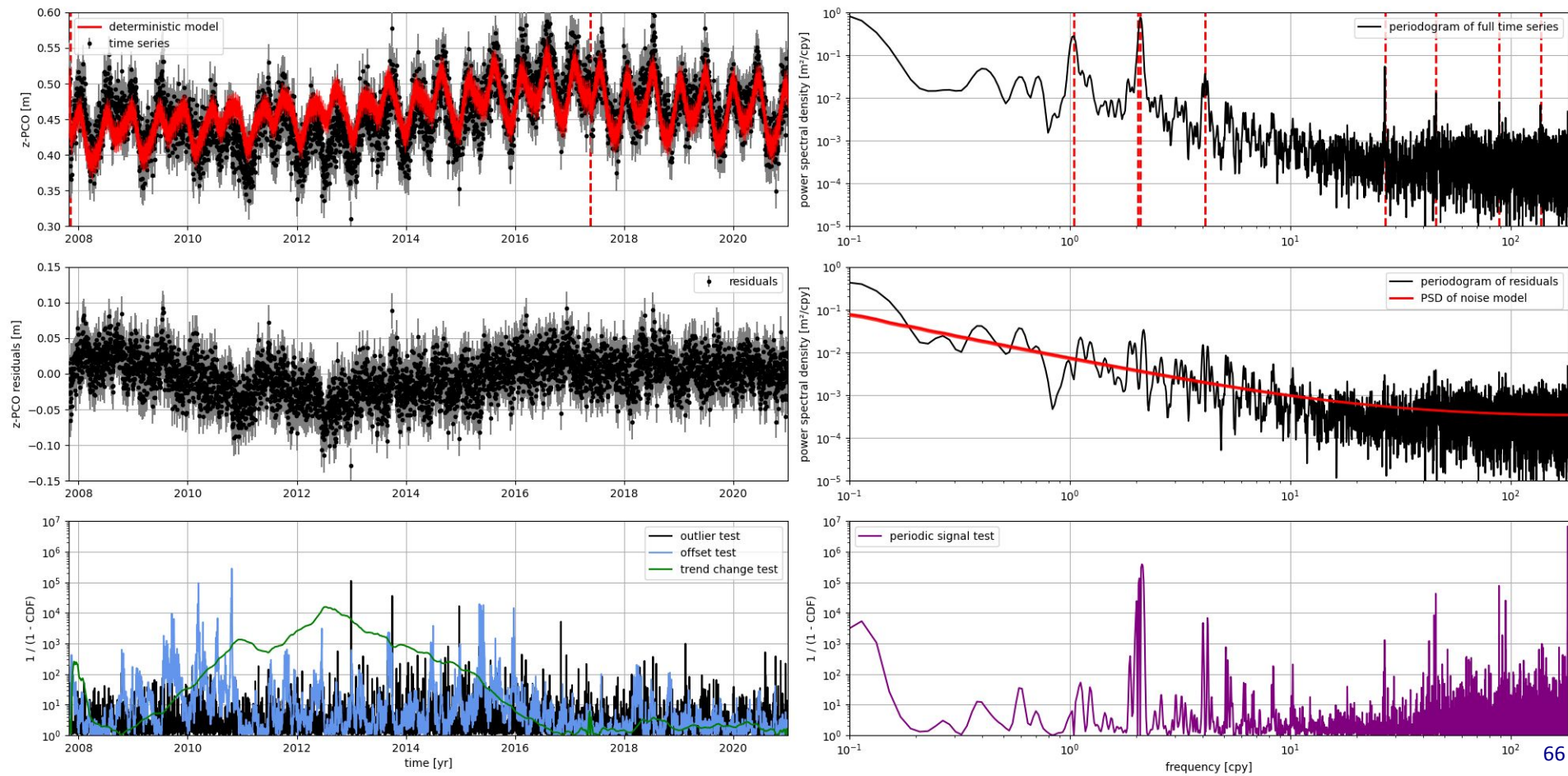
pytrf.ts – Usage example of LR tests: Step 7



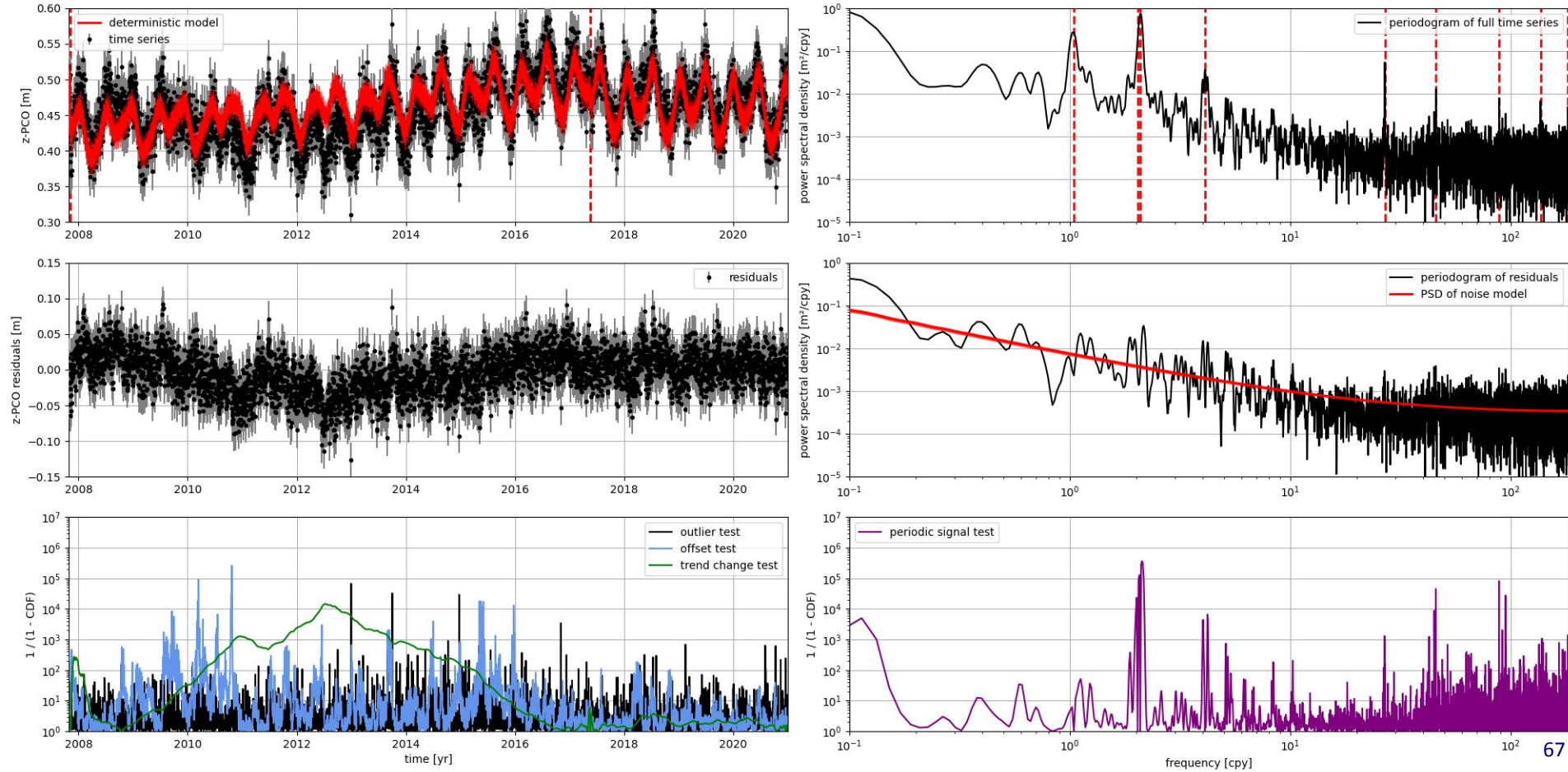
pytrf.ts – Usage example of LR tests: Step 8



pytrf.ts – Usage example of LR tests: Step 9



pytrf.ts – Usage example of LR tests: Step 10



pytrf.ts – Time series simulation

- How to:

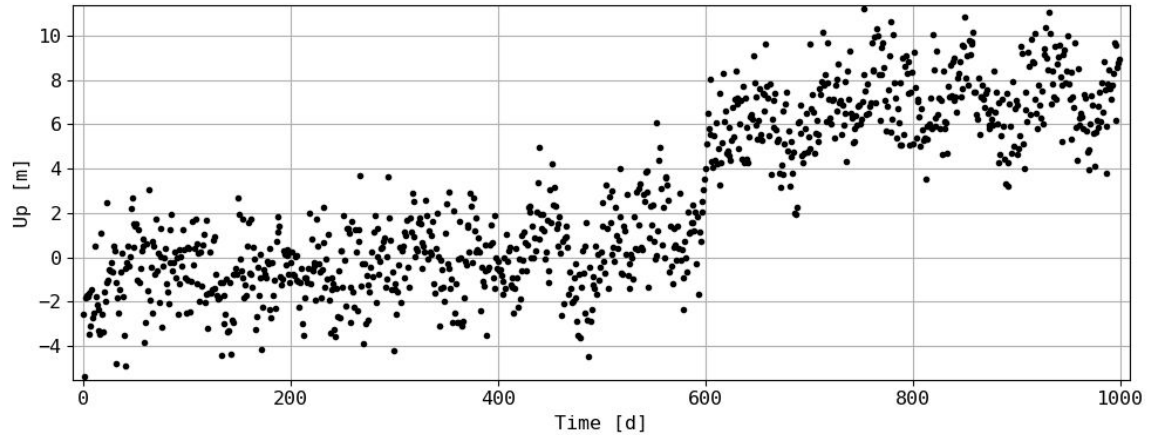
- Create an 'empty' time series.
- Create an associated model with the desired deterministic functions and noise components.
- Assign values to all parameters of the deterministic functions and noise components.
- Call `m.simulate()`.

- Example:

```
$ r = ts(1000, dims=('Up'))

$ m = model(r, deg=[0,1])
$ m.add_jumps(t=[600], deg=[0])
$ m.f[0].par[0].x = -1
$ m.f[0].par[1].x = 3
$ m.f[1].par[0].x = 0.005
$ m.add_wn(s2=1)
$ m.add_pl(s2=1, a=1)
$ m.simulate()

$ r.plot()
```



pytrf.ts – Time series simulation

- How to:

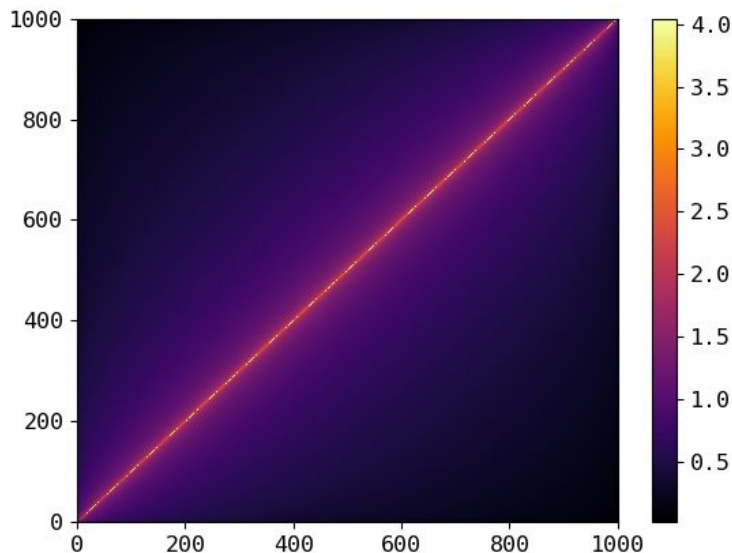
- Create an 'empty' time series.
- Create an associated model with the desired deterministic functions and noise components.
- Assign values to all parameters of the deterministic functions and noise components.
- Call `m.simulate()`.

- Example:

```
$ r = ts(1000, dims=('Up'))

$ m = model(r, deg=[0,1])
$ m.add_jumps(t=[600], deg=[0])
$ m.f[0].par[0].x = -1
$ m.f[0].par[1].x = 3
$ m.f[1].par[0].x = 0.005
$ m.add_wn(s2=1)
$ m.add_pl(s2=1, a=1)
$ m.simulate()

$ import matplotlib.pyplot as pp
$ pp.pcolormesh(m.Q, cmap='inferno')
$ pp.colorbar()
$ pp.show()
```



pytrf.ts – Time series simulation

- How to:

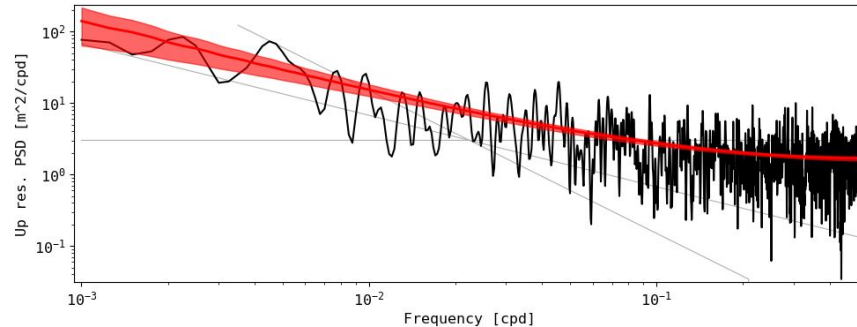
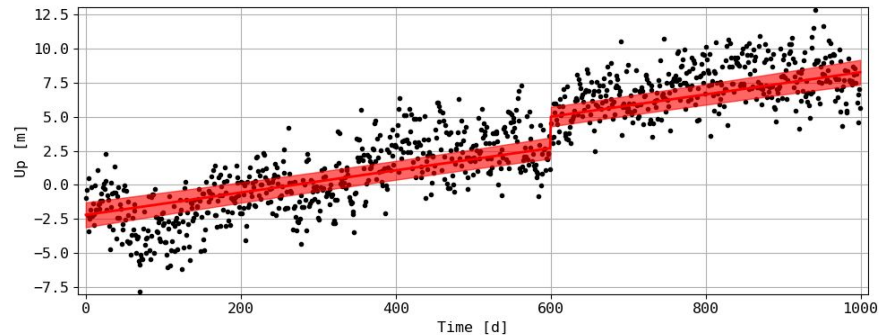
- Create an 'empty' time series.
- Create an associated model with the desired deterministic functions and noise components.
- Assign values to all parameters of the deterministic functions and noise components.
- Call `m.simulate()`.

- Example:

```
$ r = ts(1000, dims=('Up'))

$ m = model(r, deg=[0,1])
$ m.add_jumps(t=[600], deg=[0])
$ m.f[0].par[0].x = -1
$ m.f[0].par[1].x = 3
$ m.f[1].par[0].x = 0.005
$ m.add_wn(s2=1)
$ m.add_pl(s2=1, a=1)
$ m.simulate()

$ m.fit()
$ m.plot_fit()
$ m.plot_psd()
```



pytrf.ts – Save / load model objects

- **model objects can be saved in 'pickle' format, then re-loaded:**

```
$ m.clean()  
$ m.dump('CKSV.pkl')  
$ m = model.load('CKSV.pkl')
```
- **m.clean() deletes all $n \times n$ matrices which m has as attributes.**
 - A useful precaution to take if you have many long series and limited disk space.
 - These matrix attributes can easily be restored if needed, e.g., with **m.set_cov()**.

Part 2 – SINEX analysis & combination: **pytrf.sinex / pytrf.snxcmb modules**

pytrf.sinex / pytrf.snxcmb – Background

- On a daily/weekly/session basis, the Analysis Centers (ACs) of space geodetic techniques:
 - collect a set of observations: y
 - describe them as a function of parameters x : $y = f(x) + \varepsilon \approx A \cdot (x - x_0) + \varepsilon, \quad \text{var}(\varepsilon) = Q$
(x = station positions, ERPs, orbit parameters / radiosource positions, tropospheric parameters, other technique-specific biases)
 - specify constraints wrt a priori parameters: $C \cdot (x - x_0) = 0 + \varepsilon_c, \quad \text{var}(\varepsilon_c) = Q_c$
(e.g., no-net-rotation constraints with respect to ITRF)
 - construct the least-squares normal equation: $(A^T Q^{-1} A + C^T Q_c^{-1} C) \cdot (x - x_0) = A^T Q^{-1} (y - f(x_0)) \quad \leftrightarrow \quad (N + N_c) \cdot (x - x_0) = b$
 - solve for the parameters: $\hat{x} = x_0 + (N + N_c)^{-1} \cdot b, \quad \text{var}(\hat{x}) = Q_{\hat{x}} = (N + N_c)^{-1}$
- Their results are provided in the SINEX format as either:
 - unconstrained normal equations: x_0, N, b
 - constrained solutions: $\hat{x}, Q_{\hat{x}} [, x_0, N_c]$
- Solutions from individual ACs are combined on a daily/weekly/session basis (e.g., IGS, GRSP).
 - Combination $\approx$ weighted average
- Time series of solutions are stacked to form long-term terrestrial frames (e.g., ITRF, GTRF).
 - Stacking $\approx$ adjustment of piecewise linear models to time series of station position solutions

pytrf.sinex / pytrf.snxcmb – Background

- Not all estimated parameters are actually provided in AC SINEX files, but only:
 - station positions,
 - Earth rotation parameters (ERPs) – usually: daily pole coordinates and UT1-UTC at noon, pole rates and LOD,
 - [geocenter coordinates, radiosource coordinates, GNSS satellite PCOs, ...]
- The other estimated parameters are eliminated (or reduced).
 - If the full normal equation reads:
$$\begin{bmatrix} \mathbf{N}_{11} & \mathbf{N}_{12} \\ \mathbf{N}_{21} & \mathbf{N}_{22} \end{bmatrix} \cdot \begin{bmatrix} \delta \mathbf{x}_1 \\ \delta \mathbf{x}_2 \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \end{bmatrix}$$
 - The parameters of interest ($\mathbf{x}_1$) can be obtained by solving the reduced normal equation:
$$(\mathbf{N}_{11} - \mathbf{N}_{12} \mathbf{N}_{22}^{-1} \mathbf{N}_{21}) \cdot \delta \mathbf{x}_1 = \mathbf{b}_1 - \mathbf{N}_{12} \mathbf{N}_{22}^{-1} \mathbf{b}_2 \leftrightarrow \mathbf{N}_r \cdot \delta \mathbf{x}_1 = \mathbf{b}_r$$
 - Eliminated parameters are still 'implicitly' estimated.
 - The estimated parameters of interest and their covariance matrix remain the same as if the whole system was solved.
 - If needed, the eliminated parameters can then be 'backsolved': $\delta \hat{\mathbf{x}}_2 = \mathbf{N}_{22}^{-1} (\mathbf{b}_2 - \mathbf{N}_{21} \delta \hat{\mathbf{x}}_1)$
- It can sometimes be useful to instead fix some parameters to their a priori values.
 - Fixing parameters is achieved by removing rows and columns from the normal equation.
 - Fixing parameters changes the values and covariance matrix of the other estimated parameters.

pytrf.sinex / pytrf.snxcmb – Background

- **Terrestrial reference system (TRS):**

- Ideal orthonormal trihedron co-rotating with the Earth
- Defined by the origin, scale and orientation of its vectors
- Two TRS may differ by a translation, a scale factor and a rotation.

- **(Linearized) Helmert transformation:**

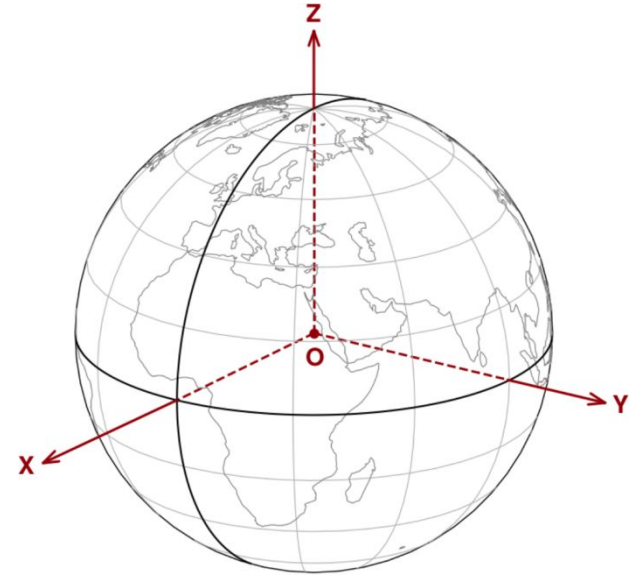
- For some station position:

$$\begin{bmatrix} X_2 \\ Y_2 \\ Z_2 \end{bmatrix} = \begin{bmatrix} X_1 \\ Y_1 \\ Z_1 \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} + d \cdot \begin{bmatrix} X_1 \\ Y_1 \\ Z_1 \end{bmatrix} + \begin{bmatrix} r_x \\ r_y \\ r_z \end{bmatrix} \times \begin{bmatrix} X_1 \\ Y_1 \\ Z_1 \end{bmatrix}$$

- For ERPs:

$$\begin{bmatrix} (x_p)_2 \\ (y_p)_2 \\ (UT1 - UTC)_2 \end{bmatrix} = \begin{bmatrix} (x_p)_1 \\ (y_p)_1 \\ (UT1 - UTC)_1 \end{bmatrix} + \begin{bmatrix} r_x \\ r_y \\ -r_z/f \end{bmatrix}$$

(f = 1.00273781191135448 cycles/day = Earth's rotation speed in space)



- For a whole station positions + ERP solution, in matrix form: $x_2 = x_1 + H \cdot \theta$
- $\theta = [t_x, t_y, t_z, d, r_x, r_y, r_z]^T$ = vector of Helmert parameters
- H = matrix of {station positions + ERP / Helmert parameters} partial derivatives = 'matrix of Helmert partial derivatives'

pytrf.sinex / pytrf.snxcmb – Background

- Space geodetic solutions have rank deficiencies.
 - For all 4 techniques, rotations of the station network can be compensated by variations of ERPs.
 - VLBI is additionally insensitive to translations of the station network.
 - Satellite techniques are insensitive to UT1-UTC.
 - Constraints are necessary to solve for station positions and ERPs.
- Datum definition constraints:
 - Given a set of reference coordinates $\mathbf{x}_0$ (e.g., from ITRF), impose: $(\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T (\mathbf{x} - \mathbf{x}_0) = \mathbf{0} + \boldsymbol{\varepsilon}_c$, $\text{var}(\boldsymbol{\varepsilon}_c) = \mathbf{Q}_c = \sigma_c^2 \cdot \mathbf{I}$
 - σ_c specifies the tightness of the constraints.
 - Contribution to normal matrix: $\mathbf{N}_c = \mathbf{H}(\mathbf{H}^T \mathbf{H})^{-1} \mathbf{Q}_c^{-1} (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T$
 - If Helmert parameters are subsequently estimated between $\hat{\mathbf{x}}$ and $\mathbf{x}_0$, they're found to be $\mathbf{0} \pm \sigma_c$: there's no net translation, scale factor nor rotation between $\hat{\mathbf{x}}$ and $\mathbf{x}_0$.
 - It is possible to apply only a subset of datum definition constraints, e.g., no-net-rotation (NNR) constraints only, by selecting the appropriate rows of $(\mathbf{H}^T \mathbf{H}^{-1}) \mathbf{H}^T$.

pytrf.sinex / pytrf.snxcmb – Background

- Datum information from space geodetic solutions may be unreliable/undesirable.
 - For instance, the origin of GNSS/DORIS solutions is known to be affected by large systematic errors.
 - IGS product users want station positions expressed wrt the ITRF origin, not the spurious 'apparent GNSS geocenter'.
- It is possible to 'isolate' such datum information by reparameterizing station positions & ERPs:
 - $\mathbf{x} = \mathbf{x}' + \mathbf{H} \cdot \boldsymbol{\theta}$
 - $\mathbf{x}'$ = station positions and ERPs in the target reference frame
 - $\mathbf{H}$ = matrix of Helmert partial derivatives (restricted to relevant Helmert parameters)
 - $\boldsymbol{\theta}$ = transformation parameters from original to target reference frame
 - The reparameterized normal equation reads:
$$\begin{bmatrix} \mathbf{N} & \mathbf{NH} \\ \mathbf{H}^T \mathbf{N} & \mathbf{H}^T \mathbf{NH} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{x}' \\ \boldsymbol{\theta} \end{bmatrix} = \begin{bmatrix} \mathbf{b} \\ \mathbf{H}^T \mathbf{b} \end{bmatrix}$$
 - It has as many additional rank deficiencies as Helmert parameters were introduced, which need to be filled by datum definition constraints (i.e., the target reference frame needs to be specified).
 - If the transformation parameters $\boldsymbol{\theta}$ (e.g., the apparent geocenter coordinates) are not of interest, they can be eliminated, leading to the following normal equation:
$$(\mathbf{N} - \mathbf{NH}(\mathbf{H}^T \mathbf{NH})^{-1} \mathbf{H}^T \mathbf{N}) \cdot \mathbf{x}' = \mathbf{b} - \mathbf{NH}(\mathbf{H}^T \mathbf{NH})^{-1} \mathbf{H}^T \mathbf{b}$$

pytrf.sinex / pytrf.snxcmb – Background

- Combination model:

$$\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} J_1 & H_1 & 0 & 0 \\ \vdots & 0 & \ddots & 0 \\ J_n & 0 & 0 & H_n \end{bmatrix} \cdot \begin{bmatrix} x \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \vdots \\ \varepsilon_n \end{bmatrix}, \quad \text{var} \left(\begin{bmatrix} \varepsilon_1 \\ \vdots \\ \varepsilon_n \end{bmatrix} \right) = \begin{bmatrix} \sigma_1^2 Q_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \sigma_n^2 Q_n \end{bmatrix}$$

- $(x_1, Q_1), \dots, (x_n, Q_n)$ input solutions (station positions + ERPs) and their covariance matrices
- x combined solution (to be estimated)
- $J_1, \dots, J_n$ matrices of ones and zeros mapping parameters in input solutions to parameters in combined solution
- $\theta_1, \dots, \theta_n$ transformation parameters from input solution to combined solution (to be estimated)
- $H_1, \dots, H_n$ matrices of Helmert partial derivatives
- $\sigma_1^2, \dots, \sigma_n^2$ corrective variance factors (to be estimated)
- For brevity, $x_1, \dots, x_n$ and x_c do not denote actual station positions and ERPs, but increments to common a priori values.

pytrf.sinex / pytrf.snxcmb – Background

- Combination model:

$$\begin{bmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_n \end{bmatrix} = \begin{bmatrix} \mathbf{J}_1 & \mathbf{H}_1 & 0 & 0 \\ \vdots & 0 & \ddots & 0 \\ \mathbf{J}_n & 0 & 0 & \mathbf{H}_n \end{bmatrix} \cdot \begin{bmatrix} \mathbf{x} \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}, \quad \text{var} \left(\begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix} \right) = \begin{bmatrix} \sigma_1^2 \mathbf{Q}_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \sigma_n^2 \mathbf{Q}_n \end{bmatrix}$$

→ **Normal equation:**
(for given $\sigma_1^2, \dots, \sigma_n^2$)

$$\begin{bmatrix} \sum \mathbf{J}_i^T \mathbf{Q}_i^{-1} \mathbf{J}_i / \sigma_i^2 & \mathbf{J}_1^T \mathbf{Q}_1^{-1} \mathbf{H}_1 / \sigma_1^2 & \dots & \mathbf{J}_n^T \mathbf{Q}_n^{-1} \mathbf{H}_n / \sigma_n^2 \\ \mathbf{H}_1^T \mathbf{Q}_1^{-1} \mathbf{J}_1 / \sigma_1^2 & \mathbf{H}_1^T \mathbf{Q}_1^{-1} \mathbf{H}_1 / \sigma_1^2 & 0 & 0 \\ \vdots & 0 & \ddots & 0 \\ \mathbf{H}_n^T \mathbf{Q}_n^{-1} \mathbf{J}_n / \sigma_n^2 & 0 & 0 & \mathbf{H}_n^T \mathbf{Q}_n^{-1} \mathbf{H}_n / \sigma_n^2 \end{bmatrix} \cdot \begin{bmatrix} \mathbf{x} \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} = \begin{bmatrix} \sum \mathbf{J}_i^T \mathbf{Q}_i^{-1} \mathbf{x}_i / \sigma_i^2 \\ \mathbf{H}_1^T \mathbf{Q}_1^{-1} \mathbf{x}_1 / \sigma_1^2 \\ \vdots \\ \mathbf{H}_n^T \mathbf{Q}_n^{-1} \mathbf{x}_n / \sigma_n^2 \end{bmatrix} \quad \leftrightarrow \quad \mathbf{N} \cdot \begin{bmatrix} \mathbf{x} \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} = \mathbf{b}$$

pytrf.sinex / pytrf.snxcmb – Background

- Combination model:

$$\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} J_1 & H_1 & 0 & 0 \\ \vdots & 0 & \ddots & 0 \\ J_n & 0 & 0 & H_n \end{bmatrix} \cdot \begin{bmatrix} x \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}, \quad \text{var} \left(\begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix} \right) = \begin{bmatrix} \sigma_1^2 Q_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \sigma_n^2 Q_n \end{bmatrix}$$

→ **Normal equation:**
(for given $\sigma_1^2, \dots, \sigma_n^2$)

$$\begin{bmatrix} \sum J_i^T Q_i^{-1} J_i / \sigma_i^2 & \sum J_i^T Q_i^{-1} H_i / \sigma_i^2 & \cdots & \sum J_n^T Q_n^{-1} H_n / \sigma_n^2 \\ H_1^T Q_1^{-1} J_1 / \sigma_1^2 & H_1^T Q_1^{-1} H_1 / \sigma_1^2 & 0 & 0 \\ \vdots & 0 & \ddots & 0 \\ H_n^T Q_n^{-1} J_n / \sigma_n^2 & 0 & 0 & H_n^T Q_n^{-1} H_n / \sigma_n^2 \end{bmatrix} \cdot \begin{bmatrix} x \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} = \begin{bmatrix} \sum J_i^T Q_i^{-1} x_i / \sigma_i^2 \\ H_1^T Q_1^{-1} x_1 / \sigma_1^2 \\ \vdots \\ H_n^T Q_n^{-1} x_n / \sigma_n^2 \end{bmatrix} \quad \leftrightarrow \quad N \cdot \begin{bmatrix} x \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} = b$$

- Not all 7 transformation parameters need to be estimated for every input solution.
- But in case some Helmert parameters are estimated for all input solutions, the combined normal matrix has rank deficiencies that need to be filled by datum definition constraints (e.g., NNR constraints are necessary if rotations are estimated for all input solutions).

→ **Constrained normal equation:**
(for given $\sigma_1^2, \dots, \sigma_n^2$)

$$(N + N_c) \cdot \begin{bmatrix} x \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} = b$$

pytrf.sinex / pytrf.snxcmb – Background

- Combination model:

$$\begin{bmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_n \end{bmatrix} = \begin{bmatrix} \mathbf{J}_1 & \mathbf{H}_1 & 0 & 0 \\ \vdots & 0 & \ddots & 0 \\ \mathbf{J}_n & 0 & 0 & \mathbf{H}_n \end{bmatrix} \cdot \begin{bmatrix} \mathbf{x} \\ \boldsymbol{\theta}_1 \\ \vdots \\ \boldsymbol{\theta}_n \end{bmatrix} + \begin{bmatrix} \boldsymbol{\varepsilon}_1 \\ \vdots \\ \boldsymbol{\varepsilon}_n \end{bmatrix}, \quad \text{var} \left(\begin{bmatrix} \boldsymbol{\varepsilon}_1 \\ \vdots \\ \boldsymbol{\varepsilon}_n \end{bmatrix} \right) = \begin{bmatrix} \sigma_1^2 \mathbf{Q}_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \sigma_n^2 \mathbf{Q}_n \end{bmatrix}$$

→ **Solution:**
(for given $\sigma_1^2, \dots, \sigma_n^2$)

$$\begin{bmatrix} \hat{\mathbf{x}} \\ \hat{\boldsymbol{\theta}}_1 \\ \vdots \\ \hat{\boldsymbol{\theta}}_n \end{bmatrix} = (\mathbf{N} + \mathbf{N}_c)^{-1} \mathbf{b}, \quad \text{var} \left(\begin{bmatrix} \hat{\mathbf{x}} \\ \hat{\boldsymbol{\theta}}_1 \\ \vdots \\ \hat{\boldsymbol{\theta}}_n \end{bmatrix} \right) = (\mathbf{N} + \mathbf{N}_c)^{-1} = \mathbf{Q}$$

→ **Residuals for every input solution:** $\hat{\mathbf{v}}_i = \mathbf{x}_i - \mathbf{J}_i \cdot \hat{\mathbf{x}} - \mathbf{H}_i \cdot \hat{\boldsymbol{\theta}}_i$
(for given $\sigma_1^2, \dots, \sigma_n^2$)

- Given the block-diagonal structure of the observation covariance matrix, it is possible to find the REML estimates of $\sigma_1^2, \dots, \sigma_n^2$ with the following iterative scheme:

$$\sigma_i^2 \leftarrow \sigma_i^2 \times \frac{\hat{\mathbf{v}}_i^T \mathbf{Q}_i^{-1} \hat{\mathbf{v}}_i}{n_i - \text{tr}(\mathbf{Q} \mathbf{N}_i)}$$

where n_i is the number of parameters in solution i ,
and $\mathbf{N}_i$ is the contribution of solution i to the combined normal matrix.

pytrf.sinex / pytrf.snxcmb – Background

- Long-term stacking model:

$$\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} J_1 & (t_1 - t_0) \cdot J_1 & H_1 & 0 & 0 \\ \vdots & \vdots & 0 & \ddots & 0 \\ J_n & (t_n - t_0) \cdot J_n & 0 & 0 & H_n \end{bmatrix} \cdot \begin{bmatrix} x \\ \dot{x} \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}, \quad \text{var} \left(\begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix} \right) = \sigma_0^2 \cdot \begin{bmatrix} Q_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & Q_n \end{bmatrix}$$

- $(x_1, Q_1), \dots, (x_n, Q_n)$ time series of input solutions (station positions) and their covariance matrices
- x combined station positions at reference epoch t_0 (to be estimated)
- $\dot{x}$ combined station velocities (to be estimated)
- $J_1, \dots, J_n$ matrices of ones and zeros mapping parameters in input solutions to parameters in long-term solution
- $\theta_1, \dots, \theta_n$ transformation parameters from input solution to long-term solution (to be estimated)
- $H_1, \dots, H_n$ matrices of Helmert partial derivatives
- σ_0^2 single corrective variance factor (to be estimated)
(It is assumed that all input solutions stem from a homogeneous processing strategy.)
- For brevity, $x_1, \dots, x_n$ and x do not denote actual station positions, but increments to common a priori values.

pytrf.sinex / pytrf.snxcmb – Background

- Long-term stacking model:

$$\begin{bmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_n \end{bmatrix} = \begin{bmatrix} \mathbf{J}_1 & (\mathbf{t}_1 - \mathbf{t}_0) \cdot \mathbf{J}_1 & \mathbf{H}_1 & 0 & 0 \\ \vdots & \vdots & 0 & \ddots & 0 \\ \mathbf{J}_n & (\mathbf{t}_n - \mathbf{t}_0) \cdot \mathbf{J}_n & 0 & 0 & \mathbf{H}_n \end{bmatrix} \cdot \begin{bmatrix} \mathbf{x} \\ \dot{\mathbf{x}} \\ \boldsymbol{\theta}_1 \\ \vdots \\ \boldsymbol{\theta}_n \end{bmatrix} + \begin{bmatrix} \boldsymbol{\varepsilon}_1 \\ \vdots \\ \boldsymbol{\varepsilon}_n \end{bmatrix}, \quad \text{var} \left(\begin{bmatrix} \boldsymbol{\varepsilon}_1 \\ \vdots \\ \boldsymbol{\varepsilon}_n \end{bmatrix} \right) = \sigma_0^2 \cdot \begin{bmatrix} \mathbf{Q}_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \mathbf{Q}_n \end{bmatrix}$$

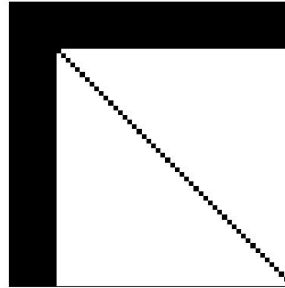
- In case some Helmert parameters are estimated for all input solutions, the stacked normal matrix has rank deficiencies that need to be filled by datum definition constraints.
- For instance, if rotations are estimated for all input solutions, the stacked normal matrix has **6** singularities.
 - Its orientation needs to be defined by NNR constraints wrt to some reference frame.
 - Its orientation **rate** needs to be defined by NNR **rate** constraints wrt to some reference frame.
- Datum **rate** definition constraints have the same form as datum definition constraints, except that they apply to station **velocities**: $(\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T (\dot{\mathbf{x}} - \dot{\mathbf{x}}_0) = 0 + \boldsymbol{\varepsilon}_c, \quad \text{var}(\boldsymbol{\varepsilon}_c) = \mathbf{Q}_c$

pytrf.sinex / pytrf.snxcmb – Background

•Long-term stacking model:

$$\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} J_1 & (t_1 - t_0) \cdot J_1 & H_1 & 0 & 0 \\ \vdots & \vdots & 0 & \ddots & 0 \\ J_n & (t_n - t_0) \cdot J_n & 0 & 0 & H_n \end{bmatrix} \cdot \begin{bmatrix} x \\ \dot{x} \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}, \quad \text{var} \left(\begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix} \right) = \sigma_0^2 \cdot \begin{bmatrix} Q_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & Q_n \end{bmatrix}$$

- Both the parameters $x, \dot{x}, \theta_1, \dots, \theta_n$ and the variance factor σ_0^2 can be estimated in a single run.
- Given the structure of the normal matrix, it is very efficient to:
 1. Eliminate the transformation parameters and solve only for $x, \dot{x}$,
 2. Backsolve the transformation parameters $\theta_1, \dots, \theta_n$
 3. Compute the residuals and the global variance factor



pytrf.sinex / pytrf.snxcmb – SINEX format

- Software/technique independent exchange format for space geodesy solutions
<https://www.iers.org/IERS/EN/Organization/AnalysisCoordinator/SinexFormat/sinex.html>
- Example:

```
%=SNX 2.02 IGN 21:098:86161 IGN 20:264:00000 20:265:00000 C 2780 2 S E
*-----
+FILE/REFERENCE
DESCRIPTION      IGS combination of daily AC repro3 solutions
OUTPUT           Daily combined repro3 solution for week 2124, day 0
CONTACT          igs-rf@ign.fr
SOFTWARE         CATREF
HARDWARE         Linux, x86_64
INPUT            Daily AC repro3 solutions for week 2124, day 0
-FILE/REFERENCE
*-----
+INPUT/ACKNOWLEDGEMENTS
*AGY _____ FULL_DESCRIPTION_____
COD Centre for Orbit Determination in Europe, Bern, Switzerland
ESA European Space Agency, Darmstadt, Germany
GFZ Geoforschungszentrum, Potsdam, Germany
GRG Groupe de Recherche en Geodesie Spatiale, Toulouse, France
JPL Jet Propulsion Laboratory, Pasadena, California, USA
MIT Massachusetts Institute of Technology, Cambridge, Massachusetts, USA
NGS National Geodetic Survey, Silver Springs, Maryland, USA
TUG Graz University of Technology, Graz, Austria
ULR Université de la Rochelle, la Rochelle, France
WHU Wuhan University, Wuhan, Hubei, China
-INPUT/ACKNOWLEDGEMENTS
*-----
+INPUT/HISTORY
*_VERSION_ CRE _CREATION_ OWN _DATA_START_ _DATA_END_ T PARAM S _TYPE_
+SNX 2.01 COD 21:022:08143 COD 20:264:00000 20:265:00000 P 711 1 S E A
+SNX 2.02 ESA 21:013:51219 ESA 20:264:00000 20:264:86382 P 456 2 S E
+SNX 2.02 GFZ 20:359:53460 GFZ 20:264:00000 20:265:00000 P 657 1 S E A
+SNX 2.00 GRG 21:008:85496 GRG 20:264:00000 20:265:00000 C 459 1 E S A - -
[...]
```

← Creator, creation date, data span, number and types of parameters

← Header lines of SINEX files contributing to this SINEX file

pytrf.sinex / pytrf.snxcmb – SINEX format

- Software/technique independent exchange format for space geodesy solutions
<https://www.iers.org/IERS/EN/Organization/AnalysisCoordinator/SinexFormat/sinex.html>
- Example (continued):

```
+INPUT/FILES
*OWN  __CREATION__  _____FILENAME_____  DESCRIPTION_____
COD  21:022:08143  CODOR03FIN_20202640000_01D_01  Daily AC repro3 solution
ESA  21:013:51219  ESA3R03FIN_20202640000_01D_01  Daily AC repro3 solution
GFZ  20:359:53460  GFZ2R03FIN_20202640000_01D_01  Daily AC repro3 solution
GRG  21:008:85496  GRG6RE3FIN_20202640000_01D_00  Daily AC repro3 solution
JPL  21:048:49505  JPLOR3TFIN_20202640000_01D_01  Daily AC repro3 solution
MIT  21:046:72516  MITOR03GE-_20202640000_01D_01  Daily AC repro3 solution
NOA  21:024:47036  NGSOR03FIN_20202640000_01D_01  Daily AC repro3 solution
TUG  20:346:00996  TUGOR03FIN_20202640000_01D_01  Daily AC repro3 solution
ULR  21:093:53562  ULROR03TST_20202640000_01D_01  Daily AC repro3 solution
-INPUT/FILES
*-----
+SOLUTION/STATISTICS
*  __STATISTICAL_PARAMETER__  _____VALUE(S)_____
NUMBER OF OBSERVATIONS                8407
NUMBER OF UNKNOWNNS                   2844
VARIANCE FACTOR                        9.9999993323970071e-01
-SOLUTION/STATISTICS
*-----
+SITE/ID
*CODE PT  __DOMES__  T  __STATION DESCRIPTION__  __LONGITUDE__  __LATITUDE__  HEIGHT__
AB48  A  70208M001  P  PortAlexanAK2005, ALAS  225 21 10.8  56 14 42.2    5.3
AB51  A  49233M001  P  PetersburgAK2005, UNIT  227 05 11.2  56 47 51.4   76.0
ABMF  A  97103M001  P  Les Abymes - Raizet ai  298 28 20.9  16 15 44.3  -25.6
ABPO  A  33302M001  P  Ambohimpanompo, MADAGA  47 13 45.2 -19  1  5.9  1553.0
AC25  A  70206M001  P  King_Cove_AK2005, ALAS  197 41  9.4  55 05 20.3  584.3
AC40  A          M    P  PortHeidenAK2007          201 22 53.1  56 55 49.3   39.9
AC58  A  49257M001  P  StPaulIsldAK2008, UNIT  189 46 55.8  57  9 21.9   19.8
AC60  A  49260M001  P  Westeast_AK2008, UNIT  174  4 34.6  52 42 52.6   18.7
AC79  A          M    P  Montague2_AK2010          212 35 49.0  59 59 52.3  287.8
[...]
```

← List of stations in this solution

pytrf.sinex / pytrf.snxcmb – SINEX format

- Software/technique independent exchange format for space geodesy solutions
<https://www.iers.org/IERS/EN/Organization/AnalysisCoordinator/SinexFormat/sinex.html>
- Example (continued):

+SITE/RECEIVER

*CODE	PT	SOLN	T	DATA_START	DATA_END	RECEIVER_TYPE	S/N	FIRMWARE
AB48	A	----	P	19:304:02760	00:000:00000	TRIMBLE NETR9	5303K	5.42
AB51	A	----	P	17:156:72000	00:000:00000	TRIMBLE NETRS	45032	1.3-2
ABMF	A	----	P	20:038:36000	00:000:00000	SEPT POLARX5	45014	5.3.2

[...]

-SITE/RECEIVER

+SITE/ANTENNA

*CODE	PT	SOLN	T	DATA_START	DATA_END	ANTENNA_TYPE	S/N	DAZ
AB48	A	----	P	05:278:00000	00:000:00000	TRM29659.00	SCIT 02203	0
AB51	A	----	P	05:273:00000	00:000:00000	TRM29659.00	SCIT 02203	0
ABMF	A	----	P	12:024:43200	00:000:00000	TRM57971.00	NONE 14411	0

[...]

-SITE/ANTENNA

+SITE/ECCENTRICITY

*CODE	PT	SOLN	T	DATA_START	DATA_END	REF	DX_U	DX_N	DX_E
AB48	A	----	P	05:278:00000	00:000:00000	UNE	0.0083	0.0000	0.0000
AB51	A	----	P	05:273:00000	00:000:00000	UNE	0.0083	0.0000	0.0000
ABMF	A	----	P	09:288:72000	00:000:00000	UNE	0.0000	0.0000	0.0000

[...]

-SITE/ECCENTRICITY

+SOLUTION/EPOCHS

*CODE	PT	SOLN	T	DATA_START	DATA_END	MEAN_EPOCH
AB48	A	1	P	20:264:00000	20:264:86370	20:264:43184
AB51	A	1	P	20:264:00000	20:264:86399	20:264:43199
ABMF	A	3	P	20:264:00000	20:265:00000	20:264:43200

[...]

← Receivers equipping GNSS stations in this solution

← Antennas equipping GNSS stations in this solution

← Eccentricities of GNSS stations in this solution
(vectors from ground marker to antenna reference point)

← Observation span of each station in this solution

pytrf.sinex / pytrf.snxcmb – SINEX format

- Software/technique independent exchange format for space geodesy solutions
<https://www.iers.org/IERS/EN/Organization/AnalysisCoordinator/SinexFormat/sinex.html>
- Example (continued):

+SOLUTION/APRIORI

*INDEX	_TYPE_	CODE	PT	SOLN	_REF_EPOCH_	UNIT	S	_APRIORI_VALUE_	_STD_DEV_
1	STAX	AB48	A	1	20:264:43200	m	2	-2.49625625884401e+06	0.00000e+00
2	STAY	AB48	A	1	20:264:43200	m	2	-2.52720621940780e+06	0.00000e+00
3	STAZ	AB48	A	1	20:264:43200	m	2	5.27965637222698e+06	0.00000e+00
4	STAX	AB51	A	1	20:264:43200	m	2	-2.38374980455932e+06	0.00000e+00
5	STAY	AB51	A	1	20:264:43200	m	2	-2.56400275148965e+06	0.00000e+00
6	STAZ	AB51	A	1	20:264:43200	m	2	5.31365610010915e+06	0.00000e+00

[...]

2773	XPO	----	--	1	20:264:43200	mas	2	2.05175464374027e+02	0.00000e+00
2774	XPOR	----	--	1	20:264:43200	ma/d	2	-9.82524748724924e-01	0.00000e+00
2775	YPO	----	--	1	20:264:43200	mas	2	3.36600075564477e+02	0.00000e+00
2776	YPOR	----	--	1	20:264:43200	ma/d	2	-1.25383328791316e+00	0.00000e+00
2777	LOD	----	--	1	20:264:43200	ms	2	2.24175713747306e-01	0.00000e+00

-SOLUTION/APRIORI

*

+SOLUTION/ESTIMATE

*INDEX	_TYPE_	CODE	PT	SOLN	_REF_EPOCH_	UNIT	S	_ESTIMATED_VALUE_	_STD_DEV_
1	STAX	AB48	A	1	20:264:43200	m	2	-2.49625625990049e+06	2.37033e-03
2	STAY	AB48	A	1	20:264:43200	m	2	-2.52720621933855e+06	2.12452e-03
3	STAZ	AB48	A	1	20:264:43200	m	2	5.27965637211122e+06	3.58722e-03
4	STAX	AB51	A	1	20:264:43200	m	2	-2.38374980620696e+06	7.55466e-04
5	STAY	AB51	A	1	20:264:43200	m	2	-2.56400275052014e+06	7.83024e-04
6	STAZ	AB51	A	1	20:264:43200	m	2	5.31365609844983e+06	1.52387e-03

[...]

2773	XPO	----	--	1	20:264:43200	mas	2	2.05194313261482e+02	2.34840e-03
2774	XPOR	----	--	1	20:264:43200	ma/d	2	-9.09565408058633e-01	9.15913e-03
2775	YPO	----	--	1	20:264:43200	mas	2	3.36581928588074e+02	2.27872e-03
2776	YPOR	----	--	1	20:264:43200	ma/d	2	-1.23702812479079e+00	9.20246e-03
2777	LOD	----	--	1	20:264:43200	ms	2	2.26022482942283e-01	6.03571e-04

-SOLUTION/ESTIMATE

[...]

← A priori station positions

← A priori ERPs

← Estimated station positions

← Estimated ERPs

pytrf.sinex / pytrf.snxcmb – SINEX format

- Software/technique independent exchange format for space geodesy solutions
<https://www.iers.org/IERS/EN/Organization/AnalysisCoordinator/SinexFormat/sinex.html>
- Example (continued):

+SOLUTION/MATRIX_APRIORI L INFO

*PARA1	PARA2	PARA2+0	PARA2+1	PARA2+2
211	211	1.64719894812436e+07		
212	211	2.61927296112524e+06	6.79626489536959e+06	
213	211	8.83000767364802e+05	-3.20453069995874e+06	1.48581533281581e+07
223	211	-3.38198458347671e+06	-1.01211448412945e+06	-1.73693833042787e+06
223	223	1.29315175398285e+07		
224	211	1.87404449208683e+06	2.40863268365947e+06	7.34476309160195e+06
224	223	-9.26772602283201e+05	1.03323031768382e+07	
225	211	9.51238450188549e+05	1.49304608193740e+06	1.07074530894765e+06
225	223	-2.19909693403929e+06	-3.14566344383775e+06	6.27199883484230e+06

[...]

-SOLUTION/MATRIX_APRIORI L INFO

*-----

+SOLUTION/MATRIX_ESTIMATE L COVA

*PARA1	PARA2	PARA2+0	PARA2+1	PARA2+2
1	1	1.56217426548580e-06		
2	1	2.77368525248460e-07	4.31514124480569e-07	
3	1	1.77504625627534e-06	4.00359675192761e-07	3.45534503709385e-06
4	1	1.20135946289585e-07	2.37877143950696e-08	1.27424322353266e-07
4	4	1.89289480015438e-06		
5	1	2.27777841976814e-08	5.49493791973885e-08	3.82107338078569e-08
5	4	5.07250451424065e-07	5.70529696573815e-07	
6	1	1.27364941390669e-07	3.82769552018864e-08	3.12703674816202e-07
6	4	2.14489374729832e-06	7.00675450641885e-07	4.03271630635318e-06

[...]

-SOLUTION/MATRIX_ESTIMATE L COVA

%ENDSNX

← Normal matrix of constraints

← Covariance matrix of estimated parameters

pytrf.sinex / pytrf.snxcmb – Discontinuities

- In a long-term solution such as the ITRF, station trajectories are described by piecewise linear models [+post-seismic deformation models] [+seasonal terms].
 - Each segment of a piecewise linear model is identified by a solution number or 'soln'.
 - The validity periods of 'solns' are given in a pseudo-SINEX file:
 - Example: [soln_IGc20.snxcmb](#)

```
%=SNX 2.02 IGN 22:201:43340 IGN 94:002:00000 21:001:00000 P 0 1 X V
```

```
*
```

```
+FILE/COMMENT
```

```
Discontinuity list to be used with IGc20.snxcmb
```

```
-FILE/COMMENT
```

```
*
```

```
+SOLUTION/DISCONTINUITY
```

```
ABMF A 1 P 00:000:00000 09:288:72000 P - antenna change
ABMF A 2 P 09:288:72000 12:024:43200 P - antenna change
ABMF A 3 P 12:024:43200 15:118:00000 P - receiver change
ABMF A 4 P 15:118:00000 00:000:00000 P -
ABMF A 1 P 00:000:00000 00:000:00000 V -
```

```
*
```

```
ABPO A 1 P 00:000:00000 08:339:00000 P - unknown
ABPO A 2 P 08:339:00000 18:135:27000 P - antenna change
ABPO A 3 P 18:135:27000 00:000:00000 P -
ABPO A 1 P 00:000:00000 08:339:00000 V - unknown
ABPO A 2 P 08:339:00000 00:000:00000 V -
```

```
[...]
```

← 3 position discontinuities (but no velocity discontinuity)

← 1 position+velocity discontinuity
followed by 1 position-only discontinuity

- Such a discontinuity list can be read with `io.read_solns()`:

```
$ from pytrf.io import read_solns
$ solns = read_solns('soln_IGc20.snxcmb')
```

pytrf.sinex – Read SINEX file into sinex object

```
$ from pytrf import sinex  
$ snx = sinex.read('IGS1R03SNX_20202640000_07D_07D_SOL.SNX')
```

- **Some attributes of `sinex` objects:**

- `snx.file` SINEX file name
- `snx.sta` List of station objects (contains the contents of the SITE/ID, SITE/RECEIVER, SITE/ANTENA, SITE/ECCENTRICITY and SOLUTION/EPOCHS blocks)
- `snx.npar` Number of parameters
- `snx.param` List of parameter objects (contains the content of the SOLUTION/ESTIMATE block – except parameter values and sigmas)
- `snx.x` Array of estimated parameter values
- `snx.sig` Array of estimated parameter formal errors (sigmas)
- `snx.x0` Array of a priori parameter values
- `snx.sig0` Array of a priori parameter formal errors (sigmas)
- `snx.Q` Covariance matrix of estimated parameters
- `snx.N` Normal matrix (without constraints)
- `snx.Nc` Normal matrix of constraints
- `snx.b` Right-hand side of normal equation

- **`sinex.read()` does not only read the SINEX file, but also:**

- Attempts to construct `snx.Nc` from whatever a priori information is available (or sets `snx.Nc` to zero).
- Sorts parameters
- Sets several arrays of indices for future fast parameter search

pytrf.sinex – Load sinex object from pickle file

- Python is slow at reading/writing large files.
- If you plan on using a particular SINEX file (e.g., IGc20.snx) many times, better:
 - Read it and dump it into a pickle file once and for all:

```
$ ref = sinex.read('IGS20.snx')  
$ ref.dump('IGS20.pkl')
```
 - Then load the pickle file when needed (much, much faster than to read the SINEX file again):

```
$ ref = sinex.load('IGS20.pkl')
```
- Whenever you need to store intermediate **sinex** objects, e.g., the inputs of a stacking, always use pickle files, never SINEX files.
- The SINEX format should only be used as an exchange format with the outside, not within your processing.

pytrf.sinex – Some sinex object methods

```
$ write()           : Write sinex instance into SINEX file
$ dump()            : Dump sinex instance into pickle file

$ check_staid()      : Check station PT codes and DOMES numbers
$ check_solns()      : Check solution numbers (solns) in an "instantaneous" solution
$ check_epochs()     : Check parameter reference epochs and SOLUTION/EPOCHS block in an "instantaneous" solution
$ check_metadata()   : Check receivers, antennas and eccentricities against site logs

$ unconstrain()      : Recover unconstrained normal equation
$ clear_const()      : Clear normal matrix of constraints

$ fix_params()       : Fix parameters of specified types in a normal equation
$ del_params()       : Delete (eliminate) parameters of specified types
$ del_sta()          : Delete (eliminate) specified stations
$ del_helmerts()     : Delete (eliminate) origin, scale and/or orientation information in a normal equation

$ add_mc()           : Add NNR, NNT and/or NNS constraints to normal matrix of constraints
$ neqinv()           : Invert (constrained) normal equation

$ compare()          : Helmert comparison between two solutions
$ compare_iter()     : Iterative Helmert comparison between two solutions

$ trim_solns()       : Delete (eliminate) solns that are not relevant for specified date
$ propagate()        : Propagate station positions to specified date using station velocities
$ add_psd()          : Add or remove post-seismic deformation models to a solution
$ add_seas()         : Add seasonal signals to a solution
```

pytrf.sinex – Prepare 'instantaneous' reference frame

```
$ from pytrf import date, sinex
$ from pytrf.io import read_solns

# Date of interest
$ t = date.from_wd(2124, 0.5).tsnx()
$ print(t)
> 20:264:43200

# Read list of discontinuities that comes with the reference frame
$ solns = read_solns('soln_IGc20.snx')

# Read post-seismic deformation models that come with the reference frame
$ psd = sinex.read('psd_IGc20.snx')

# Read reference frame into a sinex object, omitting blocks that are not needed in order to save time
$ ref = sinex.read('IGc20.snx', dont_read=['comments', 'metadata', 'matrices'])

# Remove from reference frame 'solns' that are not relevant at date t
$ ref.trim_solns(t, solns)

# Compute station positions at date t from positions at  $t_0$  + velocities
$ ref.propagate(t)

# Add post-seismic deformation models
$ ref.add_psd(psd)
```

pytrf.sinex – Prepare an IGS AC solution for combination (1/2)

```
$ from pytrf import date, sinex
$ from pytrf.io import read_solns, read_domes

# Read DOMES number catalogue
$ domes = read_domes('codomes_gps_coord.snx')

# Read 'master' list of discontinuities
$ solns = read_solns('soln.snx')

# Nominal span of AC solution
$ tstart = date.from_wd(2124, 0).tsnx()
$ t = date.from_wd(2124, 0.5).tsnx()
$ tend = date.from_wd(2124, 1).tsnx()

# Open log file
$ log = open('prep-grg.log', 'w')

# Read AC SINEX file
$ snx = sinex.read('GRG6RE3FIN_20202640000_01D_000_SOL.SNX', dont_read=['comments'])

# Check DOMES numbers and solns
$ snx.check_staid(domes, out=log)
$ snx.check_solns(solns, out=log)

# Check parameter reference epochs & SOLUTION/EPOCHS block
$ snx.check_epochs(tstart, tend, t)
```

pytrf.sinex – Prepare an IGS AC solution for combination (2/2)

```
# If needed, recover unconstrained normal equation. Else, just clear constraints.
$ if (snx.N is None):
$     snx.unconstrain()
$ else:
$     snx.clear_const()

# Fix some parameters to their a priori values
$ snx.fix_params(['UT', 'SATA', 'GC'])

# Remove (eliminate) reference frame information from normal equation
$ snx.del_helmerts('RST', 'STA')

# First inversion with NNRST constraints wrt to reference frame
$ snx.add_mc('RST', 'STA', sigma='auto', datum=ref, thr=2)
$ snx.neqinv(clear_neq=False)

# Iterative Helmert comparison between AC solution and reference frame
$ refc = ref.copy()
$ snx.compare_iter(refc, 'RST', weighting='identity', norm_res='approx', thr_raw=3, clean_ref=True, out=log)

# Final inversion with NNRST constraints wrt to clean reference frame
$ snx.clear_const()
$ snx.add_mc('RST', 'STA', sigma='auto', datum=refc, thr=2)
$ snx.neqinv(clear_neq=False)

# Close log file
$ log.close()
```

pytrf.sinex – Prepare an IGS AC solution for combination (2/2)

```
# If needed, recover unconstrained normal equation. Else, just clear constraints.
$ if (snx.N is None):
$     snx.unconstrain()
$ else:
$     snx.clear_const()

# Fix some parameters to their a priori values
$ snx.fix_params(['UT', 'SATA', 'GC'])

# Remove (eliminate) reference frame information from normal equation
$ snx.del_helmerts('RST', 'STA')

# First inversion with NNRST constraints wrt to reference frame
$ snx.add_mc('RST', 'STA', sigma='auto', datum=ref, thr=2)
$ snx.neqinv(clear_neq=False)

# Iterative Helmert comparison between AC solution and reference frame
$ refc = ref.copy()
$ snx.compare_iter(refc, 'RST', weighting='identity', norm_res='approx', thr_raw=3, clean_ref=True, out=log)

# Final inversion with NNRST constraints wrt to clean reference frame
$ snx.clear_const()
$ snx.add_mc('RST', 'STA', sigma='auto', datum=refc, thr=2)
$ snx.neqinv(clear_neq=False)

# Close log file
$ log.close()
```

This sets the tightness of datum definition constraints automatically, so that they are tight compared to the precision of the network geometry, but not too tight to avoid numerical issues.

This is a threshold for the selection of actual datum-defining stations among all available reference frame stations. Only 'well' determined stations are used. 'Loosely' determined stations are discarded.

Keeping the normal equation in the **sinex** object will save time during the combination. (No need to re-invert the covariance matrix.)

This is the threshold for the identification of outliers in the iterative comparison. Stations with residuals larger than 3 times the WRMS of residuals in any East/North/Up component are iteratively rejected.

pytrf.snxcmb – Combination of daily IGS AC solutions

- Step 1: Create a list of inputs for `snxcmb.combine_iter()`

- It can for instance be initialized from a YAML file:

```
$ from pytrf import date
$ from pytrf.io import read_yaml

$ t = date.from_wd(2124, 0.5)
$ inputs = read_yaml('inputs.yml', sed=True, t=t)
```

```
-
  name:      esa
  file:      $www/inputs/ESA3R03FIN_YYYY$doy0000_01D_01D_SOL.SNX
  description: Daily IGS AC SINEX solution
  params:    RST
-
  name:      gfz
  file:      $www/inputs/GFZ2R03FIN_YYYY$doy0000_01D_01D_SOL.SNX
  description: Daily IGS AC SINEX solution
  params:    RST
[...]
```

- Assign a `sinex` object to every input:

```
$ for ac in inputs:
$     ac.snex = sinex.read(ac.file, dont_read=['comments'])

#     Insert the whole pre-processing of the AC solution here
```

- Assign an a priori scaling factor to every input solution:

(here, such that the median 3D formal error of station positions becomes 4 mm in every input solution)

```
$ for ac in inputs:
$     senh = ac.snex.get_sighn()
$     s3d = np.sqrt(np.sum(senh**2, axis=1))
$     ac.sf = 0.004 / np.median(s3d)
```

pytrf.snxcmb – Combination of daily IGS AC solutions

- Step 2: Call `snxcmb.combine_iter()`

```
$ from pytrf.snxcmb import combine_iter
$ log = open('combine_iter.log', 'w')
$ combsnx = combine_iter(inputs,
                        t.tsnx(),
                        solns = solns,
                        check_solns = False,
                        datum = ref,
                        mc_sta = 'RST',
                        mc_sta_sig = 1e-5,
                        mc_sta_thr = 2,
                        update_sf = True,
                        thr_norm = 5,
                        flag_once = True,
                        clear_neq = False,
                        out = log)

# List of inputs
# Reference date
# Content of master discontinuity list
# Do not check solns in input solutions (again)
# Reference frame for alignment of combined solution
# Datum definition constraints for combined solution
# Sigma of datum definition constraints [m]
# Discard loosely determined stations from constraints
# Update input solution scaling factors at each iteration
# Reject stations with normalized residuals > 5
# Reject a given station from one input at a time
# Keep unconstrained normal equation in combsnx

$ log.close()
```

- At each iteration:

- AC scaling factors are updated (see equation on slide 81),
- Stations with normalized residuals > 5 along any East/North/Up component are rejected from the corresponding input solutions.
- If a station is flagged in more than one solution, it is rejected only from the solution in which it has the largest 3D normalized residual.

- Iterations stop when no more outliers are found and AC scaling factors have converged (to better than 0.1%).

pytrf.snxcmb – Combination of daily IGS AC solutions

- Step 3: Post-process combined solution

```
$ log = open('combine_iter.log', 'a')

# Eliminate transformation parameters
$ combsnx.del_params(['TRANS'])

# Iterative Helmert comparison with reference frame
$ combsnx.compare_iter(ref, 'RST', weighting='identity', norm_res='approx', thr_raw=3,
                       clean_ref=True, out=log)

# Remove previous datum definition constraints
$ combsnx.clear_const()

# Add datum definition constraints wrt clean reference frame
$ combsnx.add_mc('RST', 'STA', sigma=1e-5, datum=ref, thr=2)

# Final inversion
$ combsnx.neqinv()

$ log.close()
```

pytrf.snxcmb – Combination of daily IGS AC solutions

- Step 4: Print combination residuals & statistics in your preferred format

- Short example:

```
$ print('          WRMS          ')\n$ print(' AC      E[mm]      N[mm]      H[mm] ')\n$ print(' ----- ')\n$ for ac in inputs:\n$     print(' {0} {1[0]:9.3f} {1[1]:9.3f} {1[2]:9.3f}'.format(ac.name, ac.wrms))
```

AC	WRMS		
	E[mm]	N[mm]	H[mm]
esa	1.052	1.162	3.176
gfz	1.018	1.313	3.594
grg	1.528	1.222	3.741
jpl	1.285	1.237	3.798
mit	1.304	1.341	3.773
ngs	1.159	1.152	3.518
tug	1.050	1.061	3.103

- Other useful attributes set by `snxcmb.combine_iter()`:

- `ac.sf` Final AC scaling factor
- `ac.sigm` Median East/North/Up formal errors in rescaled AC solution
- `ac.v` Array of residuals (ordered in the same way as the parameters in `ac.snx`, but in mm and rotated to ENH frames for station positions)
- `ac.sv` Array of (approximate) formal errors of residuals
- `ac.vn` Array of normalized residuals (= `ac.v / ac.sv`)

- See more examples in `pytrf/examples/SINEX combination/1 - combination of daily IGS AC solutions`

- You can also draw maps of the combination residuals by calling, e.g.,:

```
ac.snx.map_res(ac.v, output=ac.name+'.png')
```

pytrf.snxcmb – Combination of daily IGS AC solutions

- Step 4: I

- Short c

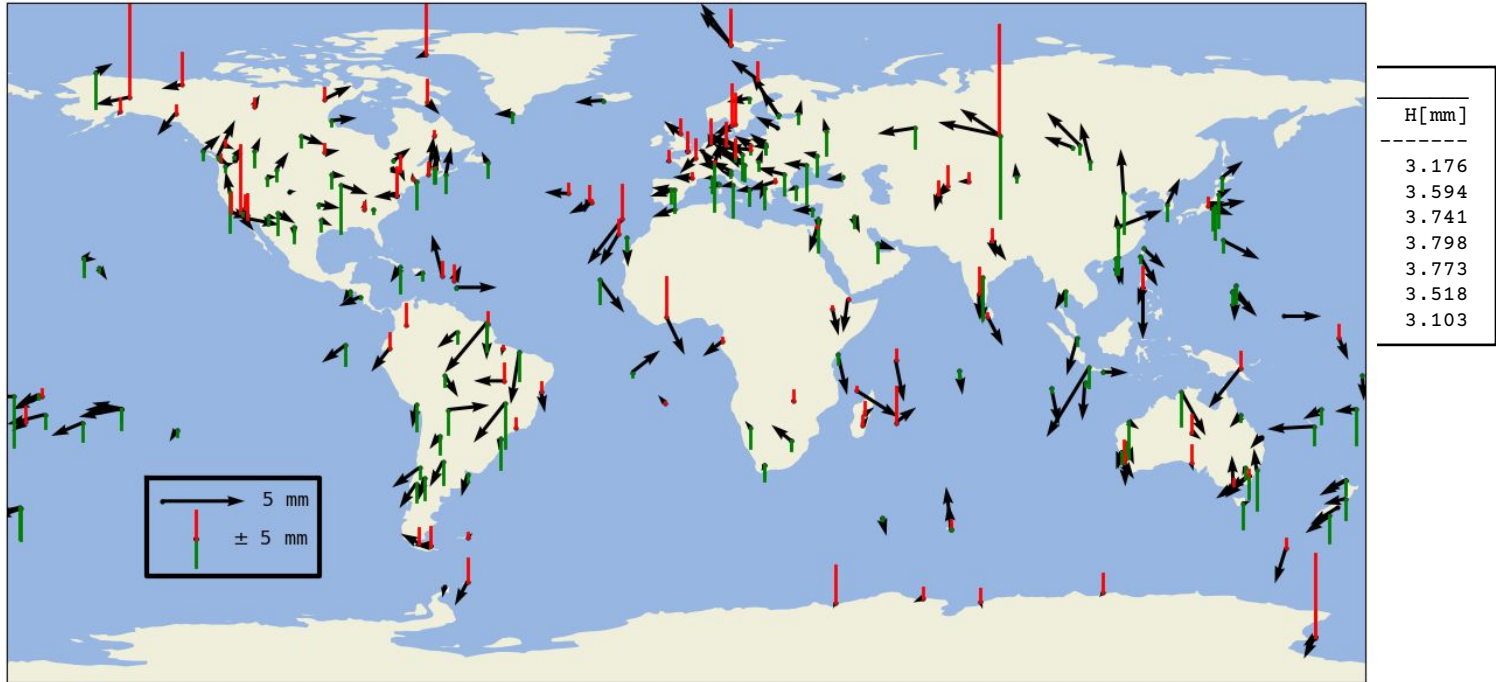
\$
\$
\$
\$
\$

- Other

–
–
–
–
–
–

- See m

- You can also draw maps of the combination residuals by calling, e.g.,:
`ac.snx.map_res(ac.v, output=ac.name+'.png')`



`pytrf.snxcmb` – Long-term stacking

- Philosophy:

1. Extract station position time series from your time series of SINEX solutions
2. Fit kinematic models to each station position time series using `pytrf.ts`
 - Identify and add possible missing discontinuities in your master discontinuity list
 - Identify outliers
3. Pre-process time series of SINEX solutions
(Make everything ready so that a single run of the stacking will be necessary.)
 - Check solns
 - Remove PSD models, if any
 - Remove previously identified outliers
4. Prepare the reference frame for the alignment of your long-term solution
5. Call `snxcmb.combine()` once. No iterations needed!
6. Post-process long-term solution
 - Refine alignment to the reference frame
7. Print combination residuals and statistics in your preferred format

pytrf.snxcmb – Long-term stacking

- Step 1: Extract station position time series

```
# Loop over input SINEX files
$ import glob
$ files = np.sort(glob.glob('inputs/*'))
$ for f in files:
$     snx = sinex.read(f)

# Loop over station positions in input SINEX file
$ for i in snx.ix:
$     t = date.from_tsnx(snx.param[i].tref).mjd
$     sta = snx.param[i].code
$     X = snx.x[i:i+3]
$     Q = snx.Q[i:i+3,i:i+3]

# Update position time series of current station
$ f = open('crd/'+sta+'.crd', 'a')
$ print('{0:7.1f} {1[0]:21.14e} {1[1]:21.14e} {1[2]:21.14e} {2[0][0]:21.14e} {2[0][1]:21.14e}
$       {2[0][2]:21.14e} {2[1][1]:21.14e} {2[1][2]:21.14e} {2[2][2]:21.14e}'.format(t, X, Q),
$       file=f)
$ f.close()
```

- To gain speed, the extraction can be parallelized over input SINEX files.
- In that case, be careful of sorting station position time series afterwards.
- See pytrf/examples/SINEX combination/3 - long-term stacking of daily IGS solutions

pytrf.snxcmb – Long-term stacking

- Step 2: Fit kinematic models to station position time series

- This is the opportunity to identify missing discontinuities in your master discontinuity list.
- Here, we assume no missing discontinuity and run this step just to identify outliers.

```
# Loop over station position time series
$ files = np.sort(glob.glob('crd/*'))
$ for f in files:
$     sta = f[4:8]

# Read time series
$ r = ts.read(f, format=('t','x','y','z','qx','qxy','qxz','qy','qyz','qz'),
$           dtrd=1, rotate=True)

# Create & fit model
$ m = model.from_solns(r, solns, code=sta, per=[365.25,182.625], noise=['vw'], psd=psd,
$           fix_amp=True, fix_tau=True)
$ m.fit_iter(thr_norm=5, quiet=True)
$ m.fit_iter(thr_norm=5, thr_mad=5, win_mad=365, quiet=True)
$ m.plot_fit(tunit='y', output='fig/'+sta+'-fit.png')
$ m.plot_res(tunit='y', output='fig/'+sta+'-res.png')

# Update list of outliers
$ for i in range(len(r.tdel)):
$     t = date.from_mjd(r.tdel[i])
$     with open('del/{0.week}/{0.dow}.del'.format(t), 'a') as fdel:
$         print(sta, file=fdel)
```

pytrf.snxcmb – Long-term stacking

- Step 2: Fit kinematic models to station position time series

- This is the opportunity to identify missing discontinuities in your master discontinuity list.
- Here, we assume no missing discontinuity and run this step just to identify outliers.

```
# Loop over station position time series
$ files = np.sort(glob.glob('crd/*'))
$ for f in files:
$     sta = f[4:8]

# Read time series
$ r = ts.read(f, format=('t','x','y','z','qx','qxy','qxz','qy','qyz','qz'),
$           dtrd=1, rotate=True)

# Create & fit model
$ m = model.from_solns(r, solns, code=sta, per=[365.25,182.625], noise=['vw'], psd=psd,
$           fix_amp=True, fix_tau=True)
$ m.fit_iter(thr_norm=5, quiet=True)
$ m.fit_iter(thr_norm=5, thr_mad=5, win_mad=365, quiet=True)
$ m.plot_fit(tunit='y', output='fig/'+sta+'-fit.png')
$ m.plot_res(tunit='y', output='fig/'+sta+'-res.png')

# Update list of outliers
$ for i in range(len(r.tdel)):
$     t = date.from_mjd(r.tdel[i])
$     with open('del/{0.week}{0.dow}.del'.format(t), 'a') as f:
$         print(sta, file=fdel)
```

To gain speed, this step can be parallelized over stations.
See [pytrf/examples/SINEX combination/3 - long-term stacking of daily IGS solutions](#)

To identify possible missing discontinuities, you would
need to visually inspect all time series plots.
You could also run likelihood ratio tests (see slide 56)
to identify the most likely missing discontinuities.

pytrf.snxcmb – Long-term stacking

- Step 3: Pre-process input SINEX solutions

```
# Loop over input SINEX files
$ files = np.sort(glob.glob('inputs/*'))
$ for f in files:
$     snx = sinex.read(f)

# Check solution numbers
$ snx.check_solns(solns, quiet=True)

# Delete parameters that shouldn't be stacked
$ snx.del_params(['ERP', 'GC'], keep_const=True)

# Delete previously identified station position outliers
$ t = date.from_tsnx(snx.param[0].tref)
$ if (os.path.isfile('del/{0.week}{0.dow}.del'.format(t))):
$     stadel = np.loadtxt('del/{0.week}{0.dow}.del'.format(t), dtype='O', ndmin=1)
$     snx.del_sta(stadel, keep_const=True)

# Remove PSD models
$ snx.add_psd(psd, remove=True, update_cov=False)

# Invert covariance matrix to save time during stacking
$ snx.N = invspd(snx.Q)
$ snx.Nc = None

# Dump pre-processed sinex object (in a pickle file!)
$ snx.dump('pkl/{0.week}{0.dow}.pkl'.format(t))
```

To gain speed, this step can be parallelized over input SINEX solutions. See [pytrf/examples/SINEX combination/3 - long-term stacking of daily IGS solutions](#)

pytrf.snxcmb – Long-term stacking

- Step 4: Prepare reference frame for the alignment of the long-term stacked solution

```
# Read reference frame SINEX file
$ ref = sinex.read('IGS20.sn timer', dont_read=['comments', 'metadata', 'matrices'])

# Keep only solns that are relevant a given date
$ t = date.from_ymdhms(2018, 7, 1).tsnx()
$ ref.trim_solns(t, solns)

# Propagate reference frame to reference date
# of the long-term stacked solution
$ ref.propagate(t, keep_vel=True)
```

When aligning a long-term solution to a reference frame, use only one soln per station.

Here, we select the solns valid at a given date.
Another possible choice would be to keep the soln with the longest observation span for each station.

Do not forget to keep station velocities in the propagated reference frame!

When aligning a long-term solution to a reference frame, it is absolutely necessary that station positions are expressed at the same reference epoch in both solutions.

pytrf.snxcmb – Long-term stacking

- Step 5: Call `snxcmb.combine()`

```
# Prepare list of inputs
$ from pytrf.utils import record
$ inputs = []
$ for w in range(1930, 2086):
$     for d in range(7):
$         r = record()
$         r.name = str(w)+str(d)
$         r.file = 'pkl/'+r.name+'.pkl'
$         r.description = 'IGS repro3 daily combined solution'
$         r.params = 'RST'
$         inputs.append(r)

# Call snxcmb.combine()
$ combsnx = combine(inputs, t, solns=solns,
                    check_solns = False,
                    set_vel = True,
                    dv_sig = 1e-6,
                    datum = ref,
                    mc_sta = 'RST', mc_sta_sig = 1e-5,
                    mc_vel = 'RST', mc_vel_sig = 1e-6,
                    norm_res = 'approx',
                    vce = 'approx',
                    store_inputs = False,
                    reduce_trans = True,
                    clear_neq = False)

# Do not check solns in input solutions (again)
# Estimate station velocities
# Sigma of equality constraints between successive velocities
# Reference frame
# Datum definition constraints
# Datum rate definition constraints
# Compute approximate normalized residuals
# Compute approximate variance factors for input solutions
# Do not store input solutions in RAM!!!
# Eliminate transformation parameters
# Keep unconstrained normal equation in combsnx
```

pytrf.snxcmb – Long-term stacking

- Step 5: Call `snxcmb.combine()`

```
# Prepare list of inputs
$ from pytrf.utils import record
$ inputs = []
$ for w in range(1930, 2086):
$     for d in range(7):
$         r = record()
$         r.name = str(w)+str(d)
$         r.file = 'pkl/'+r.name+'.pkl'
$         r.description = 'IGS repro3 daily combined solution'
$         r.params = 'RST'
$         inputs.append(r)

# Call snxcmb.combine()
$ combsnx = combine(inputs, t, solns=solns,
                  check_solns = False,
                  set_vel = True,
                  dv_sig = 1e-6,
                  datum = ref,
                  mc_sta = 'RST', mc_sta_sig = 1e-5,
                  mc_vel = 'RST', mc_vel_sig = 1e-6,
                  norm_res = 'approx',
                  vce = 'approx',
                  store_inputs = False,
                  reduce_trans = True,
                  clear_neq = False)
```

No need to specify a priori scaling factors for the input solutions. (Default is 1.)

By default, equality constraints are applied between the velocities of the successive solns of each station (except in case of velocity discontinuities).

Equality constraints between velocities of co-located stations could also be applied using the `vconst` argument.

```
# Do not check solns in input solutions (again)
# Estimate station velocities
# Sigma of equality constraints between successive velocities
# Reference frame
# Datum definition constraints
# Datum rate definition constraints
# Compute approximate normalized residuals
# Compute approximate variance factors for input solutions
# Do not store input solutions in RAM!!!
# Eliminate transformation parameters
# Keep unconstrained normal equation in combsnx
```

pytrf.snxcmb – Long-term stacking

- Step 6: Post-process long-term stacked solution

```
# Iterative comparison with reference frame
$ combsnx.compare_iter(ref, 'RST', weighting='identity', thr_raw=3, clean_ref=True)

# Final inversion with NNRST constraints applied wrt clean reference frame
$ combsnx.clear_const()
$ combsnx.add_mc('RST', 'STA', sigma=1e-5, datum=ref)
$ combsnx.add_mc('RST', 'VEL', sigma=1e-6, datum=ref)
$ combsnx.add_vc(solns, sigma=1e-6)
$ combsnx.neqinv()
```

pytrf.snxcmb – Long-term stacking

- Step 7: Print stacking residuals & statistics in your preferred format
 - See `pytrf/examples/SINEX combination/3` - long-term stacking of daily IGS solutions
 - Main useful attributes set by `snxcmb.combine()`:
 - `inputs[i].wrms` WRMS of East/North/Up residuals
 - `inputs[i].v` Array of residuals (ordered in the same way as the parameters in input solution but in mm and rotated to ENH frames for station positions)
 - `inputs[i].sv` Array of (approximate) formal errors of residuals
 - `inputs[i].vn` Array of normalized residuals (= `inputs[i].v / inputs[i].sv`)